

Algoritmi

Simone Petta

Secondo Semestre a.a. 2021/2022

Contents

1	Introduzione	7
1.1	Passi per la risoluzione di problemi	8
1.1.1	Formulazione	8
1.1.2	Soluzione	9
1.2	Cos'è un algoritmo	9
1.3	Strutture dati	10
1.3.1	ADT (Abstract Data Type)	10
2	Studio complessità	13
2.1	Studio complessità	14
2.1.1	Assunzioni	14
2.1.2	Fattori che influiscono sul tempo di esecuzione	14
2.1.3	Complessità temporale	15
2.1.4	Notazione asintotica	15
2.1.5	Notazioni asintotiche aggiuntive	18
2.1.6	Proprietà delle notazioni asintotiche	19
2.1.7	Scelta algoritmo	19
2.2	Casi Base di Complessità	20
2.3	Regola della somma	21
2.3.1	Dimostrazione	21
2.3.2	Esempio	22
2.4	Regola dei prodotti	23
2.4.1	Dimostrazione	23
2.5	Regole generali di calcolo della complessità	24
2.5.1	Esempio	24
2.6	Chiamate a funzione o procedura	25
2.7	Equazioni di ricorrenza	25
2.7.1	Esempio	26
2.7.2	Metodi risoluzione delle equazioni di ricorrenza	26
2.7.3	Metodo della sostituzione	27
2.7.4	Albero di ricorsione	27
2.7.5	Teorema dell'esperto (master theorem)	29

3	Liste	31
3.1	Lista	32
3.1.1	Operazioni	32
3.1.2	Implementazione come un array	32
3.1.3	Altre operazioni	34
3.1.4	Implementazione tramite puntatori	35
3.1.5	Vantaggi e svantaggi	36
4	Stack	37
4.1	Implementazione con array	38
5	Coda	41
5.1	Implementazione con i puntatori	42
5.1.1	ENQUEUE(x,Q)	42
5.1.2	DEQUEUE(Q)	43
5.2	Implementazione con array	43
6	Alberi	45
6.0.1	Terminologia	46
6.1	Implementazione come array	48
6.1.1	Implementazione figlio-sinistro, fratello-destro	49
6.1.2	Implementazione con lista dei figli	51
6.1.3	Implementazione con puntatori dei figli	52
6.2	Visita	52
6.2.1	Visite in profondita'	52
6.2.2	Visita in ampiezza	56
6.3	Alberi binari di ricerca (BST)	57
6.3.1	Ricerca	58
6.3.2	Operazioni comuni	59
6.3.3	Ricerca del successore di un nodo n	59
6.3.4	Inserimento in BST	60
6.3.5	Cancellazione in BST	61
6.3.6	Altezza di un albero BST	64
6.3.7	Bilanciamento perfetto	65
6.4	Alberi AVL	66
6.4.1	Rotazione Singola	67
6.4.2	Rotazione doppia	69
6.4.3	Inserimento di v in T	71
6.4.4	Cancellazione	71
6.4.5	Costo delle operazioni su AVL	71
6.5	B-TREE e B+TREE	72
6.5.1	Ricerca in B-TREE	73
6.5.2	Inserimento	74
6.5.3	Cancellazione	74

7	Insiemi	77
7.0.1	Implementazione come Bit-Vector	78
7.1	Implementazione come liste linkate	80
7.1.1	Intersezione	82
7.1.2	Unione	82
7.1.3	Differenza	82
7.1.4	MAX/MIN	83
7.1.5	Member	83
7.1.6	Codice Inserction	83
8	Dizionari	85
8.1	Hash	86
8.1.1	Open Hashing	87
8.1.2	Closed Hashing	89
8.1.3	Uniform Hashing	91
9	Grafi	97
9.1	Grado di un nodo	98
9.2	Cammino	99
9.2.1	Connessione	100
9.3	Sottografo	101
9.4	Implementazione dei Grafi	101
9.4.1	Matrice di adiacenza	101
9.5	Algoritmi di visita del grafo	103
9.5.1	BFS - Ampiezza	104
9.5.2	DFS - Profondita	106
9.6	Minimum Cost Spanning Tree (MST)	109
9.6.1	Kruskal	112
9.6.2	PRIM	114
9.7	Shortest Path	115
9.7.1	Varianti	115
9.7.2	Single source shortest path	116
9.7.3	Algoritmo di Dijkstra	117
9.7.4	Algoritmo di Bellman-Ford	121
10	Algoritmi di ordinamento	123
10.1	Bubble Sort	125
10.2	Insertion Sort	126
10.3	Selection sort	128
10.4	Merge Sort	130
10.4.1	Caso base	130
10.4.2	Ricombinazione	131
10.4.3	Costo	135
10.5	Heap Sort	135
10.5.1	Heap	135
10.5.2	Heap Sort	140

10.6 Quick Sort	144
10.6.1 Partition	145
10.6.2 Costo	148
10.6.3 Nota	150
10.7 Counting Sort	152

Chapter 1

Introduzione

Quando scriviamo un programma cerchiamo di scrivere una soluzione ad un problema che ci e' stato proposto, in modo automatizzato. Quali sono i passi da seguire se vogliamo operare nel modo migliore per risolvere un problema, non possiamo improvvisare cosi' dal nulla la risoluzione di problemi in modo strutturato cercando di ottenere una soluzione congrua al problema che abbiamo davanti e che possa andare bene per una famiglia di problemi simili a quello che dobbiamo risolvere.

1.1 Passi per la risoluzione di problemi

La risoluzione dei problemi segue una serie di passi:

- Formulare il problema: descrivere il problema in modo completo, dettagliato ed in modo informale non e' una cosa che accade sempre. Ma e' importante per noi definire bene il problema.
- Definizione della soluzione: ovviamente uno stesso problema puo' avere diversi modi per essere risolto, la nostra soluzione sara' la definizione di un algoritmo per risolvere quel problema.
- Implementare la soluzione: passo da una definizione astratta ad un implementazione concreta con il linguaggio di programmazione scelto, passiamo quindi da uno studio astratto e teorico ad una realizzazione piu' concreta
- Test: vado a testare la soluzione che ho trovato, per andare a correggere gli eventuali errori che ho fatto
- Valutazione: a posteriori una volta che la soluzione e' stata testata e distribuita nell'ambiente in cui verra' utilizzata posso valutare se la soluzione che ho definito e' buona.

Quello su cui ci concentreremo in questo corso e' la fase di definire la soluzione.

1.1.1 Formulazione

Cosa ci serve capire per poter definire un algoritmo e poter lavorare in modo concreto alla risoluzione di un problema, dobbiamo innanzitutto lavorare alla fase di formulazione, e' importante conoscere il problema in modo dettagliato per definire la soluzione dello stesso ed il problema dev'essere specificato in modo preciso e non ambiguo, non deve lasciare spazio a situazioni in cui mi ritrovo a non aver capito cosa mi e' stato chiesto. Un esempio tipico di questo e': mi e' stato chiesto di fare una torta molto buona, ma quello che puo' essere un dolce molto buono per me potrebbe non esserlo per qualcun altro, un problema formulato in questo modo lascia spazio ad ambiguita' ed interpretazione, a noi servono problemi specificati in modo preciso e non ambiguo. Per avere una formulazione precisa e non ambigua del problema sarebbe preferibile

sfruttare un modello matematico per formulare il problema stesso, perche' sappiamo che non e' ambiguo (in quanto tutti lo interpretano allo stesso modo), e' sicuramente molto preciso e perche' possiamo sfruttare le proprieta' del modello matematico per risolvere il problema e per valutarlo e studiarne le caratteristiche, ovviamente non e' sempre possibile sfruttare dei modelli matematici per formulare il problema ma ove possibile e' preferibile (e consigliato) farlo. Una volta formulato il problema disegno la soluzione.

1.1.2 Soluzione

Una volta che abbiamo espresso il problema nel modo piu' formale possibile possiamo cercare di definire una soluzione sfruttando il modello che abbiamo usato per definirlo. Quello che studieremo in questo corso e' come fare a trovare una soluzione in forma di algoritmo, il mio obbiettivo e' trovare la risoluzione al problema in termini di un algoritmo che lo risolve.

1.2 Cos'e' un algoritmo

Gli algoritmi possono essere studiati in modo formale per valutare la loro correttezza per andare a risolvere un problema. Dobbiamo andare a capire cos'e' un algoritmo. L'algoritmo e' una sequenza finita di istruzioni ciascuna delle quali ha un preciso significato (quindi non e' ambigua) e puo' essere eseguita in tempo finito. Come conseguenza un algoritmo termina sempre (qualsiasi sia l'input). Un'algoritmo spesso viene visto come un modo per risolvere un problema, possiamo pensare ad un algoritmo corretto rispetto ad un problema se per qualsiasi input dato dal problema l'algoritmo produce in output una soluzione corretta. Possiamo quindi dire che un algoritmo e' corretto se risolve il problema.

E' importante distinguere di un problema:

- La formulazione: che e' la definizione di un problema indipendentemente dal caso specifico (i dati in input); ad esempio: l'ordinamento di una sequenza di numeri interi
- L'istanza del problema: una specifica istanza del problema data dall'input; Input: 31, 41, 56, 29, 41, 58; Output: 29, 31, 41, 41, 56, 58. Esistono molti algoritmi che risolvono il problema di ordinare una sequenza di numeri interi come:
 - Bubble sort
 - Quick sort

Attenzione che l'algoritmo e' corretto per tutte le istanze del problema e deve essere finito.

1.3 Strutture dati

Tutti gli algoritmi lavorano su dei dati (input), operando su questi dati trasformandoli (producono anche risultati intermedi) e producendo un output come risultato delle operazioni. E' importante che questi dati siano organizzati in modo opportuno, per creare algoritmi migliori e affinché i dati non si perdano.

1.3.1 ADT (Abstract Data Type)

Un ADT non ha un corrispondente uno a uno con la realizzazione a livello pratico in un linguaggio di programmazione, e' un modello che mi permette di rappresentare un tipo di dato con tutte le operazioni che lo riguardano, come una lista: che e' un insieme di valori tutti dello stesso tipo piu' tutte le operazioni che operano su questo tipo di dato (inserimento di un valore, trova l'ultimo elemento, etc). In qualche modo puo' essere visto come una classe, una parte della classe sono i dati e l'altra parte sono le operazioni che operano sui dati.

E' un modello formale che modella dei dati e le operazioni definite su di essi.

Esempi:

- Liste
- Code
- Alberi

Una cosa molto importante e' che il tipo di dato astratto e' indipendente dall'implementazione, non ci preoccupiamo come verra' implementato in pratica, potrebbero anche esserci molte implementazioni del dato astratto in base a cosa mi serve. Ma tutto quello che lavora con il tipo di dato astratto non dipende dall'implementazione a basso livello del dato astratto, posso quindi cambiare l'implementazione senza dover cambiare l'algoritmo. Ho tutte le operazioni insieme ai dati su cui operano ed e' un vantaggio. Possono essere pensate come tipi base che abbiamo definito noi a nostro piacimento. Lo svantaggio e' che se ho diverse ADT che fanno la stessa operazione la devo definire piu' volte, perche' ognuna fa l'operazione a modo suo.

Implementazione delle ADT

Per l'implementazione delle ADT servono due cose:

- **Data Type:** un tipo di dato base che posso associare ad una variabile che uso per implementare una certa ADT. Definisce il dominio (i valori che una variabile puo' assumere) di una variabile, ovviamente cambia in base al linguaggio. Ad esempio:
 - interi
 - caratteri
 - booleani

- **Data Structure:** e' una collezione di variabili dello stesso tipo o diverso che sono tra loro collegate (vengono considerate come un elemento unico) e vengono utilizzate per rappresentare la parte di dati degli ADT. All'interno delle strutture dati ci sono delle celle (building block), che sono le singole variabili che le compongono (come ad esempio gli elementi di un array o di un record). Queste celle possono anche contenere riferimenti ad un'altra variabile, e quindi posso pensare come dei puntatori. Quelli che utilizzeremo saranno:
 - Array: collezione di variabili tutte dello stesso tipo (le celle sono tutte dello stesso tipo). Ha una dimensione fissa.
 - Record: e' meno frequente ed utilizzato nei linguaggi moderni, e' sempre una collezione di variabili (celle/campi) pero' sono di diverso tipo. Un tipico esempio e' un record che contiene il nome, il cognome, e l'eta' di una persona. Che sono due stringhe ed un numero.

Posso usare delle variabili che mi permettono di accedere al contenuto di una cella che non e' altro che un riferimento all'indirizzo della cella a cui voglio accedere, c'e' l'uso dei puntatori (come in C) che servono per l'implementazione di varie strutture dati perche' tramite i puntatori posso accedere direttamente alla zona di memoria dove i dati sono memorizzati. Un puntatore lo intendiamo come una variabile il cui contenuto e' l'indirizzo di accesso ad una cella nella struttura dati o in generale ad una variabile. Di solito li denotiamo con una freccia tra le celle.

Chapter 2

Studio complessita'

2.1 Studio complessita'

Ci sono alcuni aspetti che vanno tenuti in considerazione quando devo risolvere un problema per scegliere l'algoritmo migliore per risolverlo. Dato un problema esistono diversi algoritmi che lo risolvono, il problema e': qualche scegliere?

- E' importante che l'algoritmo scritto sia comprensibile e facile da implementare, non soltanto da me ma anche da chi lo dovra' riutilizzare.
- Che utilizzi bene le risorse, le risorse prese in considerazione sono:
 - il tempo: quanto tempo ci impiega l'algoritmo ad essere eseguito
 - lo spazio: quanto spazio occupa in memoria

Noi studieremo il tempo, anche perche' lo spazio ormai non e' molto rilevante, in quanto ne abbiamo tantissimo.

La risorsa piu' importante e' il tempo, noi vogliamo fare un calcolo che sia indipendente da macchina, linguaggio, compilatore e tutte le altre variabili che possono influire ma non sono strettamente legate al design dell'algoritmo. Quello che si fa nell'analisi degli algoritmi e' quello di calcolare la complessita' temporale in base alle istruzioni di base deve svolgere l'algoritmo per terminare la sua esecuzione.

2.1.1 Assunzioni

Noi consideriamo che l'algoritmo viene eseguito su una macchina RAM tradizionale (1 processore ed 1 zona di memoria) e non consideriamo una situazione di parallelismo, significa che viene svolta una istruzione alla volta.

2.1.2 Fattori che influiscono sul tempo di esecuzione

1. Dimensione dell'input
2. Qualita' del codice
3. Istruzioni di base disponibili
4. Complessita' dell'algoritmo

Gli elementi che ci interessano di piu' sono il modo in cui e' scritto l'algoritmo stesso (cicli, if, elementi di base) che sono quelli che mi danno la complessita', ma devono essere espressi in base alla dimensione dell'input.

Quindi ora data la dimensione dell'input scriveremo una funzione che ci scrive la complessita' temporale dell'algoritmo ($T(n)$). Voglio scrivere una funzione data la dimensione dell'input N mi esprima in funzione di N la complessita'.

2.1.3 Complessita' temporale

Denotiamo la complessita' temporale di un algoritmo come $T(n)$ che e' una funzione espressa in termini della dimensione n dell'input. Che puo' essere pensata come il numero di istruzioni di base eseguite dall'algoritmo. Abbiamo detto che $T(n)$ e' espressa in base alla dimensione dell'input, ma nella maggior parte dei casi anche il valore dell'input puo' cambiare la complessita' dell'algoritmo, non posso ovviamente provare tutti gli input, quello che si fa tipicamente e' considerare il caso peggiore (il caso in cui l'algoritmo prende il rampo piu' complesso). Si studiano anche il caso migliore ed il caso medio, quello medio e' difficile da realizzare preche' e' difficile da individuare il caso medio, noi considereremo sempre il caso peggiore. Il costo dipende anche dal valore dell'input e non soltanto dalla sua dimensione, quello che noi facciamo quindi e' valutare il caso peggiore perche' se funziona bene allora andra' bene a maggior ragione andra' bene anche negli altri casi. Lo studio di complessita' viene fatto per input di valore crescente, non mi interessa come si comporta l'algoritmo per input piccoli, mi interessa sapere quando la dimensione dell'input cresce tanto, si parla di complessita' asintotica, guardo quindi per $T(n)$ quando n e' grande. Tant'e' che se so che n e' piccolo fare lo studio di complessita' non mi interessa neanche, mi interessa quando n e' grande, perche' gli algoritmi devono essere scalabili. Mi interessa quindi quanto (come) cresce il tempo all'aumentare della dimensione dell'input (con N grande). Se un algoritmo domina un'altro in termini di complessita' computazionale con n crescente lo preferisco.

2.1.4 Notazione asintotica

Si usano delle notazioni specifiche che mi permettono di rappresentare insieme delle famiglie di complessita', non una funzione espressa in modo specifico con tanti termini (ma con un solo termine). Mi server per esprimere in termini dell'input la complessita' del mio problema, esprime il tempo di esecuzione di un algoritmo in funzione della dimensione dell'input ($T(n)$).

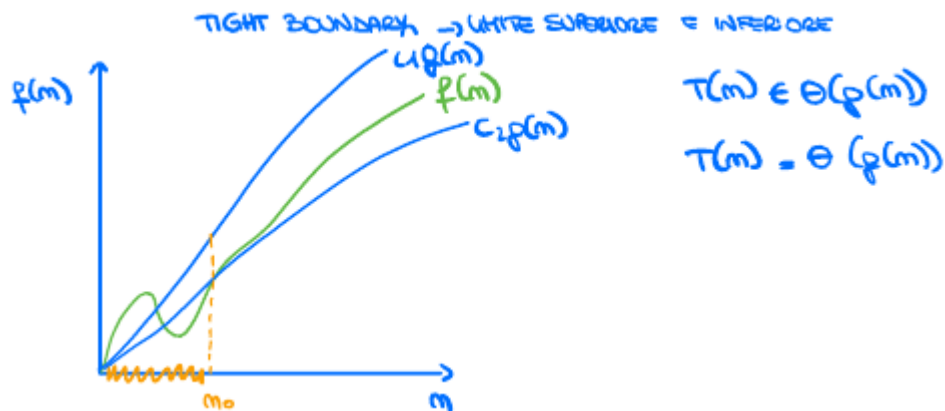
Abbiamo 3 notazioni asintotiche che si usano solitamente (i tipi di limite che si possono porre alla complessita').

$\Theta(g(n))$ (teta grande)

Mi da un limite superiore ed inferiore alla complessita' dell'algoritmo sulla base di una funzione di riferimento. Posso dire che l'andamento della funzione e' nell'ordine della funzione $g(n)$ a meno di costanti moltiplicative. E' la famiglia di $f(n)$ tale per cui esistono costanti c_1 e c_2 tale per cui vale una disuguaglianza che mi dice che $f(n)$ e' compresa tra $c_1g(n)$ e $c_2g(n)$, positivi e dev'essere maggiore per ogni $n \geq 0$. A partire da un certo numero n_0 so che la funzione si comporta come $c_1g(n)$ e $c_2g(n)$.

$$\Theta(g(n)) = f(n) : \exists c_1, c_2, n_0 > 0 | 0 \leq c_1g(n) \leq f(n) \leq c_2g(n), \forall n \geq n_0 \quad (2.1)$$

Si dice che il tetaGrande mi da un tight boundary, ovvero un limite superiore ed inferiore alla complessita' del mio algoritmo.

Figure 2.1: $\Theta(n)$

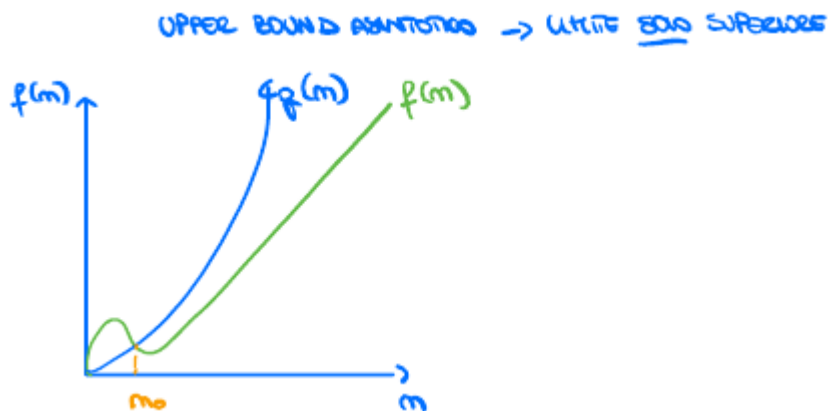
Posso dire che il mio algoritmo $T(n)$ appartiene o e' uguale a $\Theta(g(n))$

$O(g(n))$ (O Grande)

Mi da' il limite superiore (che e' il caso peggiore). $F(n)$ e' tale per cui esiste una costante c (ed n_0) tale per cui ho la funzione $F(n)$ limitata superiormente a partire da n_0 dalla funzione $g(n)$ moltiplicata per la costante c . Superato n_0 so che $f(n)$ e' dello stesso ordine di grandezza di $g(n)$.

$$O(g(n)) = f(n) : \exists c, n_0 > 0 \mid 0 \leq f(n) \leq cg(n), \forall n \geq n_0 \quad (2.2)$$

In questo caso si parla di upper bound asintotico che e' il solo limite superiore, so quindi che sopra quel valore non ci vado mai.

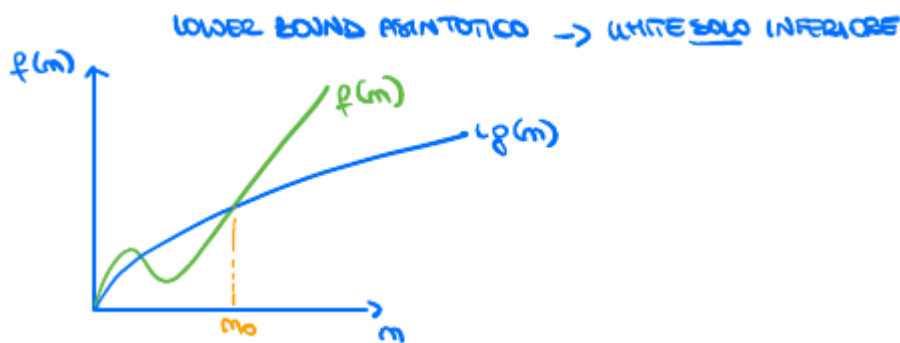
Figure 2.2: $O(n)$

$\Omega(g(n))$ (Omega Grande)

Mi da il limite inferiore. $f(n)$ e' tale per cui esiste una costante c (ed n_0) tale per cui ho la funzione $f(n)$ limitata inferiormente a partire da n_0 dalla $g(n)$ moltiplicata per la costante c .

$$\Omega(g(n)) = f(n) : \exists c, n_0 > 0 | 0 \leq c \cdot g(n) \leq f(n), \forall n \geq n_0 \quad (2.3)$$

Si dice lower bound asintotico che e' il limite inferiore a partire da n_0 , so quindi che sotto non ci vado mai.

Figure 2.3: $\Omega(n)$ **Proprieta'**

- So che la $f(n)$ non e' mai negativa
- Le costanti moltiplicative davanti alla notazione asintotica vengono eliminate (assorbite all'interno della classe)
- $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$
- Le notazioni asintotiche salgono nel caso peggiore, medio e migliore

ESERCIZIO - $f(m) = \frac{1}{2}m^2 - 3m$? $\Theta(m^2)$ $g(m) = m^2$

$$0 \leq C_1 m^2 \leq \frac{1}{2}m^2 - 3m \leq C_2 m^2 \quad \forall m \geq m_0$$

$$C_1 \leq \frac{1}{2} - \frac{3}{m} \leq C_2 \quad \forall m \geq m_0$$

$$C_1 \leq \frac{1}{2} - \frac{3}{m} \quad C_2 \geq \frac{1}{2} - \frac{3}{m} \quad \frac{3}{m} > 0 \quad \frac{1}{2} - \frac{3}{m} \leq \frac{1}{2}$$

$$C_1 = \frac{1}{2} - \frac{3}{m} \Rightarrow m=6 \quad 0 \quad C_2 = \frac{1}{2} \quad \forall m \geq 1 \quad \checkmark$$

$$C_1 = \frac{1}{14} \quad m=7 \quad > 0$$

$$m=8 \quad > 0 \quad \nearrow$$

$$m=9 \quad > 0 \quad \nearrow$$

$$m_0 = 7 \quad \checkmark$$

$$\exists C_1 = \frac{1}{14} \quad C_2 = \frac{1}{2} \quad m_0 = 7 \Rightarrow f(m) = \Theta(m^2)$$

2.1.5 Notazioni asintotiche aggiuntive

Esistono delle notazioni asintotiche aggiuntive che noi non useremo quasi mai in quanto sono più difficili da utilizzare.

$o(g(n))$

E' la famiglia di $g(n)$ tale per cui per ogni $c > 0$ esiste una costante n_0 positiva tale per cui $f(n) \leq cg(n)$. La differenza con l'O e' che ho per ogni c .

$$o(g(n)) = f(n) : \forall c > 0 \exists \text{costante } n_0 > 0 | 0 \leq f(n) \leq c \cdot g(n), \forall n \geq n_0 \quad (2.4)$$

Es: $s \cdot n = o(n^2)$ ma non e' vero che $(2n)^2 = o(n^2)$

$\omega(g(n))$

E' la famiglia di funzione $f(n)$ tale per cui per ogni costante c maggiore di 0 esiste una costante $n_0 > 0$ abbiamo $0 < c \cdot g(n) < f(n)$

$$\omega(g(n)) = f(n) : \forall c > 0 \exists \text{costante } n_0 > 0 | 0 \leq c \cdot g(n) \leq f(n), \forall n \geq n_0 \quad (2.5)$$

2.1.6 Proprieta' delle notazioni asintotiche

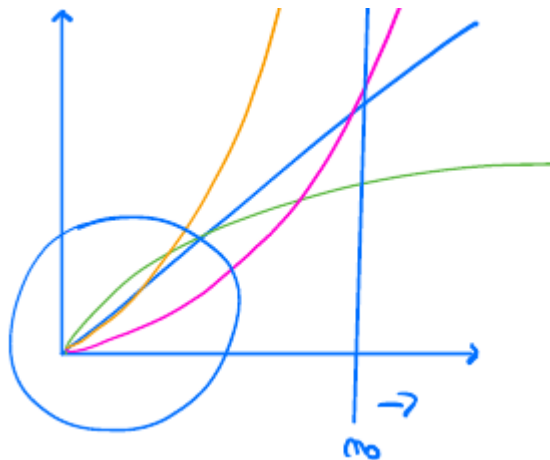
- Transitiva, nel senso che se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ allora $f(n) = \Theta(h(n))$. Lo stesso vale anche per (O e Ω).
- Riflessivita':
 - $f(n) = \Theta(f(n))$
 - $f(n) = \Omega(f(n))$
 - $f(n) = O(f(n))$
- Simmetria:
 - $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$
 - $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
 - $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$

La relazione d'ordine fra le funzioni di complessita' data dalle notazioni asintotiche stabilisce un ordine parziale (significa che potrebbero esserci coppie di funzioni che non posso confrontare).

2.1.7 Scelta algoritmo

- Considero il caso peggiore (perche' e' il caso che ci pone il limite superiore)
- Considero l'O (perche' ancora vado a guardare il costo piu' alto possibile che io posso raggiungere)

Esempio del grafico, preferisco l'algoritmo il cui costo cresce piu' lentamente rispetto all'O (grande), che vuol dire in termini pratici che ha l'O (grande) piu' piccolo.



2.2 Casi Base di Complessita'

Le classi piu' importanti di complessita' in ordine crescente sono:

- Costo costante ($O(1)$): quello che succede e' che il costo dell'algoritmo non dipende dalla dimensione dell'input, quello che vado a fare e' sempre fisso (ad esempio l'inserimento di un elemento in una lista), quindi il costo e' costante. Meno di cosi' non e' possibile, le costanti ricordiamo che non le teniamo in considerazione.
- $\log N$, costo logaritmico: in sostanza riesco a risolvere il problema andando a vedere soltanto una parte dell'input, perche' ci sono delle caratteristiche della struttura dati o dell'input che me lo permettono di fare. L'esempio tipico e' la ricerca in un albero binario che mi permette di sfruttare le caratteristiche dell'albero per fare la ricerca e non guardare tutti i nodi dell'albero.
- N , costo lineare: il costo e' lineare nella dimensione dell'input, in sostanza vado a fare una piccola quantita' di lavoro su ciascuno degli elementi dati in input, e' un costo relativamente basso (ambito piu' che accettabile) ma stiamo andando man mano crescendo. Esempio: scorrimento di una lista per leggere tutti gli elementi. Quindi chiaramente piu' la lista e' lunga piu' ci metto, ma su ciascun elemento faccio la stessa operazione (lettura).
- $N \log N$, Costo polilogaritmico: appare frequentemente perche' problemi che possono essere decomposti in sottoproblemi piu' piccoli e poi per risolvere il problema ricomponendo le soluzioni dei problemi piu' piccoli, hanno questo livello di complessita'. In generale i problemi che possono essere risolti in modo ricorsivo, quindi e' come se pagassi un po' per scomporre i sottoproblemi ed un po' per ricomporli. Esempio: Merge Sort, Heap Sort.

- N^2 (N^i), complessita' computazionale classica, costo quadratico e cosi' via (N elevato a qualcosa di costante che non dipende dall'input): la complessita' e' chiamata complessita' polinomiale, in questo caso e' come se l'algoritmo andasse a guardare tutte le coppie, terne etc dell'input e poi lavora sulle coppie, terne etc.
- 2^N (2^i), costo esponenziale: la dimensione dell'input e' all'esponente rispetto ad un valore costante che non dipende dall'input. Sto andando a processare tutte le possibili combinazioni dell'input, e' come se stessi processando tutte le possibili soluzioni dato l'input per vedere quella corretta. E' la classe piu' grande ed e' quella che preferirei evitare in quanto appena la dimensione dell'input diventa grande rispetto alle attese, il costo computazionale diventa talmente alto da non essere piu' risolvibile in termini pratici. Un algoritmo con questo costo computazionale e' un difficilmente scalabile nel mondo reale. I principali problemi con costo esponenziale sono quelli che finiscono nei problemi di ottimizzazione (trovare un percorso veloce per arrivare da qualche parte senza ripercorrere il problema), nonostante cio' e' possibile trovare un algoritmo meno furbo ma con costo computazionale minore.

2.3 Regola della somma

Quando andremo a calcolare la complessita' degli algoritmi dovremo andare a valutare la complessita' di ogni riga di istruzione e poi dovremo fare la somma della complessita' di ogni pezzo di codice. Cosa vuol dire quindi sommare (o moltiplicare) le complessita' computazionali?

Ho due algoritmi di cui conosco la complessita' computazionale A_1 con complessita' $O(f(n))$ ed un secondo algoritmo A_2 con complessita' $O(g(n))$, quanto costa eseguirli in sequenza?

$$O(f(n)) + O(g(n)) = O(\text{MAX}(f(n), g(n))) \quad (2.6)$$

La regola della somma ci dice (controintuitivamente) che la somma di due complessita' e' l'O del massimo tra le due funzioni, quello che si fa e' vedere qual'e' la funzione che domina l'altra a $+\infty$. In pratica se un'algoritmo e' composto da diverse parti, la sua complessita' e' data dal costo della parte piu' costosa.

2.3.1 Dimostrazione

Inizialmente riscriviamo cosa significa che le funzioni sono O:

$$\begin{aligned} A_1 \in O(f(n)) &\rightarrow \exists c_1, n_1 : A_1(n) \leq c_1 \cdot f(n) \forall N \geq n_1 \\ A_2 \in O(g(n)) &\rightarrow \exists c_2, n_2 : A_2(n) \leq c_2 \cdot g(n) \forall N \geq n_2 \end{aligned} \quad (2.7)$$

Ora dato che stiamo calcolando la somma dei due, semplicemente andiamo a sommare elemento per elemento nella disequazione:

$$\begin{aligned} A_1(n) + A_2(n) &\leq c_1 f(n) + c_2 g(n) \forall n \geq \text{MAX}(n_1, n_2) \\ &\leq c_1(\text{MAX} f(n), g(n)) + c_2(\text{MAX}(f(n), g(n))) \\ &= (c_1 + c_2)(\text{MAX}(f(n), g(n))) \end{aligned} \quad (2.8)$$

In conclusione se chiamiamo $(c_1 + c_2) = c_3$ abbiamo dimostrato per induzione l'ipotesi ovvero:

$$\exists c_3 = c_1 + c_2 | A_1(n) + A_2(n) \leq c_3 \text{MAX}(f(n), g(n)), \forall n \geq n_3, n_3 = \text{MAX}(n_1, n_2) \quad (2.9)$$

Quello che ha scritto in fondo non e' altro che la definizione di O del massimo tra $f(n)$ e $g(n)$ ed e' esattamente la definizione dell'ipotesi, quindi abbiamo dimostrato quella che era l'ipotesi iniziale, non si fa la somma ma si guarda quale funzione e' piu' grande dell'altra. Quello che guadagniamo e' che se l'algoritmo e' composto da piu' parti, la complessita' computazionale di tutto l'algoritmo e' data dal costo della parte piu' costosa (quella con costo computazionale piu' elevato).

Questa proprieta' vale allo stesso modo anche per Omega e Theta:

- $f = \Theta(g), l = \Theta(h) \Rightarrow f + l = \Theta(g + h) = \Theta(\text{MAX}(g, h))$
- $f = \Omega(g), l = \Omega(h) \Rightarrow f + l = \Omega(g + h) = \Omega(\text{MAX}(g, h))$

2.3.2 Esempio

Vediamo degli esempi.

Se $A_1 = O(2^n)$ ed $A_2 = O(n)$ allora $A_1 + A_2 = O(2^n)$. Aggiungendo all'equazione $A_3 = O(\log n)$ allora $A_1 + A_2 + A_3 = O(2^n)$

Vediamo un altro esempio un po' piu' complicato, definiamo

$$A_1 = \begin{cases} O(1) & \text{se } n \text{ PARI} \\ O(n^2) & \text{se } n \text{ DISPARI} \end{cases}$$

e

$$A_2 = \begin{cases} O(n) & \text{se } n \text{ PARI} \\ O(1) & \text{se } n \text{ DISPARI} \end{cases}$$

La somma $A_1 + A_2$ e' $O(n)$ se non consideriamo la distinzione tra pari e dispari (perche' prendiamo il caso peggiore tra tutti. Mentre e'

$$A_1 + A_2 = \begin{cases} O(n) & \text{se } n \text{ PARI} \\ O(n^2) & \text{se } n \text{ DISPARI} \end{cases}$$

se distinguiamo tra input pari e dispari.

Nell'ultimo la consegna e' che ho due algoritmi con due casistiche separate in base all'input, e se non dobbiamo distinguere i due casi prendiamo il caso peggiore.

2.4 Regola dei prodotti

Si applica quando abbiamo dei cicli o delle chiamate a funzioni.

Ho due algoritmi di cui conosco la complessita' computazionale A_1 con complessita' $O(f(n))$ ed un secondo algoritmo A_2 con complessita' $O(g(n))$. Come faccio a stimare quante volte devo eseguire un ciclo e quante volte viene eseguito un ciclo, devo quindi moltiplicare le notazioni asintotiche. Quindi dobbiamo capire cosa vuol dire fare $O(f(n)) \cdot O(g(n))$ ed e' semplicemente il prodotto, ovvero se ho $O(n) \times O(n)$ viene $O(n)$ al quadrato:

$$A_1 \cdot A_2 = O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n)) \quad (2.10)$$

2.4.1 Dimostrazione

Come fatto per la dimostrazione della regola della somma inizialmente esprimo cosa significa che le funzioni sono degli O:

$$\begin{aligned} A_1 \in O(f(n)) &\rightarrow \exists c_1, n_1 : A_1(n) \leq c_1 \cdot f(n) \forall n \geq n_1 \\ A_2 \in O(g(n)) &\rightarrow \exists c_2, n_2 : A_2(n) \leq c_2 \cdot g(n) \forall n \geq n_2 \end{aligned} \quad (2.11)$$

E come fatto prima cerco di dimostrare l'ipotesi per induzione, quindi nella pratica quello che vado a fare, dato che devo moltiplicare, e' moltiplicare tra di loro i termini delle due disequazioni:

$$A_1(n) \cdot A_2(n) \leq c_1 f(n) \cdot g(n), \forall n \geq \text{MAX}(n_1, n_2) \quad (2.12)$$

Ed ancora una volta se chiamo $c_1 \cdot c_2 = c_3$ ottengo esattamente l'ipotesi, che quindi abbiamo dimostrato:

$$\exists c_3 = c_1 \cdot c_2 : A_1(n) \cdot A_2(n) \leq c_3 \cdot (f(n) \cdot g(n)), \forall n \geq n_3, n_3 = \text{MAX}(n_1, n_2) \quad (2.13)$$

Ancora una volta quello che ottengo e' esattamente l'O di $f(n) \cdot g(n)$.

Anche in questo caso si applica per Omega e Theta:

- $f = \Theta(g), l = \Theta(h) \Rightarrow f \cdot l = \Theta(g \cdot h)$
- $f = \Omega(g), l = \Omega(h) \Rightarrow f \cdot l = \Omega(g \cdot h)$

Questa regola ci conferma che le costanti sono trascurabili, perche' posso includere qualsiasi costante nella c_3 .

2.5 Regole generali di calcolo della complessita'

Consideriamo per adesso algoritmi non ricorsivi. Abbiamo diverse situazioni che possiamo trovare, ma ci interessa valutare:

- Operazioni di base (somme, sottrazioni, assegnamento, letture etc dei tipi base): hanno tutte costo costante ($O(1)$).
- Sequenza di operazioni con costo noto: calcoliamo il costo facendo la somma, in pratica prendiamo il costo massimo.
- Valutazione di condizioni con diversi rami (costrutto if): valuto la condizione all'interno dell'if (che di solito e' $O(1)$) e poi, dato che sto valutando il caso peggiore, sommo il costo computazionale del ramo else con il then. In pratica prendo il peggiore tra il ramo else ed il then.
- Ciclo: abbiamo due modi per valutare il costo:
 - Si calcola il prodotto tra il numero di iterazioni eseguite ed il costo del caso peggiore di esecuzione delle istruzioni del ciclo.
 - Se voglio essere preciso dovrei valutare iterazione per iterazione e fare la somma di tutti i costi delle operazioni eseguite all'interno di un iterazione, ricordandoci sempre di valutare anche il costo della valutazione di uscita dal ciclo, perche' potrebbe essere che abbiano costo diverso.

Posso stimare il numero di iterazioni

2.5.1 Esempio

Vediamo un esempio di calcolo di costo computazionale su un algoritmo che vedremo successivamente, il Bubble Sort: prende in input un vettore di elementi A

```
for i=1 to a.length -1 do
  for j = a.length to i+1 do
    if A[j] < A[j-1]
      then t := A[j-1]
          A[j-1] := A[j]
          A[j] := T
```

Il ciclo interno parte dall'ultimo elemento e se e' piu' piccolo di quello precedente li scambia, si chiama bubble sort perche' e' come se facessimo salire gli elementi piu' piccoli (come se fossero delle bolle che salgono verso la superficie). Quando aumenta di uno il ciclo esterno e' perche' la posizione precedente del ciclo e' come se fosse bloccato, perche' ho portato su l'elemento piu' piccolo.

Valutiamo la complessita': parto dalle istruzioni di base all'interno e mano a mano vado verso l'esterno, nel nostro caso partiamo dalle istruzioni dentro il THEN, che sono tutte istruzioni base che valgono $O(1)$ e la somma tra di loro

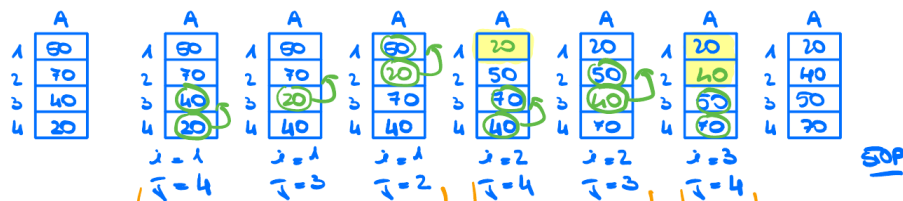


Figure 2.4: Esempio Bubble Sort

e' sempre $O(1)$, di conseguenza il ramo THEN ha complessita $O(1)$. A questo punto guardiamo l'if, il costo della condizione dell'if e' un confronto tra rami base quindi costa $O(1)$, il ramo THEN abbiamo detto costa $O(1)$ quindi la somma e' sempre $O(1)$ (nel caso in cui ci fosse stato anche il ramo ELSE si sarebbe dovuta sommare anche il costo di quest'ultimo). Passiamo al ciclo for, il ciclo esterno (diciamo che la lunghezza del vettore e' n) viene eseguito $n-1$ volte (iterazioni), il for interno invece viene eseguito $n-i$ volte (iterazioni). Se facciamo una valutazione spannometrica $n-i$ nel caso peggiore sara' $n-1$ quindi sarebbe come dire $n-1$ iterazioni di un ciclo che costa $O(n)$, quindi e' come dire $O(n) \cdot O(n) = O(n)^2$ ed e' la valutazione spannometrica del ciclo (notiamo come il -1 non viene considerato nella moltiplicazione). La valutazione precisa del ciclo: $O(n-1) + O(n-2) \dots O(1) = \sum_{i=1}^{n-1} (n-i) = O(n)^2$. Basta lasciare la sommatoria, non c'e' bisogno di svilupparla.

2.6 Chiamate a funzione o procedura

Per quanto riguarda le chiamate a funzioni non ricorsive l'unica cosa che facciamo e' calcolare il costo di quella funzione e sommarlo al costo del chiamante. La regola generale e' partire dalla funzione chiamata.

Cosa succede quando la funzione e' ricorsiva? Non possiamo usare lo stesso approccio perche' il chiamante e chiamato sono la stessa cosa, ci serve quindi esprimere il costo computazionale in modo ricorsivo utilizzando degli strumenti. Esprimiamo in pratica il costo della funzione in termini di se stessa usando l'**equazione di ricorrenza**. E' un modo per esprimere il costo della funzione in termini della funzione stessa.

2.7 Equazioni di ricorrenza

Esprime il costo $T(n)$ di una chiamata con input di dimensione n in termini di una chiamata alla stessa funzione di un input di dimensione $k < n$ piu' piccolo di quello di partenza, (il fatto che sia piu' piccolo ci permette di terminare la

ricorsione). Quindi come dicevamo prima in modo ricorsivo. In generale:

$$\begin{aligned} T(n) &= f(t(k)) \\ T(n) &= 2T\left(\frac{n}{2}\right) + n^2 \end{aligned} \quad (2.14)$$

2.7.1 Esempio

Calcoliamo il fattoriale di n :

```
if n <= 1 THEN fact := 1
      ELSE fact := n per fattoriale(n-1)
return (fact)
```

Cominciamo valutando i vari costi delle operazioni che compongono la funzione, partendo dall'interno, il ramo ELSE:

- FATTORIALE $\cdot (n-1)$ e' la chiamata ricorsiva della funzione su $(n-1)$, quindi in termini di equazione fattoriale lo esprimiamo come $T(n-1)$
- $n \cdot$ FATTORIALE: e' un operazione base e quindi ha costo $O(1)$

Esternamente invece calcoliamo il caso base, ovvero quello che sta fuori dalle chiamate ricorsive:

- La condizione dell'if e' un' operazione base ed ha costo $O(1)$
- Il ramo THEN e' semplicemente un assegnamento, quindi anchessa un operazione base con consto $O(1)$

Quindi possiamo dire che $T(n)$ ha un costo costante c se $n \leq 1$, altrimenti abbiamo il costo della moltiplicazione + il costo della chiamata ricorsiva $T(n-1)$ perche' lo esprimo in termini di se stesso:

$$T(n) = \begin{cases} c & \text{se } n \geq 1 \\ d + T(n-1) & \text{altrimenti} \end{cases}$$

Che possiamo riscrivere come equazione di ricorrenza: $T(n) = T(n-1) + O(1)$

2.7.2 Metodi risoluzione delle equazioni di ricorrenza

- Metodo della sostituzione: e' il metodo piu' facile da applicare ma difficile da fare, perche' parto da una stima. In pratica ipotizzo un costo e lo verifico (usando la notazione asintotica)
- Albero di ricorrenza: si usa piu' spesso, non serve fare ipotesi, e' il modo per rappresentare senza calcoli la struttura delle chiamate ricorsive e poi da questa struttura grafica calcolare la complessita'
- Teorema dell'esperto: ci sono dei casi e se la funzione e' fatta in un modo e rispetta una condizione il costo e' dato. Vale pero' solo in alcune situazioni.

2.7.3 Metodo della sostituzione

E' il metodo piu' facile di tutti, faccio un ipotesi e la vado a verificare usando la definizione della notazione asintotica. Assumo che una equazione di ricorrenza abbia un certo costo di notazione asintotica (generalmente useremo O perche' e' il piu' facile da usare) e controllo se effettivamente la mia ipotesi va bene, tenendo conto della definizione della notazione asintotica.

Esempio

$$T(n) = 2T\left(\frac{n}{2}\right) + n \quad (2.15)$$

Sto dicendo che ho una funzione che chiama se stessa due volte e ciascuna chiamata lavora meta' dell'input ed il costo fuori dalla ricorsione e' n .

Ipotizziamo che $T(n)$ abbia un costo computazionale: $O(n \log n)$

Se e' vero allora:

$$\exists c, n_0 > 0; T(n) \leq c \cdot n \log n \rightarrow \forall n \geq n_0 \quad (2.16)$$

Assumo che l'ipotesi sia vera per input $\frac{n}{2} \rightarrow O(\frac{n}{2} \log \frac{n}{2})$ e verifico se $T(n) = O(n \log n)$:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &\leq 2 \cdot \left(c \cdot \frac{n}{2} \log \frac{n}{2}\right) + n \rightarrow \forall n \geq n_0 \\ &= c \cdot n \log \frac{n}{2} + n \\ &= c \cdot n \log n - c \cdot n \log 2 + n \rightarrow c_1 = c \cdot \log 2 \\ &= c \cdot n \log n - c_1 \cdot n + n \\ &\leq c \cdot n \log n \rightarrow c \geq 1 \end{aligned} \quad (2.17)$$

Lei consiglia di usarlo dopo aver usato un altro approccio.

2.7.4 Albero di ricorsione

Si puo' usare sempre e non richiede di fare ipotesi. In generale un albero e' una struttura fatta da diversi nodi, caratterizzati nello stesso modo, ciascuno dei quali ha un numero di figli. C'e' quindi una relazione genitore/figlio tra i vari nodi. I nodi che non hanno figli sono chiamati fogli ed il nodo in alto che non ha genitori e' chiamato radice.

Innanzitutto dobbiamo sapere come disegnare l'albero, perche' puo' essere usato in maniera agevole per rappresentare le chiamate ricorsive. L'idea e' che man mano la ricorsione va verso una terminazione diminuendo l'input, quindi il nodo modella il costo della chiamata senza considerare le ulteriori chiamate ricorsive che sono rappresentate dai figli. Le foglie invece rappresentano il caso base della ricorsione.

Riassumendo:

- Ogni nodo rappresenta una chiamata della funzione
- I figli di un nodo sono le sue chiamate ricorsive
- Il costo complessivo della funzione e' dato dalla somma dei costi di tutti i nodi dell'albero, perche' ogni nodo rappresenta il costo della chiamata corrispondente al netto delle chiamate ricorsive che fa

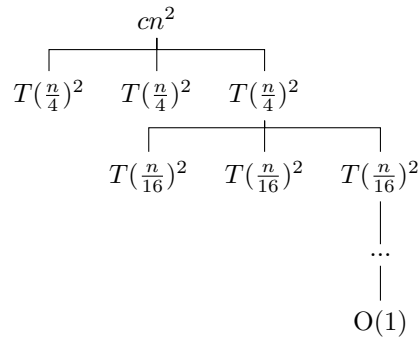
Esempio

Immaginiamo di avere un'equazione di ricorrenza fatta in questo modo:

$$T(n) = 3T\left(\frac{n}{4}\right) + n^2 \quad (2.18)$$

Mi dice che fa 3 chiamate ricorsive (la costante moltiplicativa della T, che come ricordiamo rappresenta la chiamata ricorsiva) e ciascuna lavora su un input che e' $1/4$ dell'input originale (nell'esempio di prima era $n-1$, qui semplicemente il modo in cui mi avvicino al caso base cambia) ed il costo fuori dalla chiamata ricorsiva e' n^2 .

Disegniamo l'albero: La radice dell'albero e' il costo dell'equazione tolte le chiamate ricorsive. Le chiamate ricorsive sono 3 come abbiamo detto prima, quindi il nodo padre ha 3 figli (come qualsiasi nodo dell'albero, tranne le foglie) ognuno dei quali ha un costo $t(n\frac{n}{4})$, ma cosa vuol dire, sto facendo una chiamata alla funzione con input $\frac{1}{4}$ che mi da in output n^2 che e' la parte fuori dalle chiamate ricorsive. In sostanza per la prima iterazione (il primo livello dell'albero) passo alla funzione ricorsiva l'input n , quindi in output avro' $\frac{n}{4}$ (lo deduciamo dall'equazione di ricorrenza). Alla seconda iterazione passerò alla funzione il nuovo input, che come abbiamo appena detto e' $\frac{n}{4}$, e come prima la funzione lo dividerà per 4, ottenendo in output $\frac{n}{16}$. Elevo ogni nodo al quadrato perche' e' la complessita' del caso base. Si continua così finche' ottengo in output un risultato che e' nell'ordine di $O(1)$.



Vediamo ora come calcolare la complessita' dato l'albero. Dobbiamo sommare tutto quanto, non facciamo in realta' la somma ma facciamo delle approssimazioni.

Dato il livello i nell'albero dobbiamo capire:

- com'è fatto il nodo, nel nostro caso continuiamo a dividere per 4 ad ogni discesa: $\frac{n}{4^i}$ che è la dimensione dell'input a livello i
- Costo del nodo: $(\frac{n}{4^i})^2$
- Numero di nodi al livello: sto moltiplicando per 3 ad ogni iterazioni quindi al livello i -esimo il numero di nodi sarà 3^i
- Costo del livello: Moltiplicando il costo del nodo per il numero di nodi $(\frac{n}{4^i})^2 \cdot 3^i \rightarrow (\frac{3}{16})^i n^2$

Adesso che ho capito quanto costa ogni livello mi serve capire quanti sono i livelli dell'albero, sappiamo che le foglie sono il caso base quindi è come scrivere $\frac{n}{4^i} = 1$ che è come chiedersi quanto vale la i : $i = \log_4 n$.

Ora posso calcolare il costo complessivo:

$$\sum_{i=0}^{\log_4 N} (\frac{3}{16})^i n^2 = n^2 \sum_{i=0}^{\log_4 N} (\frac{3}{16})^i \quad (2.19)$$

All'esame ci dice che quella sommatoria converge ad una costante c che tende a $\frac{1}{1-\frac{3}{16}}$ il cui costo computazionale è circa $O(n^2)$.

Va verificato il costo del livello delle foglie: 3^i con $i = \log_4 n$ allora $3^{\log_4 n} \cdot O(1) = n^{\log_4 3} = O(n)$

2.7.5 Teorema dell'esperto (master theorem)

Si basa sul fatto che ci sono delle proprietà delle equazioni di ricorrenza che possono essere sfruttate, quindi ci sono classi di equazioni di ricorrenza che possono essere risolte facilmente perché rispettano delle proprietà.

Sono le classi di equazioni di ricorrenza scritte come:

$$T(n) = aT(\frac{n}{b}) + f(n) \quad (2.20)$$

Abbiamo tre casi:

1. Se $f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0 \rightarrow T(n) = \Theta(n^{\log_b a})$
2. Se $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \log n)$
3. Se $f(n) = \Omega(n^{\log_b a + \epsilon})$, $\epsilon > 0$ e $af(\frac{n}{b}) \leq cf(n) \rightarrow T(n) = \Theta(f(n))$ per costante $c > 1$ e n sufficientemente grandi.

Non si può applicare con chiamate ricorsive che usano input diversi.

Esempio

$$T(n) = 9T\left(\frac{n}{3}\right) + n \quad (2.21)$$

Cerchiamo di capire se possiamo usare la formula del teorema:

$$\begin{aligned} a = 9, b = 3, f(n) = n \\ \log_b a = \log_3 9 = 2 \end{aligned} \quad (2.22)$$

fatto il mapping sappiamo che siamo in uno dei casi del teorema, ora proviamo a capire in quale:

$$f(n) \neq O(n^{2-\epsilon}) \rightarrow n \neq O(n^{2-\epsilon}) \rightarrow T(n) = \Theta(n^2) \quad (2.23)$$

Esercizio

$$T(n) = 2T\left(\frac{n}{3}\right) + n^2 \quad (2.24)$$

Risolviamolo in tutti e tre i modi, famolo prima con l'albero

Chapter 3

Liste

3.1 Lista

Andiamo a vedere dal punto di vista algoritmico cosa vuol dire una lista. Viene definita come una sequenza ordinata di zero o piu' elementi dello stesso tipo (di base o complesso), e' importante che tutti gli elementi siano omogenei (tutti interi etc). Una lista puo' essere fatta anche da tipi complessi. Gli elementi sono posizionati in sequenza, quindi so in che posizione si trova un elemento, non sono ordinati dal punto di vista del loro significato ma nel senso che ciascuna elemento ha la sua posizione ed io so dire quale elemento precede o succede un altro in termini della posizione. Posso scrivere una lista come a_1, a_2, a_n sapendo che a_1 e' il primo elemento ed a_n e' l'ultimo elemento. Data una lista da 1 ad n , n e' detta essere la lunghezza della lista (il numero di elementi che la compongono). a_i e' detto elemento della lista, ed e' del tipo specificato per la lista.

3.1.1 Operazioni

E' un elenco parziale e che cerca di essere generale, questo non preclude il fatto che quando definiamo il nostro specifico tipo di lista possiamo definire altre operazioni oltre a queste:

- INSERT(x, p, l): inserisce x nella lista l in posizione p . $a_1, a_2, \dots, a_n \Rightarrow a_1, a_2, \dots, a_{p-1}, x, a_p, \dots, a_n$
- LOCATE(x, l): va a trovare l'elemento x dentro la lista l e ne restituisce la posizione se l'elemento fa parte della lista, altrimenti mi ritorna che l'elemento non esiste.
- RETRIVE(p, l): mi restituisce l'elemento in posizione p della lista l .
- DELETE(p, l): cancella l'elemento in posizione p della lista l
- NEXT(p, l): torna la posizione successiva di p nella lista l
- PREVIOUS(p, l): torna la posizione precedente a p nella lista l
- FIRST(l): torna la prima posizione della lista
- MAKE NULL(l): svuota la lista l

3.1.2 Implementazione come un array

Un array e' un insieme di variabili tutte dello stesso tipo, quindi questo mi va gia' bene per la lista, inoltre di sua natura ci garantisce l'ordinamento di ogni elemento perche' e' ordinato.

Avro' nella parte iniziale dell'array la lista e successivamente nell'array dello spazio vuoto (per peremettere alla lista di crescere dinamicamente). Gli elementi della lista sono memorizzati in celle contigue dell'array, volendo, si puo' definire una variabile LAST che tiene traccia di qual'e' l'ultimo elemento della lista, all'inizio sara' $= 0$ che significa che la lista e' vuota.

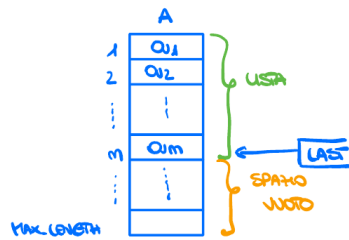


Figure 3.1: Lista implementata come un array

INSERT(x, p, A)

Vediamo com'è implementata l'operazione Insert in pseudocodice per una lista implementata come array. x è l'elemento da inserire, p è la posizione (intero), A è l'array.

```
//controlla che la lista non sia piena
  IF LAST >= MAXLENGTH
    THEN ERROR "LISTA PIENA"

//controlla LAST+1 perche' inserisco l'elemento in ultima posizione
  IF (P > LAST + 1) OR (P < 1)
    THEN ERROR "POSIZIONE INESISTENTE"

//sposta avanti di una posizione l'elemento in posizione p
  FOR q = LAST DOWN TO P DO
    A[q + 1] = A[q]

//inserimento nella lista
  LAST = LAST + 1
  A[p] = x
```

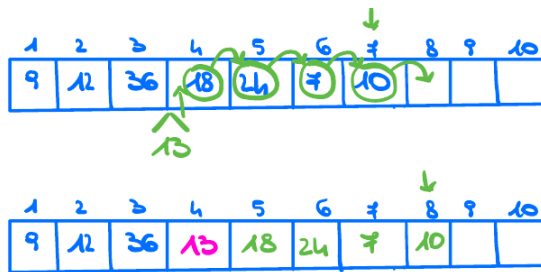
Il secondo if controlla che la posizione sia valida, fa il controllo a $LAST+1$ perche' potremmo voler aggiungere l'elemento in coda alla lista, e dev'esserci spazio per poterlo fare. Il for invece serve per spostare avanti di una posizione l'elemento in posizione $q+1$, in modo da poter inserire l'elemento nella posizione giusta, operazione fatta con l'ultima riga del codice.

Facendo il calcolo della complessita', vediamo che tutte e due gli if hanno complessita' $O(1)$, cosi' come l'inserimento (ultima riga). Mentre il for, l'operazione interna l'assegnamento ha costo $O(1)$, mentre il for dipende dall'input, quindi $O(n)$. Quindi il costo totale è $O(n)$, lineare.

LOCATE(x, A)

Individua la posizione di x nella lista.

```
FOR q = 1 TO LAST DO
```



```

if A[q] = x
    THEN RETURN (q)
RETURN "ELEMENTO NON PRESENTE"

```

L'if ha costo $O(1)$, mentre il for a costo lineare $O(n)$. Quindi il costo totale e' $O(n)$

DELETE(p, A)

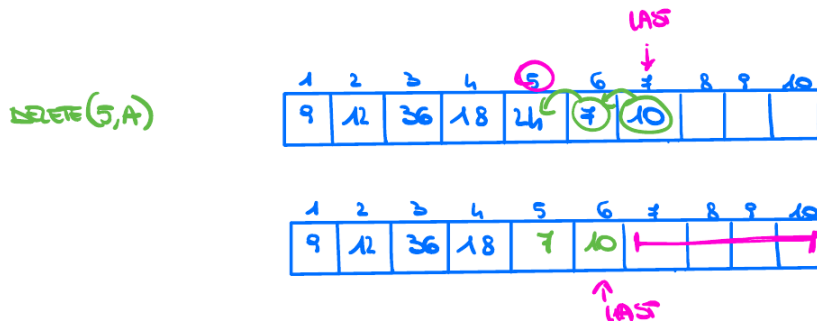
Cancello l'elemento in posizione p.

```

IF (p > LAST) OR (p < 1)
    THEN ERROR "POSIZIONE NON ESISTE"
FOR q = p TO LAST DO
    A[q] = A[q+1]
LAST = LAST - 1

```

Il costo computazionale: l'if ha costo $O(1)$, cosi' come l'ultima associazione. Mentre il for ha costo lineare $O(n)$. Quindi il costo totale e' $O(n)$



3.1.3 Altre operazioni

```

RETRIVE(p, A)
    RETURN A[p]

```

```
NEXT(p,A)
  RETURN p+1
```

```
PREVIOUS(p,A)
  RETURN p-1
```

```
FIRST(A)
  RETURN 1
```

```
FIRST.ELEMENT(A)
  RETURN A[1]
```

```
MAKENULL(A)
  LAST = 0
```

Tutte ovviamente con costo $O(1)$.

3.1.4 Implementazione tramite puntatori

Head e' il puntatore al primo elemento e TAIL e' il puntatore all'ultimo elemento (che non ha successori). In questa implementazione ogni elemento ha associato un puntatore all'elemento successivo nella lista.

INSERT(x, p, l)

```
TEMP = p.NEXT
ALLOCATE(p.NEXT)
(p.NEXT).KEY = x
(p.NEXT).NEXT = TEMP
```

LOCATE(x, l)

```
p = l.HEAD
WHILE p != NIL DO
  IF p.KEY = x
    THEN RETURN p
  p = p.NEXT
RETURN p
```

DELETE(p,l)

E' molto facile perche' baipasso un puntatore e faccio puntare il mio elemento al successivo di quello che voglio cancellare:

```
p.NEXT = (p.NEXT).NEXT
```

Nota: per cancellare l'elemento con valore x devo fare locate + delete.

3.1.5 Vantaggi e svantaggi

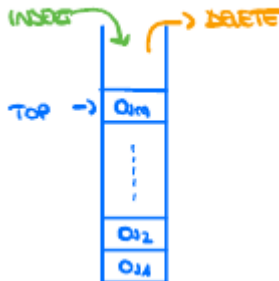
	ARRAY	PUNTATORI
INSERT	$O(n)$	$O(1)$
DELETE	$O(n)$	$O(1)$
LOCATE	$O(n)$	$O(n)$
PREVIOUS	$O(1)$	$O(n)$

I puntatori sono meno costosi, con i puntatori non devo conoscere la lunghezza massima della lista, con i puntatori per trovare il predecessore devo scorrere la lista. Quello che si fa per risolvere il problema del predecessore e' usare delle liste doppiamente linkate, con un link sia a NEXT che a PREVIOUS. Un modo per implementare le liste doppiamente puntate e' fare una lista circolare, quindi il primo elemento ha come predecessore l'ultimo ed l'ultimo ha come successore il primo, il vantaggio e' che non abbiamo il caso particolare del primo elemento e dell'ultimo (che nel caso di lista non circolare sono linkati a null).

Chapter 4

Stack

E' un tipo particolare di lista caratterizzato da un set di operazioni che sono un sottoinsieme di quelle della lista, e le operazioni di inserimento e cancellazione operano ad una sola estremita' che si chiama top dello stack. Inoltre e' importante notare che e' un'ADT che opera secondo la logica LIFO (Last In First Out).

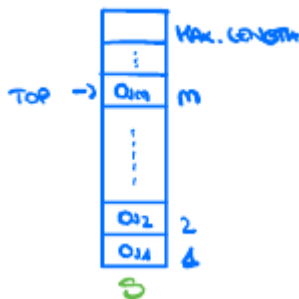


Si parla di push per l'inserimento e di pop per la cancellazione. Operazioni che caratterizzano lo STACK:

- PUSH(x, S): inserisce x sul top dello stack
- POP(S): toglie (e ritorna) l'elemento in posizione top dallo stack
- TOP(S): ritorna (ma non rimuove) l'elemento in posizione top
- MAKENULL(s): svuota lo stack
- EMPTY(s): true se lo stack e' vuoto

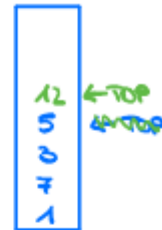
4.1 Implementazione con array

Va a ripercorrere la rappresentazione visiva, cioe' se mi immagino un array rappresentato in verticale che chiamiamo S: Array S[1 ... MAX_LENGTH] di elementi del tipo dello stack.



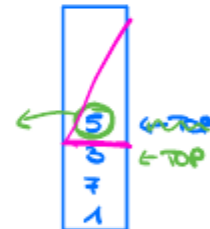
Guardiamo tutte le operazioni che operano sullo stack implementato tramite array:

PUSH(x, S) /* INSERISCE x IN AUTO */ $O(1)$
 IF (S.TOP $\geq$ MAXLENGTH) $O(1)$
 THEN ERROR "STACK PLEN"
 S.TOP = S.TOP + 1 $O(1)$
 S[S.TOP] = x $O(1)$



EMPTY(S) $O(1)$
 IF S.TOP = 0
 THEN RETURN TRUE
 ELSE RETURN FALSE

POP(S) /* ESTRAE L'ELEMENTO IN AUTO NEW STACK */ $O(1)$
 IF EMPTY(S) $O(1)$
 THEN ERROR "UNDERFLOW" $O(1)$
 ELSE S.TOP = S.TOP - 1 $O(1)$
 RETURN S[S.TOP + 1] $O(1)$

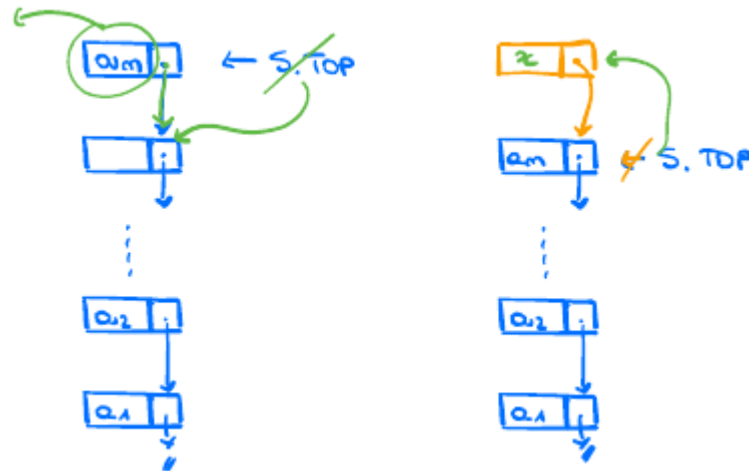


TOP(S) /* INDIVIDUA L'ELEMENTO IN AUTO NEW STACK */ $O(1)$
 IF EMPTY(S)
 THEN ERROR "UNDERFLOW"
 ELSE RETURN S[S.TOP]

MAKENULL(S) $O(1)$
 S.TOP = 0

Implementazione con puntatori

L'unica cosa da vedere e' come collegare gli elementi all'interno dello stack. Il next dell'elemento a cosa punta? I puntatori vanno sempre verso il basso, perche' l'unico movimento che puo' fare il top e' verso il basso, quando devo spostare il top verso l'alto e' perche' ho inserito un nuovo elemento e quindi sicuramente ne conosco la posizione.



Chapter 5

Coda

È un tipo particolare di lista dove le operazioni di inserimento e cancellazione vengono fatti alle due estremità della lista, gli inserimenti alla coda (tail), testa e le cancellazioni alla testa(head) della coda.

Segue una logica FIFO (First In First Out).

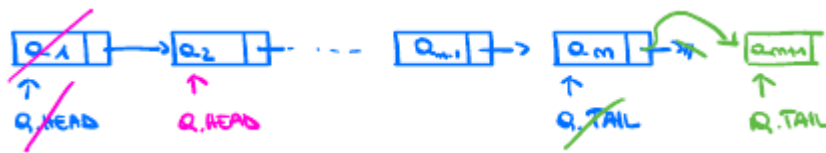


Operazioni:

- ENQUEUE(x, q): inserisco x nella coda q (in fondo nella posizione TAIL)
- DEQUEUE(x, q): rimuove e ritorna l'elemento in testa alla coda
- FRONT(q): mi ritorna (ma non rimuove) l'elemento che sta' in testa alla coda
- MAKENULL(q): svuota la coda
- EMPTY(q): true se la coda e' vuota

5.1 Implementazione con i puntatori

I puntatori vanno dalla head alla tail perche' quando faccio l'estrazione di un elemento dalla head devo sapere qual'e' l'elemento che lo precede.



5.1.1 ENQUEUE(x,Q)

Inserisce x in Q. Q.TAIL e' la prima posizione vuota dopo la coda ($a_n + 1$) e' la posizione dove effettivamente vado a fare l'inserimento.

```

Q[Q.TAIL] = x
IF Q.TAIL = Q.MAXLENGTH
  THEN Q.TAIL = 1
  ELSE Q.TAIL = Q.TAIL + 1

```

Dato che la coda non e' piena inserisco il valore che voglio inserire nella TAIL, dopo spostato in avanti la TAIL nella prima posizione vuota, la cosa normale da fare sarebbe un $+ 1$, ma se sono arrivato in fondo alla posizione finale del vettore la rimetto uguale ad 1, in modo da avere un array circolare.

5.1.2 DEQUEUE(Q)

```

x = Q[Q.TAIL]
IF Q.HEAD = Q.MAXLENGHT
    THEN Q.HEAD = 1
    ELSE Q.HEAD = Q.HEAD + 1
RETURN x

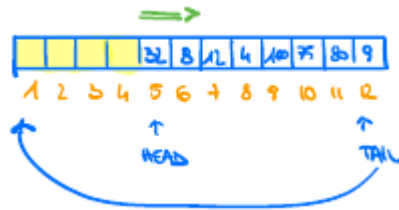
```

Ricordiamo che quando tail ed head coincidono la coda puo' essere piena o vuota, si usa una variabile booleana per distinguere i due casi.

5.2 Implementazione con array

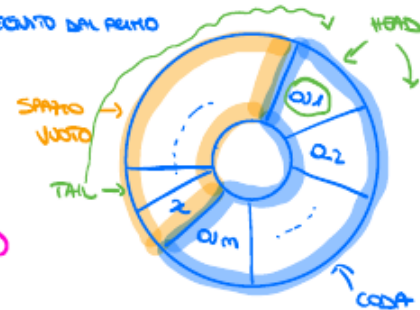
Ho un array del tipo di cui e' stata definita la coda. Inoltre definiamo due variabili intere:

- HEAD: ci dice la posizione del primo elemento
- TAIL: dice la posizione dell'ultimo elemento



L'idea e' che continuo a inserire e cancellare, quando arrivo in fondo all'array riempiendo l'ultima posizione la parte iniziale e' vuota quindi c'e' potenzialmente spazio per inserire nuovi elementi. La soluzione a questo problema e' considerare un'array circolare, in cui l'ultimo elemento e' seguito dal primo.

ARRAY CIRCOLARE = ULTIMO ELEMENTO È SECONDO DAL PRIMO



ENQUEUE (z, Q) /* INSERISCE z IN Q */ $O(1)$

$Q[Q.TAIL] = z$ $O(1)$

IF $Q.TAIL = Q.MAXLENGTH$ $O(1)$

THEN $Q.TAIL = 1$ $O(1)$

ELSE $Q.TAIL = Q.TAIL + 1$ $O(1)$

$Q.TAIL$ = POSIZIONE JUSTA
Dopo l'ultimo
elemento nella coda

DEQUEUE (Q) /* RIMOZIONE */ $O(1)$

$z = Q[Q.HEAD]$ $O(1)$

IF $Q.HEAD = Q.MAXLENGTH$

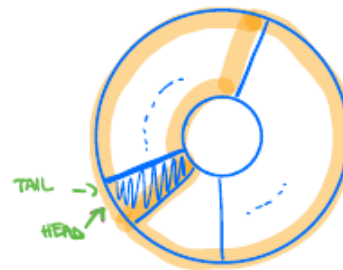
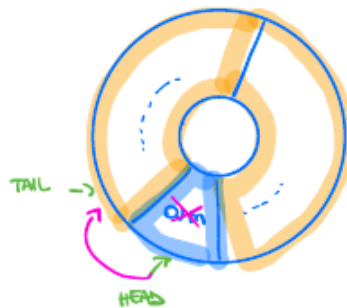
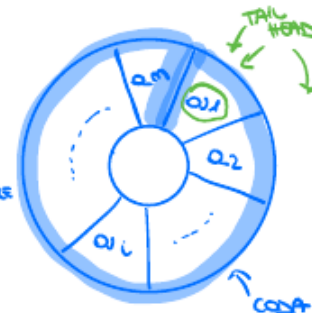
THEN $Q.HEAD = 1$

ELSE $Q.HEAD = Q.HEAD + 1$ $O(1)$

RETURN z $O(1)$

NOTA = HEAD = TAIL $\Rightarrow$ CODA PENA
HEAD = TAIL $\Rightarrow$ CODA VUOTA

SI USA UNA VARIABLE
BOOLEANA PER DISTINGUERE
I DUE CASI



Chapter 6

Alberi

E' una prima ADT che possiamo considerare complessa. Stiamo parlando di alberi radicati perche' esistono altri tipi di alberi che hanno delle caratteristiche un po' diverse. Gli alberi radicati sono un ADT caratterizzato dal fatto che sono una collezione di elementi (dati di base) tutti dello stesso tipo che sono chiamati nodi, uno di questi nodi e' la radice, i nodi sono collegati tra loro tramite una relazione gerarchica genitore/figlio. Quindi dato qualsiasi nodo dell'albero so sempre qual'e' il padre (tranne la radice che non ha padre).

Si puo' pensare l'albero anche in modo ricorsivo: un singolo nodo e' un albero ed e' anche la radice di se stesso (ed e' in qualche modo il caso base della ricorsione). A questo punto immaginiamo che n sia un nodo e che abbiamo un insieme di alberi radicati nei nodi, la struttura che si ottiene definendo n come genitore dei nodi degli alberi e' un albero radicato in n . Un caso particolare e' un albero senza alcun nodo, che e' un albero vuoto.

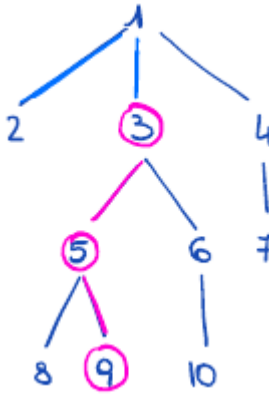


Ogni nodo dell'albero ha esattamente un genitore (tranne la root) $PARENT(P)$.

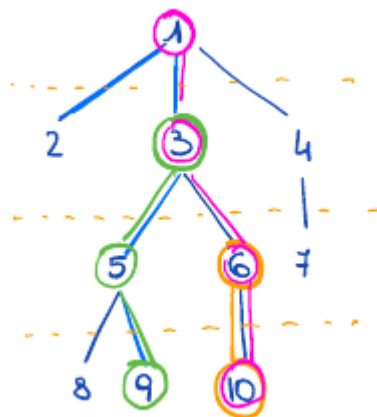
6.0.1 Terminologia

- Esiste una sola root
- Ogni nodo ha un genitore: $PARENT(3) = 1$.
- I nodi con lo stesso genitore sono detti fratelli **SIBLING**
- Il numero di figli di un nodo e' detto $grado(n)$, il massimo grado di un nodo nell'albero e' il grado dell'albero
- I nodi senza figli sono detti foglie
- I nodi non foglia sono detti nodi interni
- Il cammino e' una sequenza di nodi
- La lunghezza del cammino e' il numero dei nodi che compongono il cammino meno 1 che e' uguale a dire il numero di archi che percorro

Altra terminologia:



- Predecessore (ANCESTOR): di un nodo n , p e' predecessore di n in T se esiste in T un cammino da p ad n . (un nodo e' anche l'ancestor di se stesso, perche' ho sempre il cammino lungo 0).
- Discendente (DESCENDANT): di un nodo n , d e' successore di n in T se esiste in T un cammino da n a d .
- Altezza: di un nodo n e' la lunghezza del cammino piu' lungo da n a una foglia.
- Altezza di un albero: e' l'altezza della radice
- Profondita' di un nodo n e' la lunghezza del cammino (unico) dalla radice a n

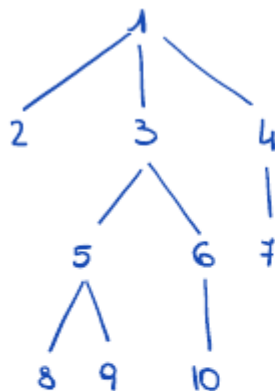


Operazioni sull'albero:

- $\text{PARENT}(n, T)$: ritorna il genitore di n in T
- $\text{LEFT_MOST_CHILD}(n, T)$: ritorna il figlio piu' a sinistra di n in T
- $\text{RIGHT_SIBLING}(n, T)$: ritorna il fratello destro di n in T
- $\text{LABEL}(n, T)$: ritorna l'etichetta del nodo n
- $\text{CREATE}(n_1, T_1, \dots, T_k)$: crea un albero con radice n genitore delle radici di $T_1 \dots T_k$
- $\text{ROOT}(T)$: ritorna la radice di T
- $\text{MAKE_NULL}(T)$: svuota l'albero

6.1 Implementazione come array

Ho un array con una cella per ogni nodo dell'albero, per ogni indice dell'array che rappresenta il nodo, metto dentro l'indice del genitore, per convenzione diciamo che il genitore della radice e' 0.



0	1	1	1	3	3	4	5	5	6
1	2	3	4	5	6	7	8	9	10

ARRAY CON UNA CELLA PER OGNI NODO NELL'ALBERO

$T[i]$ RAPPRESENTA IL NODO i
ID DEL GENITORE DEL NODO i

$T[i] = 3 \Rightarrow 3$ E' GENITORE DI i NELL'ALBERO

$T[i] = 0$ SE i E' ROOT



1	2	3	4
A	B	C	D

CONTENUTO

1	2	3	4
0	1	1	2

STRUTTURA (DEL GENITORE-FIELDS)

Vantaggi di questa implementazione:

- Efficiente nel trovare il genitore di n , $O(1)$

Svantaggi:

- Costoso trovare i figli di un nodo n , perché dobbiamo cercare nel vettore, $O(N)$ dove N è il numero di nodi in T , quindi nel caso peggiore devo scorrere tutti gli elementi del vettore.
- Perdo l'ordine dei figli di n (a meno di adottare una convenzione)

6.1.1 Implementazione figlio-sinistro, fratello-destro

Ho due vettori:

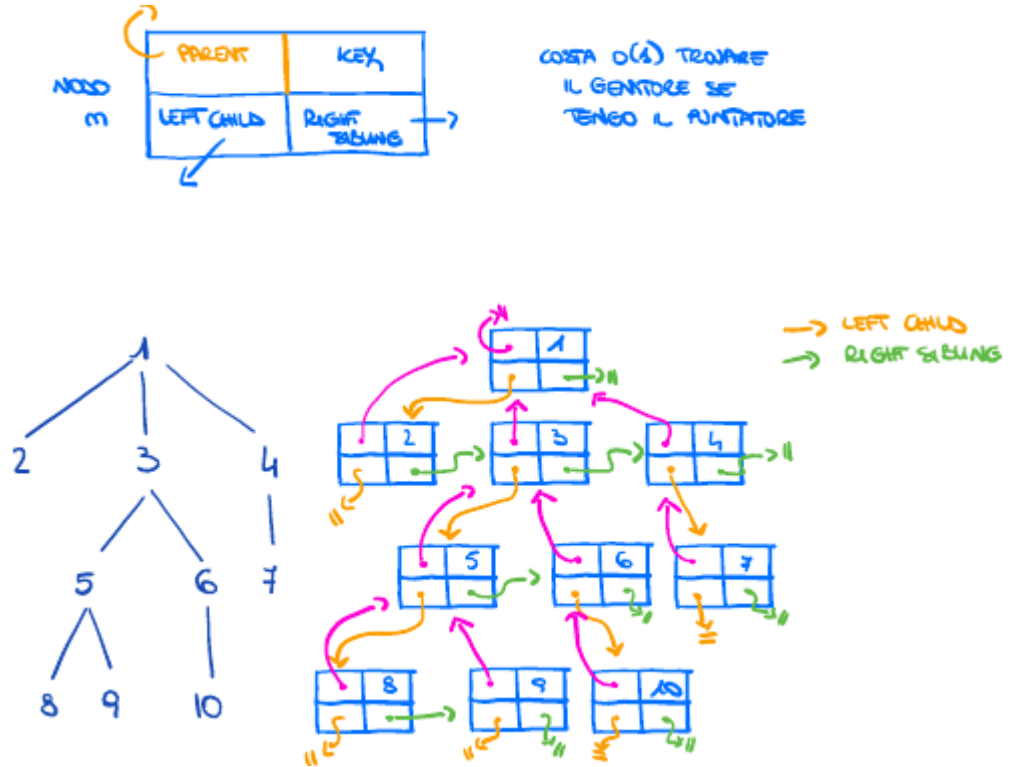
- LEFT CHILD di n è l'id del suo figlio
- RIGHT SIBLING di n è l'ID del suo fratello a destra (NULL se non c'è per n)



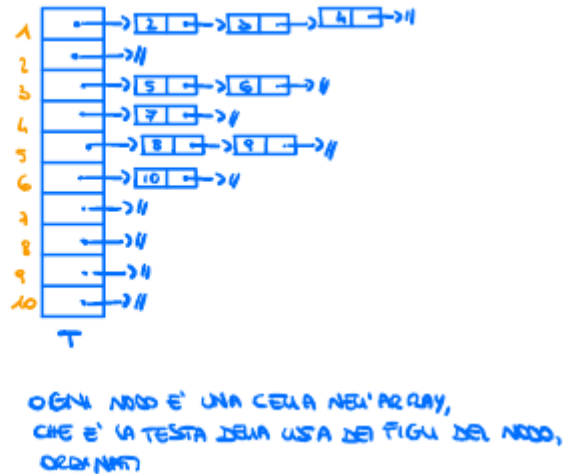
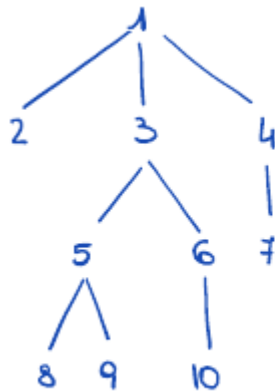
Vantaggi e svantaggi di implementare i figli e fratelli con i vettori:

- Trovo velocemente i figli di N (LEFT CHILD[n] e poi RIGHT SIBLING a catena), ha un costo $O(g)$ dove g è il grado di T .
- Mantengo l'ordine dei figli di n
- Svantaggio: è costoso trovare il genitore di n , perché devo scorrere LEFT_CHILD e RIGHT_SIBLING cercando n e poi man mano i fratelli trovati fino al primo. $O(g \cdot N)$

Vediamo ora l'implementazione tramite i puntatori:



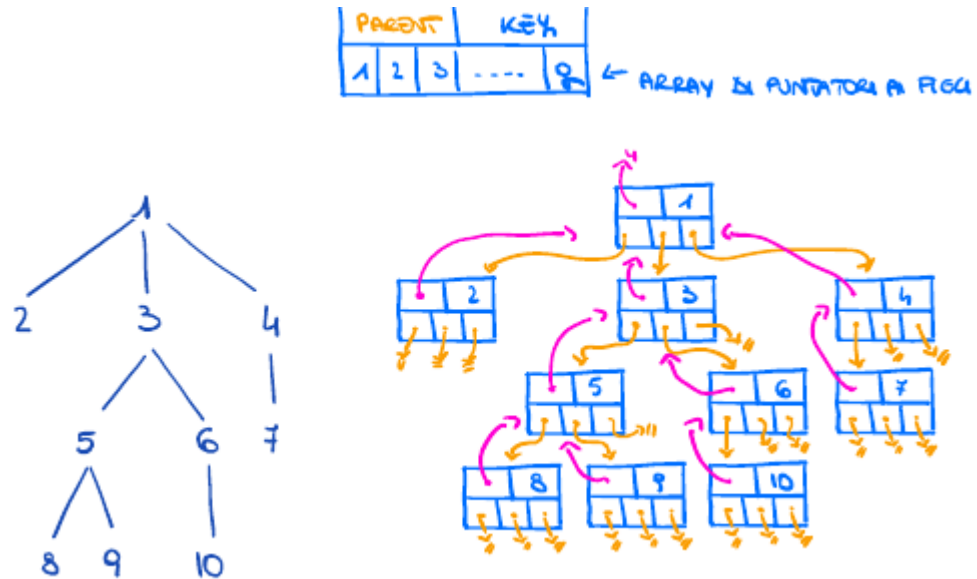
6.1.2 Implementazione con lista dei figli



Vantaggi e svantaggi:

- E' efficiente trovare i figli di n , perche' devo scorrere la lista puntata da $T[n]$, quindi ha costo $O(g)$.
- E' costoso trovare il padre di n , perche' devo scorrere tutte le liste cercando n , quindi ha un costo di $O(N)$, la lunghezza totale delle liste e' $N-1$, perche' ogni nodo tranne la radice e' in una lista

6.1.3 Implementazione con puntatori dei figli



E' piu' efficiente trovare il genitore di n , perche' e' puntato, costa $O(1)$.

6.2 Visita

La visita di un albero T raggiunge in modo sistematico TUTTI i nodi in T senza guardare piu' volte lo stesso nodo. Esistono varie tipi di visita che guarderemo:

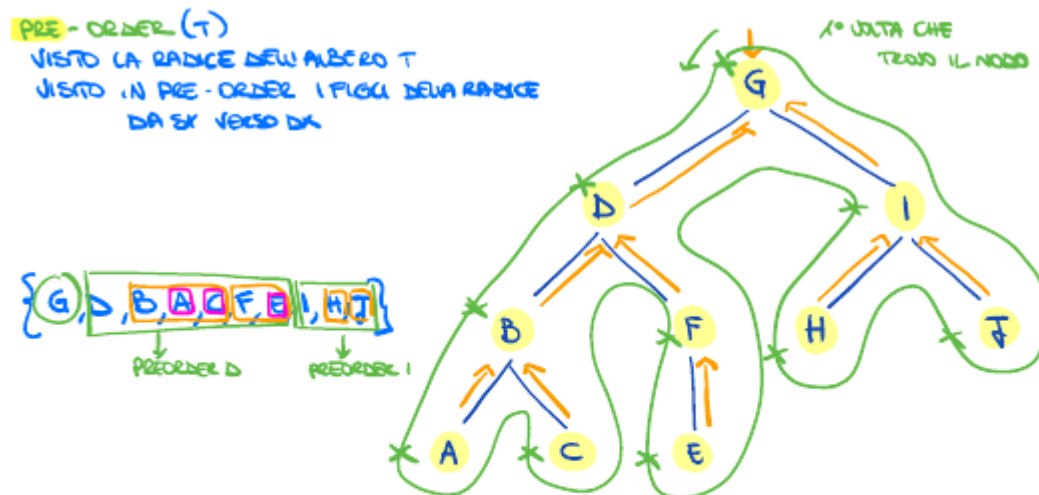
- PROFONDITA, in cui percorro i rami in verticale:
 - PRE ORDER
 - IN ORDER
 - POST ORDER
- AMPIEZZA, in cui divido l'albero per livelli (in orizzontale)

6.2.1 Visite in profondita'

Partiamo da dei casi particolari, nel caso in cui ho l'albero vuoto la visita risultera' vuota, mentre nel caso in cui ho un albero con un solo nodo il risultato della visita e' il nodo stesso.

PRE-ORDER(T)

Visito la radice dell'albero T, successivamente visito in PRE-ORDER i figli della radice da sinistra verso destra.



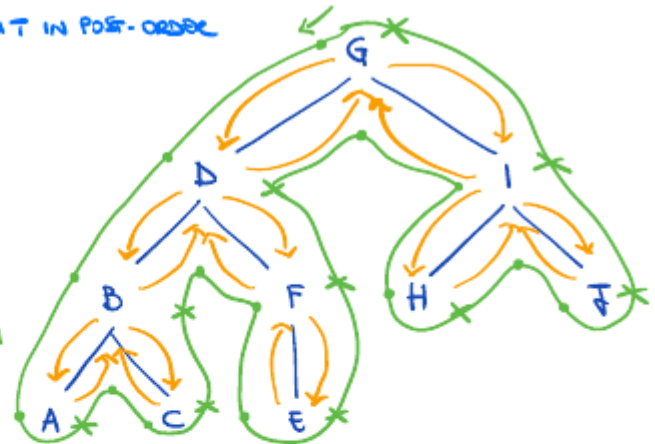
PRE-ORDER (m)
 o(1) LABEL (m) /* GUARDA m */
 o(1) C = m. CHILDREN
 WHILE C ≠ NULL DO
 PREORDER (C)
 o(1) C = C. NEXT] VISITA I FIGLI IN PRE-ORDER

IN-ORDER(T)

- Visito IN-ORDER il figlio sinistro della radice
- Visito la radice di T
- Visito da sinistra verso destra tutti gli altri figli di T

POST-ORDER (T)

- VISITO I FIGLI DELLA RADICE DI T IN POST-ORDER DA SX VERSO DX
- VISITO LA RADICE DI T

**POST-ORDER (m)**

```

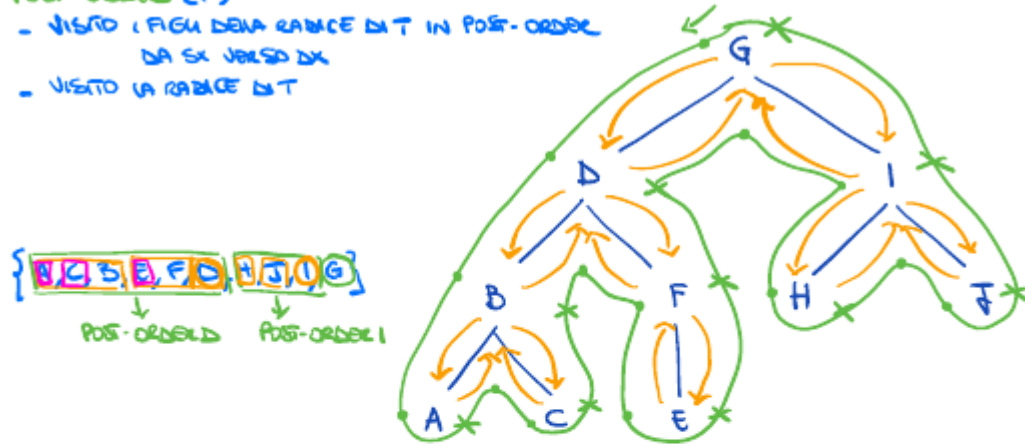
C = m.CHILDREN o(s)
WHILE C ≠ NULL DO o(s)
    POST-ORDER (C)
    C = C.NEXT o(s)
    POST-ORDER (A) CH m
LABEL (m) o(s)      NOSS m
  
```

POST-ORDER(T)

- Visito i figli della radice di T in POST-ORDER da sinistra verso destra
- Visito la radice di T

POST-ORDER (T)

- VISITO I FIGLI DELLA RADICE DI T IN POST-ORDER DA SX VERSO DX
- VISITO LA RADICE DI T

**POST-ORDER (m)**

```

C = m.CHILDREN o(s)
WHILE C ≠ NULL DO o(s)
    POST-ORDER(C)
    C = C.NEXT o(s)
    LABEL (m) o(s)
  
```

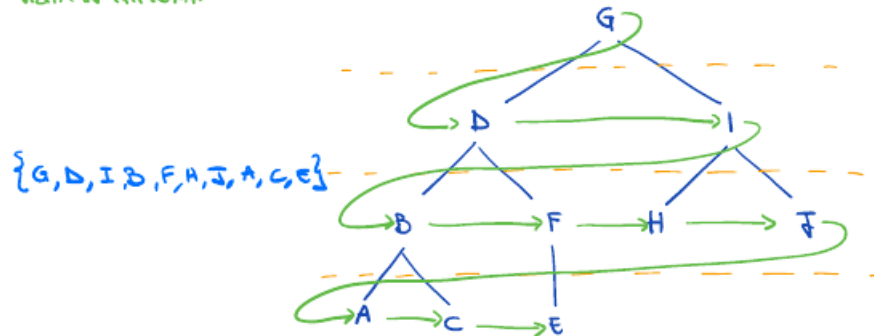
POST-ORDER (FLEU DI m)
NODO m

Notiamo che il costo delle visite in profondita' e' sempre $O(n)$, perche' ogni nodo ha solo un genitore (mai piu' di uno), quindi avro' non piu' di una chiamata ricorsiva per ciascun nodo. Inoltre ogni nodo ha 1 genitore (non meno di uno), quindi avro' almeno una chiamata.

6.2.2 Visita in ampiezza

Possiamo pensare di dividere l'albero a "fette" dividendo i livelli e pensare di scorrere l'albero seguendo il livelli dall'alto verso il basso e dal nodo di sinistra a quello di destra.

VISTA IN AMPIEZZA



BFS

```

BREADTH-FIRST-SEARCH(T)  O(m)
  O(1) q = NEW-QUEUE()  /* NON ENCODA DA VISITARE */
  O(1) ENQUEUE(T.ROOT, q)
  WHILE NOT EMPTY(q) DO
    p = DEQUEUE(q)  O(1)
    LABEL(p)  O(1)
    C = p.CHILDREN  O(1)
    WHILE C ≠ NULL DO
      ENQUEUE(C, q)  O(1)
      C = C.NEXT  O(1)
  
```

OGNI NODO VIENE INSERITO IN CODA

- NON PIÙ DI UNA VOLTA → HA UN SOLO GENITORE
- NON PIÙ DI UNA VOLTA → PERCHÉ PRATO DALLA RADICE E TUTTI I NODI SONO RAGGIUNGIBILI DALL'

INSERISCO IN LA CODA
SOLO QUANDO VISITO
IL SUO GENITORE

```

q: -
q: G      ENQUEUE(G)
q: G      G = DEQUEUE
q: -      ENQUEUE(D, I)
q: D, I   D = DEQUEUE
q: I      ENQUEUE(B, F)
q: I, B, F I = DEQUEUE
q: B, F   ENQUEUE(H, J)
q: B, F, H, J B = DEQUEUE
q: F, H, J ENQUEUE(A, C)
q: F, H, J, A, C F = DEQUEUE
q: H, J, A, C ENQUEUE(E)
q: H, J, A, C, E H = DEQUEUE
q: J, A, C, E J = DEQUEUE
q: A, C, E A = DEQUEUE
q: C, E    C = DEQUEUE
q: E       E = DEQUEUE
q: -
  
```


Devo avere una strategia sistematica per visitare i livelli, quindi non mi basta la semplice relazione genitore/figlio dei nodi, ma dobbiamo anche tenere traccia di quali sono i nodi del livello.

L'idea e' quella di usare una coda per tenere traccia di tutti i nodi che vado a visitare per tenere traccia dei figli fino a che arrivo all'altro livello.

La troveremo indicata come BFS, come prima cosa definisce una coda che chiamiamo q, come primo passo inserisco la radice dell'albero nella coda, la coda in sostanza mi tiene traccia dei nodi che devo ancora visitare. Poi c'e' un ciclo while, in cui definisco una variabile p che mi tiene traccia del nodo da togliere dalla coda, poi con la funzione lable scrivo il valore della radice, a questo punto setto un'altra variabile c che mi tiene traccia dei figli del nodo che ho tolto, a questo punto tolgo dalla coda il primo nodo ed inserisco i figli di quel nodo in coda alla coda. Vado avanti cosi' fino a che la coda non e' vuota.

Quindi guarda l'albero una fetta alla volta utilizzando una coda, perche' nella coda avro' tutti i nodi di un livello oppure all'inizio i nodi di un livello ed alla fine quelli al livello sottostante. Non avro' mai all'interno della coda dei nodi a piu' di un livello di distanza.

6.3 Alberi binari di ricerca (BST)

E' un albero, quindi rispetta le caratteristiche degli alberi, e' binario, quindi ogni nodo ha massimo 2 figli ed ha delle caratteristiche particolari di ordinamento di valori all'interno dell'albero, quindi mi permette di ricercare i valori che voglio in modo piu' efficiente rispetto a scorrere tutto l'albero.

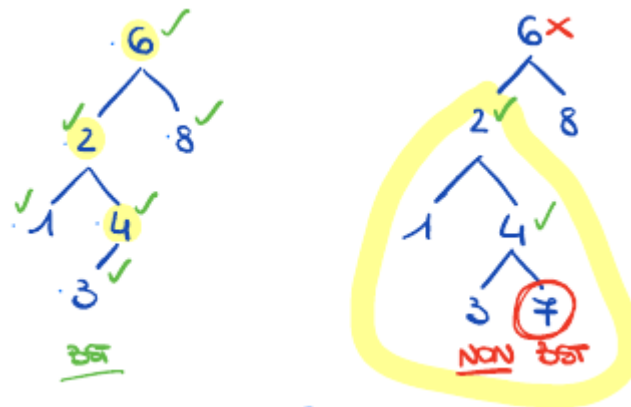
Un albero binario e' un albero vuoto oppure un albero i cui nodi anno:

- 0 figli
- 1 figlio sinistro
- 1 figlio destro
- 2 figli (1 sinistro, 1 destro)

Valgono tutte le tecniche di visita e le implementazioni viste per gli alberi generali. Un albero binario di ricerca rappresenta un insieme ordinato, di elementi (relazione di ordine totale sul dominio dell'albero D , $\forall a, b \in D \rightarrow a > b$ o $a < b$ o $a = b$). Ogni nodo n e' associato ad un valore:

- Le chiavi nel sottoalbero sinistro di n sono piu' piccole di n
- le chiavi nel sottoalbero destro di n sono piu' grandi di n

Ocio che come sottoalbero si intende tutto quello che e' radicato nel figlio di sinistra o destra.



Se faccio la visita IN ORDER (figlio SX, n, figlio DX) di un BST mi restituisce l'elenco ordinato dei suoi elementi.

6.3.1 Ricerca

Uno dei vantaggi dell'BST e' che se devo cercare un elemento x nell'albero NON devo visitarlo tutto, ma posso sfruttare le proprieta' del BST stesso: io parto sempre dalla radice, k e' il target della mia ricerca ed x e' la radice. Faccio un ciclo while e controllo se k e' minore o maggiore del nodo in cui sono, se x e' minore della radice cerco nel sottoalbero sinistro, altrimenti cerco nel sottoalbero destro e poi procedo iterivamente. Nel caso peggiore devo scorrere l'altezza dell'albero, perche' sto andando a cercare la foglia piu' lontana dalla radice, quindi il costo computazionale sara' $O(h)$.



```

TREE-SEARCH( $x, k$ )  /*  $k$  TARGET RICERCA,  $x$  NODO CORRENTE */
    IF  $x = \text{NULL}$  OR  $k = x.\text{KEY}$ 
        THEN RETURN  $x$ 
    IF  $k < x.\text{KEY}$ 
        THEN RETURN TREE-SEARCH( $x.\text{LEFT}, k$ )  /*  $k < x.\text{KEY}$  */
    ELSE RETURN TREE-SEARCH( $x.\text{RIGHT}, k$ )  /*  $k > x.\text{KEY}$  */

```

6.3.2 Operazioni comuni

In un BST abbiamo un set di operazioni comuni:

- Ricerca del minimo: e' il nodo piu' a sinistra
- Ricerca del massimo: e' il nodo piu' a destra

```

TREE-MINIMUM( $x$ )  /* SCENDE A SINISTRA */
    WHILE  $x.\text{LEFT} \neq \text{NULL}$  DO
         $x = x.\text{LEFT}$ 
    RETURN  $x$ 

TREE-MAXIMUM( $x$ )  /* SCENDE A DESTRA */
    WHILE  $x.\text{RIGHT} \neq \text{NULL}$  DO
         $x = x.\text{RIGHT}$ 
    RETURN  $x$ 

```

6.3.3 Ricerca del successore di un nodo n

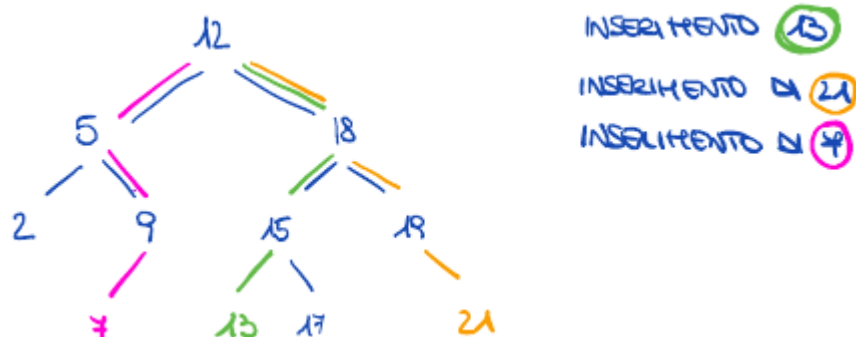
So per certo che e' il valore minimo nel sotto albero destro (perche' e' il piu' piccolo tra i valori piu' grandi di n) e cerco il minimo. In generale e' il valore minimo nel sotto albero di destra, il motivo e' che il successore e' il piu' piccolo tra i valori piu' grandi.

$\text{TREE_SUCCESSOR}(x)$ ^{PUNTIATORE} $O(h)$
 $O(h)$ $\left[\begin{array}{l} \text{IF } x.\text{RIGHT} \neq \text{NULL} \text{ /* CERCO IL VALORE PIÙ PICCOLO NEL SOTTOALBERO DESTRO */} \\ \text{THEN RETURN TREE_MINIMUM}(x.\text{RIGHT}) \end{array} \right.$
 $O(h)$ $y = x.\text{PARENT}$ /* y E' GENITORE DI x */
 $O(h)$ $\left[\begin{array}{l} \text{WHILE } y \neq \text{NULL AND } x = y.\text{RIGHT DO} \\ \quad x = y \\ \quad y = y.\text{PARENT} \end{array} \right. \rightarrow \text{RISALGO NEW ALBERO}$
 RETURN y

Nota: la ricerca, il MAX, il MIN, il successore, il predecessore costano sempre $O(h)$. Perché nel caso peggiore percorrono il cammino in T e la lunghezza massima di un cammino in T è h .

6.3.4 Inserimento in BST

L'inserimento deve mantenere l'ordinamento, per garantire che l'albero rimanga BST. Per garantire questa cosa l'inserimento avviene in posizione di foglia (il nuovo elemento sarà una foglia dell'albero). Funziona come una ricerca solo che al posto di ritornare l'errore che il valore non esiste ma in quel punto faccio l'inserimento.



$O(h)$ **TREE_INSERT**(T, z) /* T è l'albero, z è l'elemento da inserire */

$O(1)$ $y = \text{NULL}$ /* variabile per tenere il genitore di z */

$z = T.\text{ROOT}$ /* variabile per scorrere un path int */

$O(h)$ **WHILE** $z \neq \text{NULL}$ **DO** /* trovo il genitore di z */

$y = z$

IF $z.\text{KEY} < z.\text{KEY}$

THEN $z = z.\text{LEFT}$

ELSE $z = z.\text{RIGHT}$ → visita di T cercando $z.\text{KEY}$

$O(1)$ $z.\text{PARENT} = y$ /* lega z al suo genitore int */

IF $y = \text{NULL}$ /* lego z a suo padre */

THEN $T.\text{ROOT} = z$ $O(1)$

ELSE IF $z.\text{KEY} < y.\text{KEY}$

THEN $y.\text{LEFT} = z$ $O(1)$

ELSE $y.\text{RIGHT} = z$ $O(1)$

6.3.5 Cancellazione in BST

Come per l'inserimento devo mantenere le proprietà del BST. Per prima cosa, cerco il valore da cancellare

1. Il nodo z da rimuovere è FOGLIA: cerco z e sostituisco il nodo con uno vuoto (NULL)
2. Il nodo z da rimuovere ha un solo figlio (SX o DX): cerco il nodo z e sostituisco l'unico figlio di z a z (collego direttamente il genitore di z a suo figlio).
3. Il nodo z da rimuovere ha entrambi i figli: cerco il nodo z e sostituisco il valore di z con quello del suo SUCCESSORE (o PREDECESSORE in modo consistente) e cancello il nodo dove c'era il successore, il nodo successore è foglio o ha un solo figlio, e' il minimo nel sottoalbero destro di z .



```

DELETE(x, z) /* z NOVO VISITATO (ROOT ALL'INIZIO), z IL VALORE DA
              RIMUOVERE */
  IF x ≠ NULL
  THEN
    IF z < x.KEY THEN DELETE(x.LEFT, z)
    ELSE IF z > x.KEY THEN DELETE(x.RIGHT, z)
    ELSE IF (x.LEFT = NULL) AND (x.RIGHT = NULL) THEN x = NULL
    ELSE IF x.LEFT = NULL THEN x = x.RIGHT
    ELSE IF x.RIGHT = NULL THEN x = x.LEFT
    ELSE x.KEY = DELETE-MIN(x.RIGHT)
  END IF
  RETURN x

```

DETAILED EXPLANATIONS:

- DELETE(x, z):** Recursiva che rimuove il nodo con chiave z dall'albero binario di ricerca x.
- IF x ≠ NULL:** Se l'albero è vuoto, non c'è nulla da rimuovere.
- IF z < x.KEY THEN DELETE(x.LEFT, z):** Se la chiave da rimuovere è minore della chiave del nodo corrente, si cerca nella sottoalbero sinistro.
- ELSE IF z > x.KEY THEN DELETE(x.RIGHT, z):** Se la chiave da rimuovere è maggiore della chiave del nodo corrente, si cerca nella sottoalbero destro.
- ELSE IF (x.LEFT = NULL) AND (x.RIGHT = NULL) THEN x = NULL:** Il nodo corrente è una foglia e non ha figli. Se è la chiave da rimuovere, lo si cancella.
- ELSE IF x.LEFT = NULL THEN x = x.RIGHT:** Il nodo corrente non ha figlio sinistro. Si sostituisce il nodo con il suo figlio destro.
- ELSE IF x.RIGHT = NULL THEN x = x.LEFT:** Il nodo corrente non ha figlio destro. Si sostituisce il nodo con il suo figlio sinistro.
- ELSE x.KEY = DELETE-MIN(x.RIGHT):** Il nodo corrente ha entrambi i figli. Si trova il minimo nel sottoalbero destro (che è il successore di x) e si cancella quel nodo. Il valore del minimo viene restituito e sostituisce la chiave del nodo corrente.

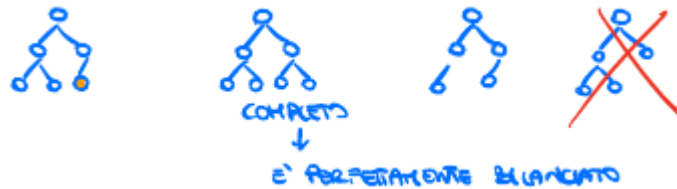
DELETE-MIN(x): Funzione che cancella il minimo di un albero binario di ricerca.

- IF x.LEFT = NULL:** Se il nodo non ha figlio sinistro, è il minimo. Si restituisce la sua chiave e si cancella il nodo.
- THEN MIN = x.KEY:** Il minimo è la chiave del nodo corrente.
- x = x.RIGHT:** Si sostituisce il nodo con il suo figlio destro.
- ELSE MIN = DELETE-MIN(x.LEFT):** Se il nodo ha un figlio sinistro, si cerca il minimo nel sottoalbero sinistro.
- RETURN(MIN):** Restituisce la chiave del minimo.

Il costo è $O(h)$, perché nel caso peggiore devo visitare il cammino massimo dell'albero.

6.3.7 Bilanciamento perfetto

Per ogni nodo in T il numero di nodi nel sottoalbero di sinistra di n differisce al più di 1 rispetto al numero di nodi nel sottoalbero destro di n .



Garantire il bilanciamento perfetto costa troppo per le operazioni di inserimento e cancellazione. Quindi per mantenere ragionevoli i costi di inserimento e cancellazione usiamo definizioni alternative (meno forti) di bilanciamento che garantiscono comunque $h = O(\log_2 n)$.

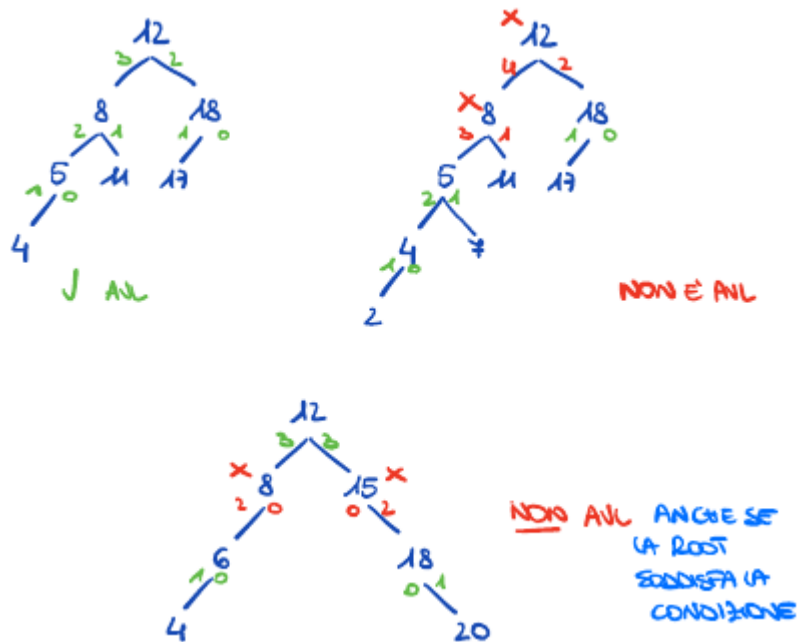
Esempi di bilanciamento alternativo:

- Basati su rotazioni:
 - Alberi AVL
 - Alberi rosso-neri
 - Treap
- Split/Merge: nodi che contengono diversi valori e hanno più figli fondo/-divido i nodi per avere sempre tutte le foglie allo stesso livello. Ad esempio B-TREE, B+TREE, 2-3...

6.4 Alberi AVL

Sono alberi particolari che mi danno una caratteristica di ordinamento, due figli destro (valori piu' grandi del nodo) e sinistro (valori piu' piccoli del nodo). Riusciamo a fare ricerche confrontando il valore che voglio trovare con il nodo e se e' piu grande vado a destra e se piu' piccolo a sinistra.

Gli alberi AVL sono bilanciati in altezza, (sono chiaramente dei BST). Il criterio di bilanciamento: per ciascun nodo X dell'albero T l'altezza del sottoalbero di sinistra e di destra differisce al piu' di 1, qualsiasi altra condizione non e' consentita, attenzione che questa proprieta' deve valere per ciascun nodo all'interno dell'albero e non solo per la radice. E' stato dimostrato a livello teorico che un albero AVL non e' mai piu' del 45% piu' alto di un albero perfettamente bilanciato contenente gli stessi valori, vado quindi a perdere il 45% di prestazioni ma nonostante cio' gli alberi AVL sono piu' facili da tenere bilanciati (con inserimenti ed eliminazioni).



Per quanto riguarda le foglie la proprieta' del bilanciamento e' ovviamente soddisfatta, dato che non hanno figli. Per capire se soddisfa la proprieta' si parte dalle foglie e nodo per nodo si sale vedendo l'altezza del ramo sinistro e destro di ogni nodo. Se facendo questa operazione troviamo una differenza tra le altezze che non e' 1, possiamo dire che l'albero non e' AVL.

Le operazioni viste fino ad ora per gli alberi ABL si applicano allo stesso modo con gli alberi AVL. Quello che cambia dal punto di vista della caratteristica importante e' come funzionano gli inserimenti e le cancellazioni, perche'

potrebbe essere che l'altezza dell'albero potrebbe cambiare. Ad esempio nell'inserimento inseriamo sempre nella posizione foglia, quindi anche i sottoalberi (e l'albero) aumenta di altezza, potrebbe essere che rimanga buono il bilanciamento oppure no. Quindi se opero con un albero AVL devo garantire che le operazioni di inserimento e cancellazione non rovinino la proprieta' dell'albero, quindi hanno una fase di ribilanciamento dell'albero, tramite operazioni di rotazione che riorganizzano i nodi localmente.

Le operazioni di Insert e Delete possono causarne lo sbilanciamento perche' il posto in cui fare l'inserimento o la cancellazione di un nodo non lo posso decidere io, ma dipende dal suo valore rispetto a quelli gia' nell'albero (dalla loro relazione d'ordine).

Quindi per ristabilire il bilanciamento AVL si utilizzano le operazioni di rotazione. Si distinguono in due classi:

- Rotazioni singole
- Rotazioni doppie (sono una composizione di rotazioni singole)

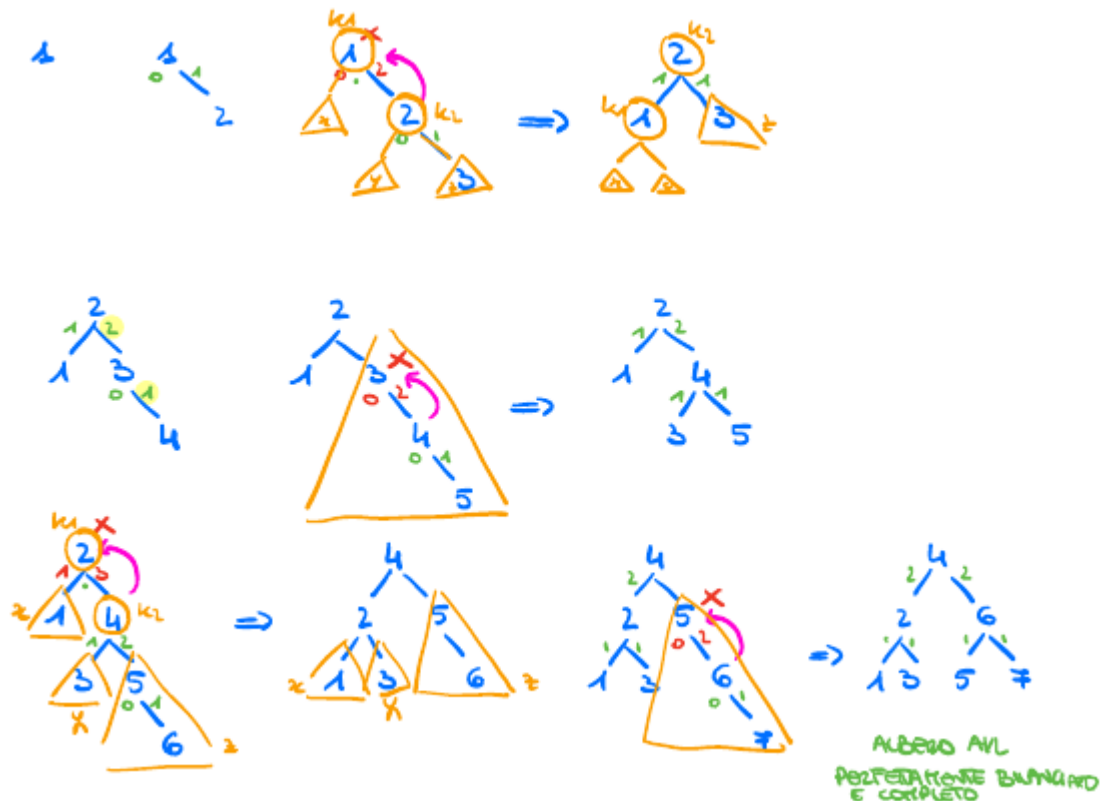
6.4.1 Rotazione Singola



Partiamo da un albero (a sinistra) che e' BST, dove K1 e K2 sono valori, mentre x, y e z sono sottoalberi (quindi avranno una loro altezza). Una rotazione singola che va a ruotare in senso orario, chiamata rotazione destra, opera su K1 e K2 principalmente portandosi dietro i sottoalberi x, y e z come side effect della rotazione.

Immaginiamo come se l'albero fosse fatto da blocchetti tondi e a triangoli collegati dalle catenelle (archi), immaginiamo a mettere un chiodino sotto K2 e adesso tirare la catenelle in senso orario, succede quindi che K1 rimane in alto e K2 scende verso il basso diventando un figlio destro di K1. z che era attaccato al lato destro di K2 rimane come figlio destro. Quello che succede e' che x rimane attaccato a K1 sul lato sinistro, y era attaccato a K1 ma adesso K1 ha gia' due figli, viene attaccato come figlio sinistro di K2.

L'altezza di x e' diminuita di 1, e' successo che l'altezza di y e' rimasta invariata e' che l'altezza di z e' aumentata di 1, quindi la rotazione destra serve nel caso in cui il ramo piu' pesante che sbilancia l'albero e' il ramo x . La rotazione sinistra e' esattamente come la rotazione destra ma fatta al contrario. E' quella che porta K2 ad essere genitore di K1. Mi va bene se il ramo che voglio accorciare e' z . Le rotazioni singole mi cambiano l'altezza di x e z di uno, non cambia pero' l'altezza di y .



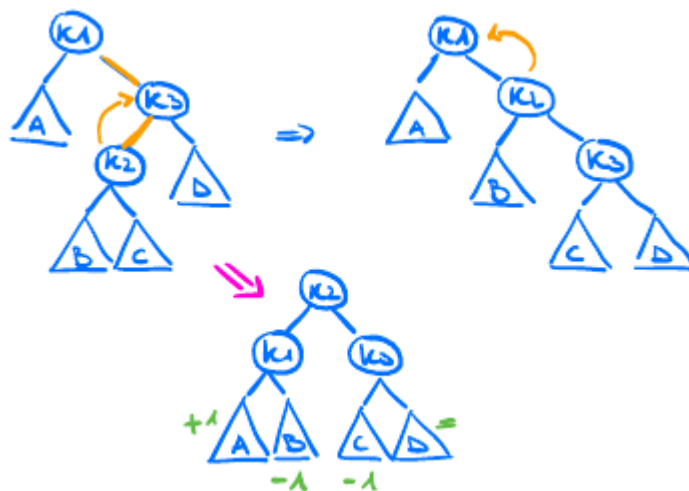
Costruisco un albero AVL inserendo i valori nell'ordine dall'1 al 7 in ordine. Notiamo che arrivando al 3 l'albero non e' piu' AVL, quindi devo fare una rotazione a sinistra perche' devo togliere 1 al ramo di destra, voglio spostare il 2 nella posizione dell'1 in modo da avere i rami uguali. Con il 4 tutto a posto lo metto a destra del 3. Il 5 andra' a destra di 4, ho uno sbilanciamento perche' l'altezza di 3 e' 02, quindi faccio una rotazione a sinistra non toccando pero' i nodi superiori al 3. Il 6 viene inserito alla destra di 5, arrivando a controllare la proprieta' del bilanciamento noto che la radice 2 ha altezze 1 e 3, quindi non rispetta la proprieta' del bilanciamento, il ramo piu' pesante e' quello di destra, quindi devo fare una rotazione a sinistra mettendo un chiodino sotto il 4, l'unica cosa che devo sistemare e' il 3, perche' il 4 non puo' avere piu' di 2 figli, quindi

lo metto dall'altra parte e diventa figlio destro di 2. Il 7 viene inserito a destra di 6, sul 5 ho uno sbilanciamento 0 e 2, faccio quindi ancora una rotazione a sinistra.

Dimentichiamoci quello che non e' soggetto a rotazione e riscriviamolo subito.

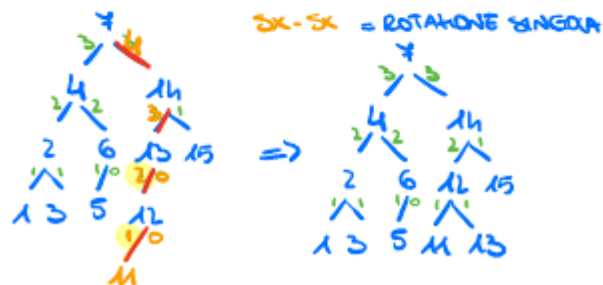
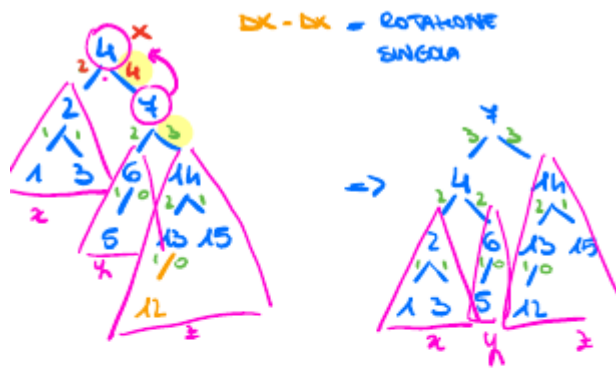
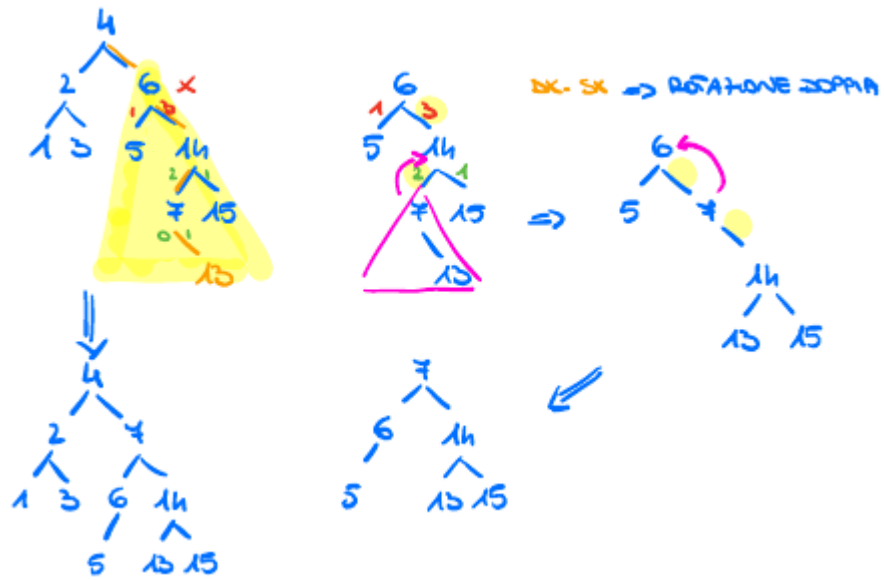
6.4.2 Rotazione doppia

La rotazione doppia e' la composizione di due rotazioni singole. La prima rotazione sposta a destra K2. La seconda rotazione mi porta K2 in alto al posto di K1 e rimette a posto tutto il bilanciamento.



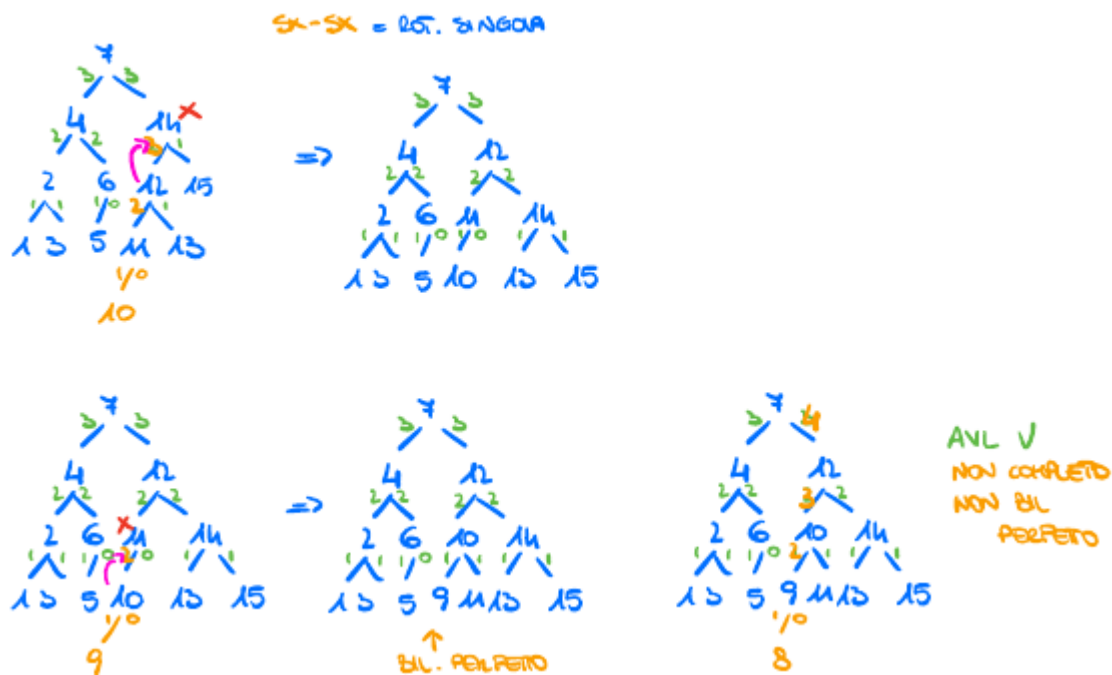
Le rotazioni singole si usano quando lo sbilanciamento e' "concorde" vuol dire che lo sbilanciamento dato il nodo sbilanciato lo trovo sempre andando nella stessa direzione. La rotazione doppia si applica quando lo sbilanciamento e' "discordo" cambio direzione per trovare il ramo piu' lungo.

Inseriamo i valori da 13 a 8 in ordine decrescente:



6.4.3 Inserimento di v in T

Opero come in un BST, potrebbe cambiare l'altezza dei nodi lungo il cammino dove ho fatto l'inserimento (da x alla ROOT). L'altezza di questi nodi aumenta al più di uno. La rotazione accorcia di uno l'altezza dell'albero più alto. Quando inserisco un nodo x in T risalgo il cammino da x alla radice, se trovo uno sbilanciamento lo correggo con la rotazione, mi fermo quando ho la garanzia che l'albero rispetti il bilanciamento AVL dopo la prima rotazione.



6.4.4 Cancellazione

1. Eseguo la cancellazione classica del BST
2. Partendo dal nodo rimosso fisicamente (nodo o suo successore), controllo il bilanciamento dei nodi lungo il cammino fino alla ROOT. Se un nodo è sbilanciato faccio la rotazione.

Nota: non mi fermo alla prima rotazione, potrei sbilanciare il padre quando ruoto.

Esempio: Rimuovere il nodo 12:

6.4.5 Costo delle operazioni su AVL

- Rotazione di un nodo: $O(1)$



- Inserimento: inserimento BST + rotazione + scorrere il path: $O(h) = O(h) + O(1) + O(h) = O(\log N)$
- Cancellazione: cancellazione BST + rotazioni lungo il path + scorrere il path: $O(h) = O(h) + O(h) + O(h) = O(\log N)$

Notiamo che $h = O(\log N)$ perché AVL è bilanciato.

6.5 B-TREE e B+TREE

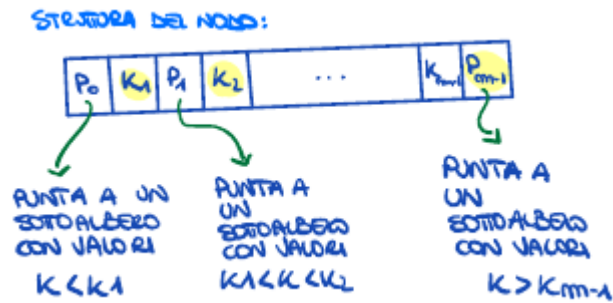
Sono alberi di ricerca bilanciati, tipicamente hanno un numero elevato di figli (grado branching factor) per mantenere basso l'albero. Un albero B di grado m (fan-out) è tale per cui:

- La radice è foglia o ha almeno 2 figli
- Ogni nodo interno (non radice) ha fra $\lceil \frac{m}{2} \rceil$ e m figli
- Ogni cammino dalla radice ha una foglia a la stessa lunghezza, ovvero le foglie sono tutte allo stesso livello

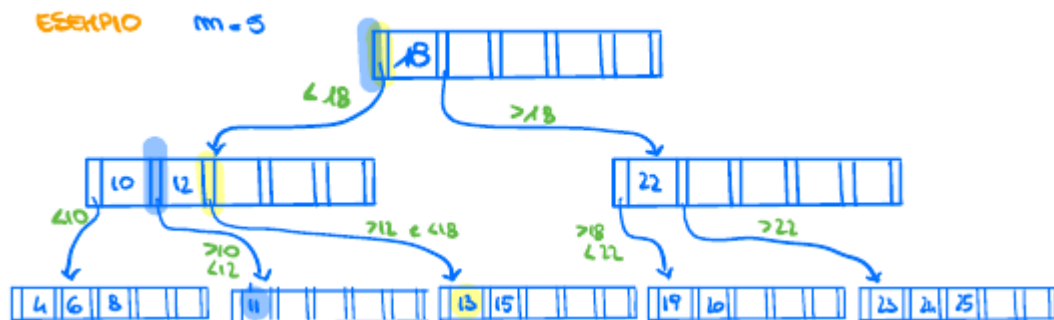
Struttura del nodo:

Caratteristiche del nodo:

- $k_1 \dots k_{m-1}$ sono valori ordinati : $k_1 < k_2 < k_3 < \dots < k_{m-1}$



- $P_0, P_1, \dots, P_{m-1}$ sono i puntatori ai figli
- I nodi foglia non hanno puntatori
- Se un nodo ha x valori ha $x+1$ figli



6.5.1 Ricerca in B-TREE

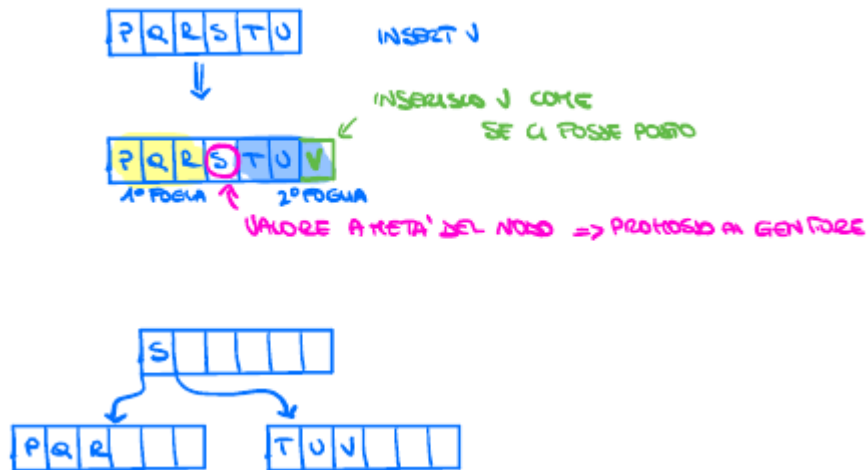
Scorro la lista dei valori in n cercando T :

- Se trovo v mi fermo
- Altrimenti mi fermo alla prima chiave k tale che $v < k$; proseguo la ricerca per P_{i-1} che è il sottoalbero con i valori fra k_{i-1} e k_i .
- Se arrivo alla foglie e non trovo v , vuol dire che v non è nell'albero.

Il costo Dello scorrimento della lista di valori in n , quindi $O(m)$, dato che visito un cammino con al più h nodi il costo è $O(h \cdot m)$, quindi il costo in definitiva è $O(m \cdot \log N)$, dove N è il numero di valori in T .

6.5.2 Inserimento

1. Cerco il valore di v nell'albero
2. Quando arrivo alla foglia:
 - Se la foglia non e' piena: inserisco v mantenendo l'ordinamento fra i valori
 - Se la foglia e' piena: split:
 - Spezzo la foglia in due e promuovo il valore a meta' nel genitore
 - Se il genitore e' pieno : split
 - Se split della ROOT: $h = h+1$



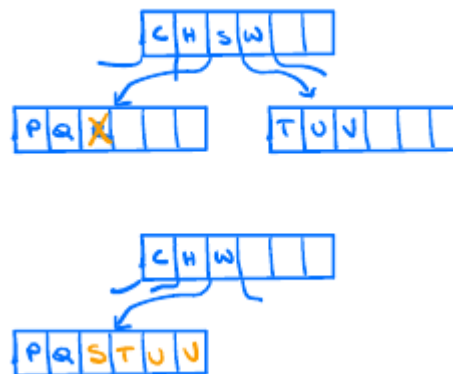
6.5.3 Cancellazione

1. Cerco il valore di v da cancellare in T
2. Se trovo v in un nodo interno lo sostituisco con il suo successore (o predecessore), si trova una foglia. Notiamo che la cancellazione finisce sempre a livello di foglia.
3. Se il nodo foglia e' sotto soglia (ha meno di $\frac{m}{2}$ valori) lo devo sistemare tramite il bilanciamento

Bilanciamento: chiedo a un nodo sibling di cedermi un valore:



Merge: se non posso bilanciare perche' manderei soto soglia i sibling, li unisco in una sola foglia insieme al valore nel padre che le divide. Si puo' propagare al genitore fino alla ROOT, $h = h-1$.



Chapter 7

Insiemi

E' un tipo di dato astratto definito come collezioni di elementi dello stesso tipo, gli elementi sono chiamati membri dell'insieme e ciascun membro e' esso stesso un insieme, quindi non e' nient'altro che un gruppo di elementi dello stesso tipo. Un insieme di un solo elemento e' detto atomo, due copie dello stesso elemento nell'insieme non si trova mai. Il tipo di base sue cui e' definito l'insieme e' caratterizzato da ordinamento totale, quindi dato un elemento posso dire se e' piu' grande o piu' piccolo di un altro sempre appartenente all'insieme, come negli insiemi matematici.

Gli elementi di solito sono di tipo di base, ma potrebbero anche tipi complessi, la cosa fondamentale e' che anche i tipi complessi devono avere una relazione d'ordinamento lineare sul dominio dell'insieme. Una relazione di ordinamento lineare definita su un insieme S e' tale per cui $\forall a, b \in S \Rightarrow a < b$ oppure $a > b$ oppure $a = b$.

Operazioni su insieme S :

- UNION(A,B,C): $C = A \cup B$
- INTERSECTION(A,B,C): $C = A \cap B$
- DIFFERENCE(A,B,C): $C = A - B$ ($C = A/B$)
- MERGE(A,B,C): Se $A \cap B \neq 0 \Rightarrow C = A \cup B$
- MEMBER(x,A): True Se $x \in A$, False altrimenti
- INSERT(x,A): Inserisce x in A (Se x non appartiene ad A)
- DELETE(x,A): Rimuove x da A
- MIN(A): restituisce l'elemento piu' piccolo in A
- MAX(A): Restituisce l'elemento piu' grande in A

7.0.1 Implementazione come Bit-Vector

Nel momento in cui un insieme e' definito su un dominio che e' relativamente piccolo ed ogni elemento del dominio puo' essere mappato ad un numero intero, la cosa piu' naturale da fare e' usare un vettore dove ogni elemento puo' assumere 0 o 1 ed ogni posizione rappresenta un elemento del nostro insieme universo. Per fare questa operazione e' importante anche che gli elementi del dominio siano facilmente mappabili a numeri interi $1 \dots N$ dove N e' il numero di elementi del dominio.

In un bit vector l' i -esimo elemento rappresenta la presenza o l'assenza dell'elemento i del dominio che stiamo andando a rappresentare.

$$s[i] = 0 \Rightarrow i \notin S$$

$$s[i] = 1 \Rightarrow i \in S$$

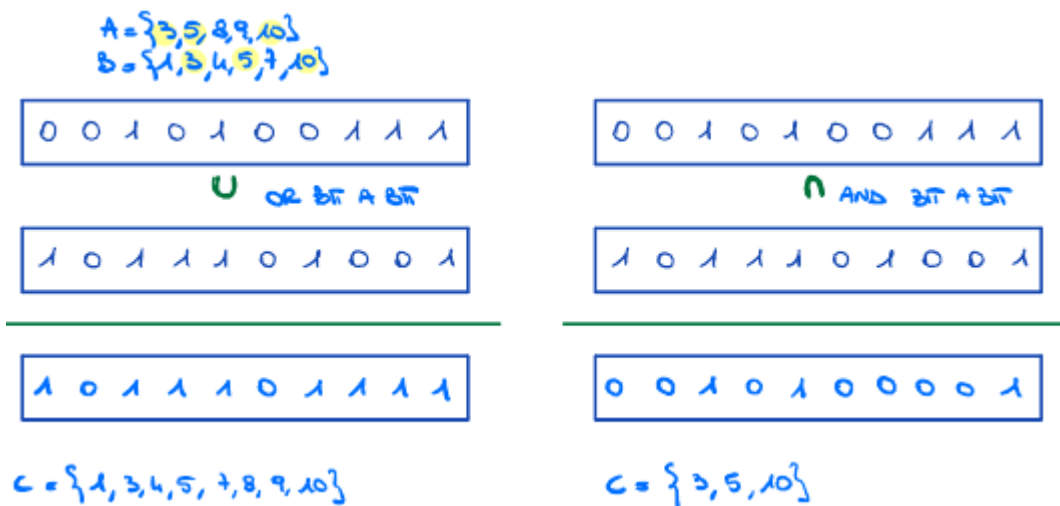
Operazioni con costo $O(1)$:



UNIVERSO = $\{1, 2, 3, \dots, 10\}$
 $S = \{3, 5, 8, 9, 10\}$

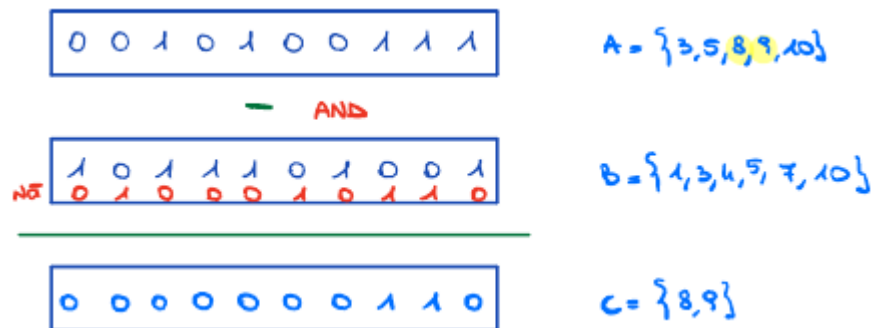
- $\text{MEMBER}(x, A)$: $A[x] = 1$ TRUE $S[5]$
- $\text{INSERT}(x, A)$: $A[x]=1$ $\text{INSERT}(7, 5)$
- $\text{DELETE}(x, A)$: $A[x]=0$ $\text{DELETE}(5, 5)$

Le operazioni di unione, intersezione e differenza posso tradurle in operazioni bit a bit sulle stringhe dei bit-vector ed in particolare l'AND (logico) mi rappresenta l'intersezione, l'OR (logico) rappresenta l'unione. Mentre la differenza la ottengo negando l'AND. Ed hanno tutte costo $O(N)$, dove N e' la dimensione dell'insieme universo.



L'ultima operazione che dobbiamo fare e' l'operazione di differenza, se un elemento appartiene ad A e B va tolto, mentre se appartiene solo ad A o B va tenuto. Quello che facciamo e' negare con un NOT l'insieme B e poi fare un AND tra gli insiemi, cosi' ci torna tutti gli elementi che appartengono anche a B oltre che ad A, mentre gli elementi che appartengono solo ad A sopravvivono.

Queste ultime operazioni hanno costo $O(N)$ dove N e' la dimensione dell'insieme universo, il problema di questa implementazione e' quando l'insieme universo diventa grande, non ha piu' senso perche' dovrei definire vettori enormi e poi usarne pochi elementi. L'implementazione che si preferisce in questi casi e' quella con liste linkate.



7.1 Implementazione come liste linkate

Gli elementi dell'insieme sono rappresentati come elementi di una lista implementata con i puntatori. I vantaggi rispetto all'implementazione tramite bit-vector sono principalmente due:

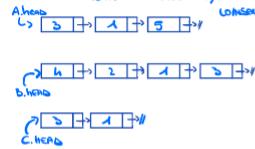
- Lo spazio occupato, perché nel bit vector è proporzionale all'insieme universo, mentre in questo caso è proporzionale agli elementi effettivi dell'insieme (elementi di S e non di U).
- Non è necessario conoscere a priori l'insieme universo U .

Le liste possono essere realizzate come liste ordinate o non ordinate, nelle liste viste fino ad ora non prendiamo nessuna assunzione sull'ordine in cui gli elementi compaiono nella nostra lista, quando una lista è ordinata invece gli elementi compaiono nella sequenza in modo ordinato (crescente o decrescente). Chiaramente ci sono vantaggi e svantaggi nel gestire liste ordinate, ad esempio nell'inserimento se non mi interessa l'ordine posso farlo con costo costante mettendo l'elemento in coda o in testa, mentre se è ordinata devo vedere dove inserire l'elemento. Il vantaggio delle liste ordinate è che avendo l'elemento in un determinato ordine nel trovare un valore posso fermarmi al primo valore più grande del numero che sto cercando. Con le liste ordinate è più facile rappresentare gli insiemi, in modo che le operazioni sugli insiemi siano più facili da realizzare. Quindi da un lato pago un po' di più per mantenere la lista ordinata, dall'altro ne traggio un vantaggio perché le altre operazioni sono più semplici.

Quindi per effettuare in modo efficiente unione, intersezione e differenza preferisco le liste ordinate.

LISTA NON ORDINATA

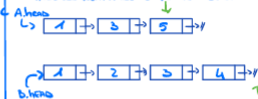
INTERSEZIONE = SCORRO A E POSIZIONAMENTO
X IN A LO CUIOS IN B, SE C'ESTO



COSTO $\rightarrow$ SCORRO IN WORD Moltiplicato
A e B
SE A e B SONO UNICHE $O(m)$
PAGO $O(m^2)$

LISTA ORDINATA

INTERSEZIONE = SCORRO IN PARALLELO LE DUE LISTE
FACENDO AVANTAGE IL PIÙ PICCOLO



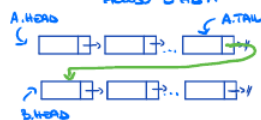
$a_1 = 1 = b_1 = 1 \rightarrow$ INSERISCI IN C E JACO
AVANTI SU A E B

$a_1 = 1 < b_1 = 2 \rightarrow$ PORTO AVANTI B

$a_1 = 1 > b_1 = 1 \rightarrow$ INSERISCI IN C E JACO
AVANTI SU A E B

$a_1 = 2 > b_1 = 1 \rightarrow$ PORTO AVANTI B

UNIONE = SE $A \cap B = \emptyset$
COSTO $O(n)$
ALCORSO B AD A

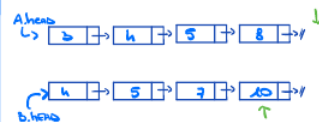


SE NON SO SE $A \cap B = \emptyset$
 $\Rightarrow$ COLAGE IN B OGNI ELEMENTO DI A
PER EVITARE DUPLICATI
 $O(m^2)$

DIFFERENZA = COLLO OGNI ELEMENTO X DI A
IN B E LO ELIPETO IN C SOLO SE
NON LO TROVIA B

COSTO $\rightarrow$ SCORRIMENTO NELLA LISTA
 $O(m)$

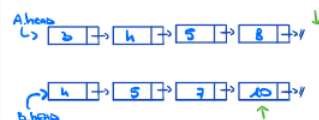
UNIONE = DEVO TENERE LE LISTE ORDINATE
 $\Rightarrow$ SCORRIMENTO PARALLELO
COSTO $O(m)$



$3 < 4 \rightarrow A+$
 $4 = 4 \rightarrow$ INSERISCI C
 $5 = 5 \rightarrow$ INSERISCI C
 $8 > 7 \rightarrow B+$
 $8 < 10 \rightarrow A+$

COSTO $\rightarrow$ SCORRO UNA SOLA VOLTA
OGNI LISTA $O(m)$

DIFFERENZA = SCORRIMENTO PARALLELO
COSTO $O(m)$



MAX/MIN = SCORRERE LA LISTA $O(m)$ MEMBER = SCORRERE LA LISTA CERCANDO x $O(m)$	MAX/MIN = MAX ULTIMO ELEMENTO $O(1)$ MIN PRIMO ELEMENTO $O(1)$ MEMBER = SCORRERE LA LISTA E MI FERMO AL PRIMO VALORE $\geq x$ COSTO $O(m)$ IN MEDIA MEGLIO RESTA LISTA
--	--

7.1.1 Intersezione

Nell'intersezione delle liste ordinate l'idea e' di scorrere contemporaneamente le liste, se scorrendo le due liste trovo due elementi uguali porto avanti entrambi i puntatori, nel caso in cui gli elementi sono diversi porto avanti solo il puntatore della lista che ha il valore minore e lo porto avanti fino a che non trovo che l'elemento di quella lista sia uguale o piu' grande del valore dell'altra lista, in questo caso paghero' $O(n)$, perche' scorro una volta sola le liste. Mentre nell'intersezione con liste non ordinate devo scorrere per ogni elemento della lista A tutta la lista B, con costo $O(n^2)$.

Quindi notiamo che con le liste ordinate sto risparmiando notevolmente.

7.1.2 Unione

La questione e' un po' piu' facile, posso avere due situazioni, se so che gli insieme non hanno nessun elemento in comune allora il fatto che la lista sia non ordinata mi da' un vantaggio perche' posso accodare una lista all'altra senza dover modificare niente ed il costo sara' $O(1)$. Se non so che l'intersezione tra A e B e' nulla devo cercare in B gli elementi di A per evitare i duplicati, quindi il costo sara' $O(n^2)$.

Mentre con le liste ordinate dato che devo tenere le liste ordinate non posso fare altro che fare ancora uno scorrimento parallelo ed il costo sara' ancora $O(n)$. La particolarita' e' che quando ho sono arrivato alla fine di una lista posso prendere tutto quello che rimane nell'altra lista e metterlo nel risultato.

7.1.3 Differenza

Funziona come l'intersezione, quindi vado sempre a fare una doppia ricerca. Nel caso delle liste non ordinate cerco ogni elemento x di A in B e lo riporto in C solo se NON lo trovo in B, quindi il costo e' lo scorrimento nidificato (come l'intersezione) $O(n^2)$.

Nel caso di liste ordinate scorro le liste in parallelo e se l'elemento x di A non compare in B lo riporto, altrimenti vado avanti. Ancora una volta il costo e' $O(n)$.

7.1.4 MAX/MIN

Con le liste non ordinate devo scorrere tutta la lista, quindi costo $O(n)$.

Con le liste ordinate l'elemento piu' grande sara' in fondo e quello piu' piccolo in testa (dipende dall'ordine con cui le facciamo), quindi il costo sara' $O(1)$.

7.1.5 Member

Se la lista non e' ordinata devo scorrere tutta la lista, quindi costo $O(n)$.

Con la lista e' ordinata devo scorrere la lista ma posso fermarmi al primo valore che e' maggiore o uguale ad x . Il costo e' $O(n)$, all'atto pratico pero' visitero' meta' lista.

7.1.6 Codice Inserction

Ho le liste A e B che sono l'input e devo calcolare C che e' l'intersezione tra A e B . Quindi come prima cosa creo una lista C vuota, ho due puntatori $A.current$ e $B.current$ che tengono traccia delle due liste, all'inizio sono posti alla testa delle liste. Finche' i puntatori sono diversi da $NULL$ (ci sono ancora elementi da valutare) confronto i valori puntati dalle liste, se sono uguali metto l'elemento nella lista C e faccio avanzare entrambi i puntatori di scorrimento. Se invece il valore che sto guardando non e' uguale faccio scorrere il puntatore dell'elemento piu' piccolo.

L'unica differenza con l'unione e la differenza e' che l'operazione di inserimento sara' in posizione diversa.

```

INTERSECTION(A, B)    /* C = A ∩ B */
    C.HEAD = NEW(ELEMENT) /* CREA UNA C NUOVA */
    A.CURRENT = A.HEAD
    B.CURRENT = B.HEAD } PUNTORI CHE SCORRONO A E B
    WHILE A.CURRENT ≠ NULL AND B.CURRENT ≠ NULL DO
        IF A.CURRENT.KEY = B.CURRENT.KEY
        THEN
            ALLOCATE(C.NEXT)
            IF C = NULL THEN C.HEAD = C.NEXT
            (C.NEXT).KEY = A.CURRENT.KEY
            C = C.NEXT
            /* INSERISCE IL VALORE IN C */
            A.CURRENT = A.CURRENT.NEXT
            B.CURRENT = B.CURRENT.NEXT
            /* AVANZA IL PUNTATORE DI SCORRIMENTO DI A E B */
        ELSE IF A.CURRENT.KEY < B.CURRENT.KEY
        THEN A.CURRENT = A.CURRENT.NEXT /* PORTO AVANTI A */
        ELSE B.CURRENT = B.CURRENT.NEXT /* PORTO AVANTI B */
    C.NEXT = NULL
  
```

Chapter 8

Dizionari

E' un tipo particolare di insiemi che supporta in particolare le operazioni di member, insert e delete, quindi le implementazioni vista per gli insiemi vanno bene, pero' esiste un'implementazione particolare per i dizionari per rendere efficienti le operazioni di inserimento, cancellazione e membership anche se l'insieme universo e' grosso rispetto all'insieme utilizzato, Questa implementazione e' quella tramite funzioni di hash, i vantaggi:

- Non richiede assunzioni sull'insieme universo
- Nel caso medio, le operazioni di INSERT, DELETE e MEMBER costano $O(1)$. Nel caso peggiore costano $O(K)$ dove K e' il set di valori rappresentato dalla tabella (solo se costruisco male la tabella di hash).
- Riduce lo spazio di memorizzazione a $\Theta(|k|)$. In generale $|u| \gg |k|$.

Se costruisco bene la tabella, il caso medio si verifica quasi sempre.

Un altro vantaggio delle tabelle di hash e' che riduce tantissimo lo spazio di memorizzazione che e' proporzionale all'insieme di elementi dell'insieme degli elementi che vado ad inserire (c'e' un po' di spazio vuoto ma e' poco).

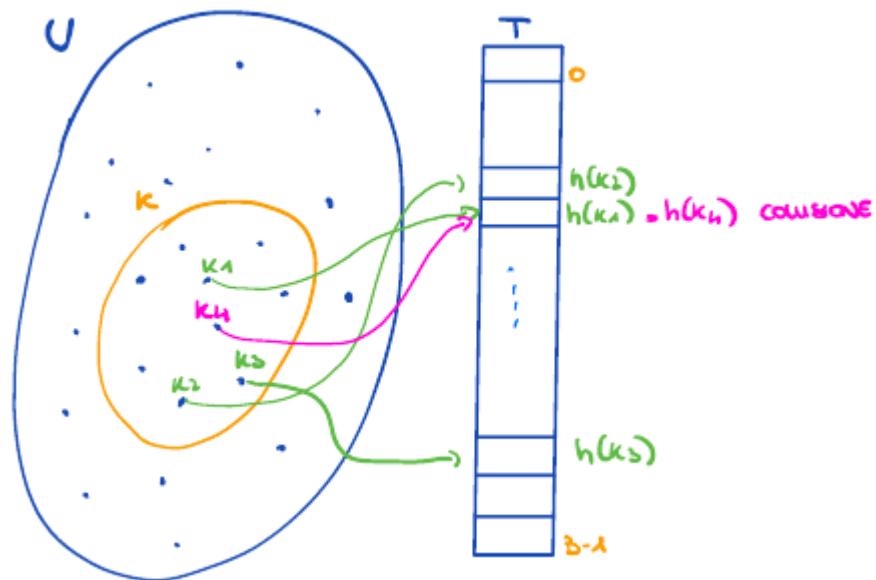
8.1 Hash

La tabella di hash e' una struttura dati che memorizza gli elementi dell'insieme (chiavi o valori) ed e' realizzata tramite un vettore T che ha dimensione B che tipicamente e' leggermente sovradimensionati rispetto al numero di elementi che mi aspetto nel vettore, in genere un elemento con chiave(valore) k viene mappato/memorizzato nello slot in posizione $h(k)$. La seconda cosa che devo avere e' la funzione di hash che e' l'elemento che mi collega gli elementi alla tabella, quindi io applico la funzione al valore dell'elemento ed il risultato e' un numero che va da 1 al numero degli elementi nel vettore, ed il numero mi dice qual'e' la posizione dell'elemento, il valore ritornato di chiama BUCKET, e' una situazione che causa collisione capita che piu' elementi dell'insieme vengono mappati allo stesso BUCKET. $h : U \rightarrow \{0, \dots, B - 1\}$.

L'ideale sarebbe evitare le collisioni, ma dato che la cardinalita' di U e' molto maggiore di quella di B non e' possibile farlo. Posso pero' cercare di ridurle definendo bene la funzione H (di hash) il che vuol dire rendere la funzione di hash il piu' possibile casuale quindi non troppo legata al valore che sto andando a mappare.

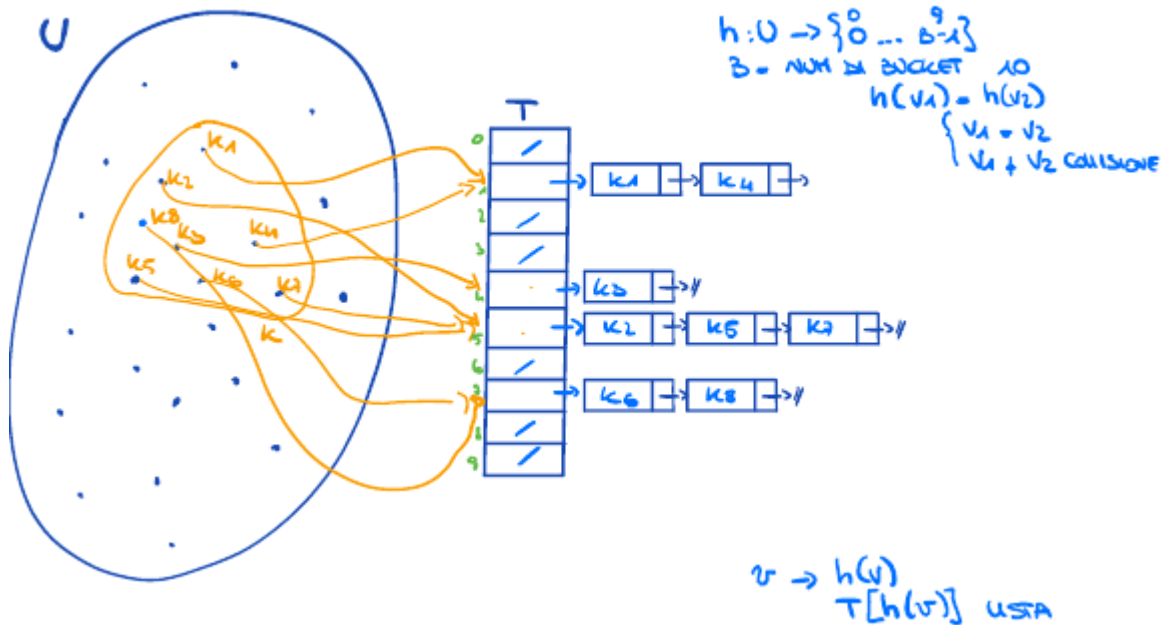
Uso inoltre un metodo per risolvere le collisioni:

- Open Hashing: sono disposto a memorizzare i dati che collidono in spazio aggiuntivo utilizzando liste di collisione
- Closed Hashing: non uso spazio aggiuntivo, ma ricolloco l'elemento che ha causato la collisione (il secondo che si e' mappato) tramite una sorta di re-hashing.



8.1.1 Open Hashing

Risolve le collisioni con il chaining, i valori che collidono sullo stesso valore di hash formano una lista. Ho una tabella di hash che è un'insieme di teste di liste in cui memorizzo i valori, man mano che ho delle collisioni aggiungo in coda alle liste gli elementi. Quindi i valori che coincidono sullo stesso valore di hash formano una lista.



Quando cerco v :

- Calcolo $h(v)$ e mi restituisce un valore b , costo $O(1)$
- $T[h(v)]$ lista da scorrere cercando v , costo $O(n)$ con n lunghezza della lista

Quanto e' lunga la lista di collisione:

- Caso migliore = 0 o 1 elemento, costo $O(1)$
- Caso peggiore = tutti i valori nel dizionario collidono nello stesso bucket, quindi con m chiavi ho una lista lunga m quindi il costo e' $O(m)$
- Caso medio (il piu' frequente se h e T sono costruite bene), ricordiamo che il numero di bucket e' B , supponiamo di avere m chiavi inserite, parliamo di load factor di una tabella di hash per dire il numero medio di chiavi in una tabella di hash, quanto e' lunga la lista di collisione, circa $\alpha = \frac{m}{B}$ che potrebbe essere minore di 1 se la tabella e' piu' grande rispetto ai valori che ho inserito, uguale a 1 se i valori sono uguali o maggiore di 1 se ci sono piu' valori da inserire, nell'ultimo caso ho sicuramente collisioni. Se la funzione di hash e' fatta bene e distribuisce uniformemente le chiavi nei bucket la probabilita' di un elemento di essere mappato in un bucket e' $\frac{m}{B}$, di conseguenza il costo e' $O(\frac{m}{B})$ ma se il numero B di bucket e' proporzionale (dello stesso ordine di grandezza) al numero di chiavi m ($B = O(m)$) allora $\alpha = \frac{m}{B} = \frac{m}{O(m)} = O(1)$. Concludendo nel caso medio

il costo e' costante $O(1)$ (attenzione che vale solo se la funzione di hash e' fatta bene e distribuisce in modo uniforme i valori nei bucket). Buona norma e' sovrastimare la dimensione della tabella di un 20%

Funzione di hash

L'obiettivo e' quello di distribuire in modo uniforme i valori di chiave nei bucket. E' chiaro che se sapessi in anticipo quali sono i valori che devo andare a mappare posso costruire una funzione che mappa in modo ideale i valori da mettere nella tabella pero' nessuno mi dice quali sono i valori da inserire nella tabella, devo quindi essere il piu' possibile generico. Diamo prima un'assunzione, le chiavi devono essere dei numeri naturali, se non lo fossero devo mappare le chiavi in numeri naturali.

Un tipico esempio di funzione di hash che dato un valore di chiave compreso tra 0 e 1, la funzione calcola il valore di k moltiplicato per il numero di bucket B e ritorna il valore assoluto $h(k) = |k \cdot B|$. Se k e' distribuito in modo uniforme tra 0 e 1 questa e' ottima.

Metodi:

- Metodo della divisione: mappa il valore k al resto di una divisione per il numero B di bucket $h(k) = k \text{ MOD } B$. Il vantaggio e' che e' molto semplice, lo svantaggio e' che la scelta di B e' critica, non dobbiamo scegliere potenze di 2, inoltre dovrebbe essere un numero primo. Esempio: $B = 12$, $k = 100$, $h(k) = 100 \text{ MOD } 12 = 4$.
- Metodo della moltiplicazione: prevede che $h(k)$ sia il risultato dell'arrotondamento per difetto di B che moltiplica k per A modulo 1. A e' un valore costante compreso tra 0 e 1 scelto in modo che distribuisca meglio le chiavi e B scelto in modo arbitrario. $h(k) = \lfloor B(k \cdot A \text{ MOD } 1) \rfloor$

8.1.2 Closed Hashing

Calcolo l'hash del valore e trovo la cella, se e' occupata ne trovo una libera, lo faccio in modo deterministico tramite due funzioni di hash che mi danno una probe sequence che mi da tutte le celle che devo scorrere per trovare una cella libera.

Non usa strutture esterne per memorizzare gli elementi (tutti i valori sono inseriti in un bucket in tabella T). Il load factor (e' il numero medio di valori che si mappa su una cella) $L \geq 1$ (altrimenti non ho spazio).

Per tenere traccia di dove sono memorizzati i valori e uso un metodo rehashing per gestire le collisioni, uso una sequenza dove un valore viene potenzialmente inserito, quindi ogni volta che ho una collisione vado avanti nella sequenza stessa. Quindi in sostanza se ho una collisione cerco un'altra cella in T dove memorizzare v . Devo farlo in modo deterministico altrimenti le ricerche non funzionano piu'. La probe sequence e' una sequenza di posizioni all'interno della tabella dove metto il valore in caso di collisioni. Per una chiave k la probe sequence e' la permutazione di valori da 0 a $B-1$ (nel caso ideale si tratta di una

permutazione, include tutte le possibili posizioni nella tabella in ordine diverso), che corrisponde all'ordine in cui valuto lo slot in cui memorizzare la chiave k in caso ci sia stata una collisione.

La funzione di hash dato k ed il numero di collisioni osservate restituisce il bucket, $h : U \times \{0, \dots, B-1\} \rightarrow \{0, \dots, B-1\}$. Quindi in sostanza la probe sequence non e' nient'altro che la valutazione della funzione di hash per k fino a che non ho riscontrato $B-1$ collisioni.

Inserimento

A livello logico calcolo la funzione di hash per k e zero collisioni, mi restituisce un indice vado a vedere nella tabella T e vedo se quella cella e' vuota, altrimenti valuto la funzione di hash per $k, 1$, se l'indice che mi restituisce e' vuoto assegno il valore alla cella, altrimenti vado avanti aumentando di uno il numero di collisioni. Mi fermo quando ho trovato una posizione vuota oppure sono arrivato alla fine della probe sequence e non sono riuscito a trovare un posto libero.

```

HASH_INSERT( $T, k$ ) /* INSERISCO CHIAVE  $k$  NELLA TABELLA  $T$  */
 $i = 0$  /* NUM COLLISIONI */
REPEAT
     $j = h(k, i)$  /* POSIZIONE */
    IF  $T[j] = \text{NULL}$  /* BUCKET VUOTO */
        THEN  $T[j] = k$ 
        RETURN( $j$ )
    ELSE  $i = i + 1$ 
UNTIL  $i = B$ 
RETURN ERROR

```

Mediamente il costo dell'inserimento e' costante, perche' mi aspetto di avere poche collisioni.

Ricerca di K

Procedo come l'inserimento, valuto la funzione di hash per il valore di k e successivamente se non e' nella posizione restituita dalla funzione guardo nella probe sequence andando avanti a valutarla finche' non trovo k o trovo un blocchetto vuoto.

```

HASH - SEARCH(T, k)
  i = 0      /* NOT COLLISION! */
  REPEAT
    j = h(k, i)
    IF T[j] = k THEN RETURN(j)
    i = i + 1
  UNTIL T[j] = NULL OR i = B
  RETURN (NULL)

```

$\hookrightarrow k \notin T$ HO GUARDATO TUTTA LA PROBE SEQUENCE (TUTTA LA TABELLA)

Cancellazione di k da T

Cerco k in T , come abbiamo appena fatto per la ricerca, non sostituisco k con NULL (perche' altrimenti interrompo la probe sequence e non funzionano piu' le ricerche per i valori), quindi uso un valore ad-hoc che se cerco e lo trovo vuol dire che devo continuare a scorrere, mentre se devo inserire un valore posso sovrascrivere quel valore.

8.1.3 Uniform Hashing

La probe sequence di ogni chiave ha uaugale probabilita' di essere una delle $B!$ possibili permutazioni degli indici, le soluzione che vediamo non sono uniformi ($B!$) ma sono buone (B^2)

Linear Probing

Si basa sul fatto che ho una funzione di hash ausiliaria (che dato un numero di chiave mi restituisce un numero tra 0 e $B-1$). Dice che se c'e' una collisione

guarda la cella che c'è dopo, quando arriviamo infondo alla tabella ripartiamo dalla posizione 0, come se la tabella fosse chiusa su se stessa.

funzione di hash: $h(k, i) = (h'(k) + i) \text{MOD } B$ e lo faccio da i che va da 0 a $B-1$.

ESEMPIO $h'(x) = x \text{ mod } B$ $h(x, i) = (h'(x) + i) \text{ mod } B$
 $B = 10$ $K = 89, 18, 49, 58, 69, 79$

									89
0	1	2	3	4	5	6	7	8	9

$$h'(89) = 89 \text{ mod } 10 = 9$$

$$h(89, 0) = 9 + 0 \text{ mod } 10 = 9$$

								18	89
0	1	2	3	4	5	6	7	8	9

$$h'(18) = 18 \text{ mod } 10 = 8$$

$$h(18, 0) = 8$$

49								18	89
0	1	2	3	4	5	6	7	8	9

$$h'(49) = 49 \text{ mod } 10 = 9$$

$$h(49, 0) = 9 \quad \times \text{ collisione}$$

$$h(49, 1) = (9 + 1) \text{ mod } 10 = 0$$

49	58							18	89
0	1	2	3	4	5	6	7	8	9

$$h'(58) = 8 \quad \times \text{ collisione}$$

$$h(58, 0) = 8 \quad \times$$

$$h(58, 1) = 9 \quad \times$$

$$h(58, 2) = 0 \quad \times$$

$$h(58, 3) = 1 \quad \checkmark$$

49	58	69						18	89
0	1	2	3	4	5	6	7	8	9

$$h'(69) = 9 \quad \times \text{ collisione}$$

49	58	69	79					18	89
0	1	2	3	4	5	6	7	8	9

$$h'(79) = 9 \quad \times \text{ collisione}$$

11 collisioni

Il vantaggio di questa tecnica è la facilità di uso ed implementazione. Lo svantaggio è che ho primary clustering, ovvero che se ho collisioni mano mano che aggiungo un elemento aumento il numero di celle contigue piene e quindi le collisioni, inoltre il numero di probe sequence diverse generate è B (dati B

elementi), inoltre se due elementi distinti collidono sullo stesso primo elemento avranno la stessa probe sequence e collideranno su tutti gli altri elementi.

Quadrating Probing

Cerca di evitare il primary clustering, invece che andare alla cella successiva quando ne trovo una piena faccio dei salti piu' lunghi, per fare questi salti utilizzo una funzione che dipende in modo quadratico dal numero di collisioni.

Ho la funzione di hash ausiliaria che funziona sempre allo stesso modo. Mentre la funzione di hash e' quadratica: $h(k, i) = (h - (k) + c1i + c2i^2)$. $c1$ e $c2$ devono essere costanti e $c2$ non puo' essere nulla. Quindi la distanza tra gli elementi nella probe sequence dipende dal quadrato del numero di collisioni i .

Il vantaggio e' che il primary clustering non succede piu', non aumento la larghezza del blocchetto pieno. Svantaggi:

- Il numero di probe sequence generate e' sempre B , perche' se due valori collidono sullo stesso valore iniziale avranno la stessa sequenza
- secondary clustering: se ho tanti valori che vengono mappati allo stesso valore dalla funzione hash ausiliaria (come primo elemento) continuano a collidere come se fossero lo stesso valore

ESEMPIO $h'(x) = x \bmod B$ $B = 10$ $h(x, i) = (h'(x) + i^2) \bmod B$ $K = 89, 18, 49, 58, 69, 79$ $C_1 = 0$
 $C_2 = 1$

i	$+ \Delta$
0	0
1	1
2	4
3	9
4	16

									89
0	1	2	3	4	5	6	7	8	9

 $h'(89) = 9$
 $h(89, 0) = 9 + 0 \bmod 10 = 9$

								18	89
0	1	2	3	4	5	6	7	8	9

 $h'(18) = 8$
 $h(18, 0) = 8$

49								18	89
0	1	2	3	4	5	6	7	8	9

 $h'(49) = 9$ 1 collision
 $h(49, 0) = 9 \times$
 $h(49, 1) = (9 + 1) \bmod 10 = 0 \checkmark$

49		58						18	89
0	1	2	3	4	5	6	7	8	9

 $h'(58) = 8$ 2 collisions
 $h(58, 0) = 8 \times$
 $h(58, 1) = (8 + 1^2) \bmod 10 = 9 \times$
 $h(58, 2) = (8 + 2^2) \bmod 10 = 2 \checkmark$

49		58	69					18	89
0	1	2	3	4	5	6	7	8	9

 $h'(69) = 9$ 2 collisions
 $h(69, 0) = 9 \times$
 $h(69, 1) = (9 + 1^2) \bmod 10 = 0 \times$
 $h(69, 2) = (9 + 2^2) \bmod 10 = 3 \checkmark$

49		58	69	79				18	89
0	1	2	3	4	5	6	7	8	9

 $h'(79) = 9$ 4 collisions
 $h(79, 0) = 9 \times$
 $h(79, 1) = (9 + 1^2) \bmod 10 = 0 \times$
 $h(79, 2) = (9 + 2^2) \bmod 10 = 3 \times$
 $h(79, 3) = (9 + 3^2) \bmod 10 = 8 \times$
 $h(79, 4) = (9 + 4^2) \bmod 10 = 5 \checkmark$

9 collisions

Duple Hashing

Utilizza due funzioni di hash ausiliarie h_1, h_2 . La funzione di hash che genera la probe sequence: $h(k, i) = (h_1(k) + ih_2(k)) \bmod B$. Provo con la posizione T di $h_1(k)$ se e' pieno mi sposto di $h_2(k)$ posizioni, quindi lo spostamento in caso

di collisioni dipende da k .

Il vantaggio è che due valori che collidono sullo stesso primo valore di della probe sequence non avranno la stessa probe sequence (la avranno solo se collidono sul primo e secondo elemento). Inoltre ho un numero di probe sequence pari a B^2 . Lo svantaggio è che le due funzioni di hash ausiliarie h_1 ed h_2 devono essere scelte per coprire tutta la tabella e con collisioni "simili" ad esempio:

- B potenza di 2 e h_2 produce solo numeri dispari
- B primo e h_2 valori più piccoli di B

Ricordiamoci che non vado a mettere l'elemento in posizione h_2 , ma sommo h_2 , moltiplicato per il numero di collisioni, ad h_1 .

ESEMPIO $h_1(x) = x \bmod B$ $h_2(x) = 7 - (x \bmod 7)$ $h(x, i) = (h_1(x) + i \cdot h_2(x)) \bmod B$
 $B = 10$ $K = 89, 18, 49, 58, 69, 79$

									89
0	1	2	3	4	5	6	7	8	9

$$h_1(89) = 9$$

$$h(89, 0) = 9$$

							18	89	
0	1	2	3	4	5	6	7	8	9

$$h_1(18) = 8$$

$$h(18, 0) = 8$$

						49		18	89
0	1	2	3	4	5	6	7	8	9

$$h_1(49) = 9$$

$$h(49, 0) = 9 \quad \times \text{collisione}$$

$$h_2(49) = 7 - (49 \bmod 7) = 7$$

$$h(49, 1) = (9 + 7) \bmod 10 = 6 \quad \checkmark$$

			58			49		18	89
0	1	2	3	4	5	6	7	8	9

$$\begin{aligned}
 h_1(58) &= 8 \\
 h(58, 0) &= 8 \times \text{collision} \\
 h_2(58) &= 7 - (58 \bmod 7) = 5 \\
 h(58, 1) &= (8 + 5 \cdot 1) \bmod 10 = 3 \checkmark
 \end{aligned}$$

69			58			49		18	89
0	1	2	3	4	5	6	7	8	9

$$\begin{aligned}
 h_1(69) &= 9 \\
 h(69, 0) &= 9 \times \text{collision} \\
 h_2(69) &= 7 - (69 \bmod 7) = 1 \\
 h(69, 1) &= (9 + 1 \cdot 1) \bmod 10 = 0 \checkmark
 \end{aligned}$$

69			58	79		49		18	89
0	1	2	3	4	5	6	7	8	9

$$\begin{aligned}
 h_1(79) &= 9 \\
 h(79, 0) &= 9 \times \text{collision} \\
 h_2(79) &= 7 - (79 \bmod 7) = 5 \\
 h(79, 1) &= (9 + 5 \cdot 1) \bmod 10 = 4 \checkmark
 \end{aligned}$$

4 collisione

$$\begin{aligned}
 h(79, 2) &= (9 + 5 \cdot 2) \bmod 10 = 19 \bmod 10 \\
 &= 9
 \end{aligned}$$

Chapter 9

Graf

E' una struttura dati composta da due elementi

- V: insieme dei vertici (nodi) e' un insieme finito, sono in sostanza gli elementi del grafo.
- E: insieme degli archi, ovvero le connessioni che stanno tra i nodi.

A differenza degli alberi abbiamo bisogno di una struttura che ci tiene traccia dei nodi. Due tipi di archi: orientati con freccia ed ordine specifico dei nodi che collega l'arco, non orientati senza freccia e senza ordine specifico dei nodi. Un arco orientato e' una coppia ordinata (v_1, v_2) dove v_1 e v_2 appartengono a V, l'arco e' rappresentato $v_1 \rightarrow v_2$ v_1 e' la cosa, v_2 e' la testa. Esistono archi che si chiudono su se stesso. Un arco non orientato non ha testa e coda.

Un grafo orientato ha tutti gli archi orientati, un non orientato ha archi non orientati.

Un concetto molto importante e' quello di incidenza, In un grafo non orientato se abbiamo un arco v_1v_2 si dice che un arco e' incidente in v_1 v_2 e questi sono nodi vicini

Si dice che c'e' una relazione di adiacenza nel momento in cui c'e' un arco che collega due vertici, nel caso di arco orientato l'adiacenza e' la direzione in cui percorro l'arco, mentre in caso di arco non orientato ho adiacenza simmetrica.

9.1 Grado di un nodo

Consideriamo due gradi per un nodo orientato:

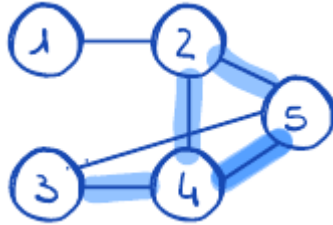
- In-Degree: sono il numero di archi in ingresso di un nodo
- Out-Degree: sono il numero di archi in uscita di un nodo

In generale, per un grafo orientato (e' in degree + out degree, il numero di archi in cui il nodo e' coinvolto).

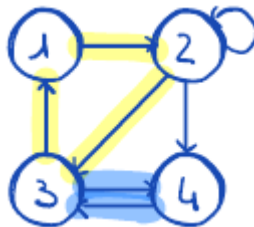
Per i grafi non orientati non abbiamo distinzione tra in ed out perche' possiamo andare in entrambe le direzioni sul grafo.

9.2 Cammino

Un cammino in un grafo e' una sequenza di nodi che sono collegati a coppie tramite un arco. La lunghezza e' uguale al numero di archi (numero di nodi -1), attenzione che il cammino e' una sequenza quindi i nodi devono essere rappresentati in un determinato ordine.



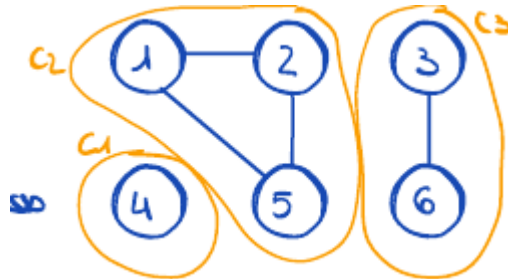
E' detto cammino semplice un cammino di cui tutti i nodi sono distinti, ad eccezione del primo e ultimo in caso di ciclo. Un ciclo e' un cammino semplice che inizia e finisce sullo stesso nodo. Il self loop e' esso stesso un ciclo di lunghezza 1.



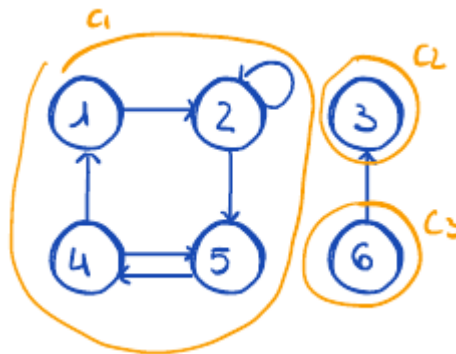
Un grafo orientato senza cicli e' detto semplice. Attenzione che noi abbiamo presente il grafo cosi' com'e' rappresentato, consideriamo che l'implementazione del grafo non e' una rappresentazione visiva, il calcolatore ha solo le informazioni su qual'e' un grafo e qual'e' un nodo.

9.2.1 Connessione

Parliamo di grafo connesso, partendo da un grafo non orientato, e' possibile partendo da qualsiasi nodo all'interno del grafo raggiungere tutti i nodi del grafo (tutti i nodi sono collegati tra di loro). Di fatto esiste un cammino che connette ogni coppia di nodi. Un grafo non connesso e' fatto da varie componenti, nelle quali vale il concetto di raggiungibilita', ma che tra di loro non sono raggiungibili. Se il grafo ha una sola componente e' considerato connesso.

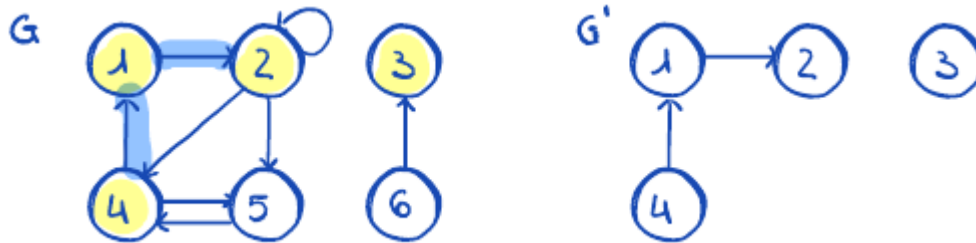


Un grafo fortemente connesso e' un grafo orientato da cui per ogni nodo esiste un cammino per arrivare a qualsiasi nodo del grafo e VICEVERSA (deve valere la condizione di mutua raggiungibilita').

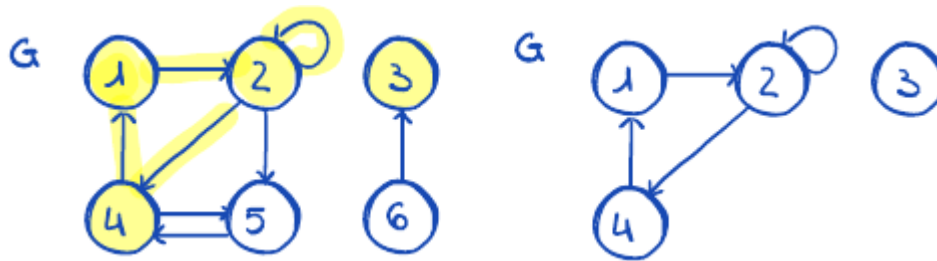


9.3 Sottografo

Un sottografo di un grafo e' un grafo ottenuto considerando un sottoinsieme dei nodi e degli archi del grafo di partenza.



Un particolare tipo di sottografo e' il SOTTOGRAFO INDOTTO, che e' un sottografo che si ottiene selezionando una parte dei nodi perche' gli archi sono quelli che collegano quei nodi nel grafo originale.



9.4 Implementazione dei Grafi

Tipicamente ci sono due strategie per l'implementazione:

- Matrici di adiacenza
- Liste di adiacenza

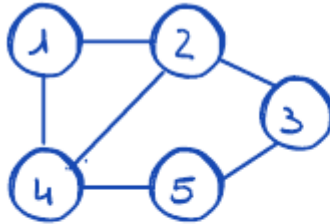
Voglio rappresentare nell'implementazione che due nodi sono collegati da un arco (adiacenza).

9.4.1 Matrice di adiacenza

Creo una matrice con una riga e una colonna per ogni nodo, quindi e' una matrice di dimensione $|V| \times |V|$, dove A se e' la matrice di adiacenza in posizione i,j ho valore 1 se l'arco (i,j) appartiene all'insieme E, 0 altrimenti.

$A[i,j]$ puo' essere anche diverso da 1 ed e' considerato il peso del nodo (si chiama grafo pesato).

Notiamo che se il grafo non è orientato la matrice è simmetrica (perché ogni arco è rappresentato due volte), inoltre notiamo che se il grafo è orientato sulla diagonale troviamo tutti zeri (perché non possono esserci self-loop). Lo spazio occupato è $\Theta(|V|^2)$.



SE G È NON ORIENTATO
 A È SIMMETRICA:

$$A[i, j] = A[j, i]$$

SPAZIO OCCUPATO DALLA MATRICE $\Theta(|V|^2)$

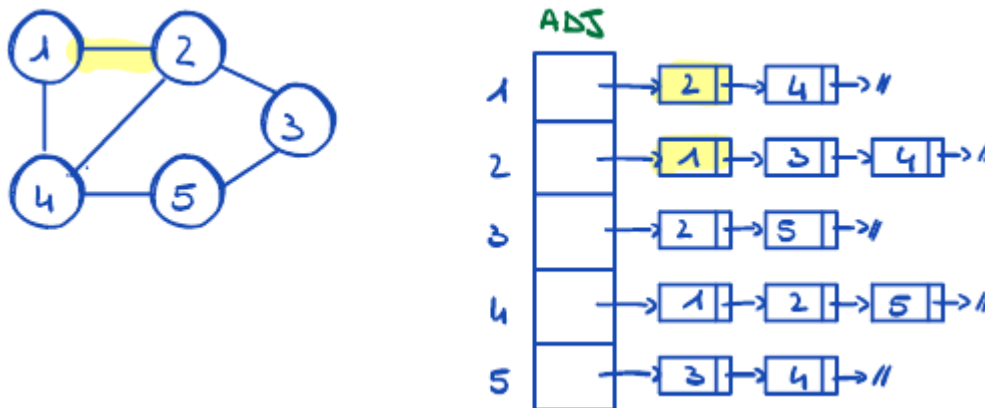
	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	1	0
3	0	1	0	0	1
4	1	1	0	0	1
5	0	0	1	1	0

Liste di adiacenza

Per ogni nodo tengo una lista che tiene traccia dei nodi che sono raggiungibili direttamente da lui (gli archi che escano dal nodo), l'elemento del vettore e' un puntatore alla lista adiacente.

Ho un array con $|v|$ elemento (uno per ogni nodo in v), ciascun elemento e' la lista di nodi adiacenti al nodo v .

Lo spazio occupato: ho un vettore con cardinalita' $|v|$ numero di celle. Ogni blocchetto nella lista mi sta rappresentando un arco (ogni arco lo troviamo due volte se il grafo e' orientato una se e' orientato). Quindi lo spazio occupato dalle liste e' 2 volte la cardinalita' di E (insieme degli archi) se il grafo non e' orientato, $|E|$ se il grado e' orientato. Lo spazione complessivo lo posso rappresentare come $\Theta(|v| + |E|)$.



Pro e contro implementazioni

Matrice	Lista
Determinare se $(n, v) \in E$ costa $O(1)$	Determinare se $(n, v) \in E$ costa $O(v)$, devo scorrere lista
Consuma piu' spazio $ v ^2 > E $	Occupano spazio proporzionale $ E $
Ok se grafo e' denso	Ok se grafo e' sparso

9.5 Algoritmi di visita del grafo

Se siamo capaci di attraversare un grafo siamo capaci di costruire algoritmi che sono in grado di modificare l'albero. Voglio trovare un modo sistematico che mi permetta di guardare tutti i nodi in un albero senza andare piu' volte a guardare lo stesso. Se il grafo non e' connesso puo' succedere che la visita tocchi solo una parte dei nodi perche' gli altri nodi non sono connessi da archi. Ci sono due approcci per la visita:

- Ampiezza: BFS Breadth first search
- Profondita' (no differenze perche' i nodi sono tutti considerati alla pari): DFS Depth First Search

9.5.1 BFS - Ampiezza

Parte da un determinato nodo, ma a differenza degli alberi non abbiamo un punto di partenza univoco (la radice), ma ci viene dato e si chiama nodo sorgente. Da questo nodo sorgente parte la visita e guarda tutti i nodi a distanza 1 del nodo sorgente e man mano da ogni nodo che visita passa a quelli collegati di uno piu' esterno (prima a distanza 1, 2, 3 ...), l'idea e' quella di andare un passo piu' in la' rispetto al punto di partenza.

Ci permette di individuare all'interno di un grafo il cammino piu' corto per arrivare ad un nodo partendo dalla sorgente.

Questo algoritmo lavora su grafi orientati e non tenendo in considerazione l'orientamento degli archi, li seguo quindi nella loro direzione, mi propago solo se l'arco esce dal nodo verso l'esterno, se invece trovo un arco che entra non esco perche' lo percorrerei nella direzione sbagliata.

Dato un grafo $G(V, E)$ orientato o non orientato ed un nodo $S \in V$ sorgente la BFS esplora gli archi in G partendo da S allargandosi "a macchia d'olio". Come side effect calcola la distanza minima (numero di archi) di ogni nodo $v \in V$ dalla sorgente S .

La BFS costruisce un albero, BFS TREE, che contiene tutti i nodi del grafo raggiungibili da S . Il cammino da S a v nell'albero e' quello piu' corto possibile in G .

Partiamo da un nodo, tocchiamo tutti quelli vicini a lui e metto questi nodi all'interno di una coda, perche' dopo voglio vedere i vicini dei nodi a distanza 1 e cosi' via. Potrebbe essere che i nodi a distanza 3 possono essere collegati a nodi piu' lontani o vicini rispetto alla sorgente, nel caso in cui andando avanti trovo un nodo che ho gia' visitato che era piu' vicino alla sorgente rispetto al nodo da cui lo visito non lo metto in coda, dato che l'ho gia' visitato. Devo colorare i nodi:

- Bianchi: nodi che ancora non ho visto
- Grigi: nodi che ho visto ma non ho ancora valutato i vicini
- Neri: nodi che ho visto ed ho anche considerato i vicini, quindi tutto a posto

Algoritmo:

```

BFS( $G, \Delta$ ) /*  $\Delta$  SORGENTE,  $\Delta \in V$  */

 $O(V)$  FOR EACH VERTEX  $w \in G.V - \{\Delta\}$  DO
     $w.COLOR = WHITE$  /* COLORE DEL NODO */
     $w.D = \infty$  /* DISTANZA DA  $\Delta$  */
     $w.P = NULL$  /* PARENT DI  $w$  IN BFS TREE */
] INITIALIZATION

 $O(1)$   $\Delta.COLOR = GRAY$  /* PUNTO DI PARTENZA */
 $\Delta.D = \phi$  /* LUNGHEZZA DEL CAMMINO DA  $\Delta$  A  $\Delta$  */
 $\Delta.P = NULL$  /*  $\Delta$  È LA ROOT */
 $Q = \phi$  /* CODA */

 $O(1)$  ENQUEUE( $Q, \Delta$ ) /* INSERISCE  $\Delta$  IN CODA */

WHILE  $Q \neq \phi$  DO
     $O(1)$   $w = DEQUEUE(Q)$  /* ESTRAE UN NODO DALLA CODA */
    FOR EACH  $v \in G.ADJ[w]$  DO /* GUARDA TUTTI GLI ARCHI CHE TOCCANO  $w$  */
         $O(1)$  IF  $v.COLOR = WHITE$  /*  $v$  È UN NODO MAI VISTO PRIMA */
            THEN  $v.COLOR = GRAY$ 
                 $v.D = w.D + 1$  /* DISTANZA */
                 $v.P = w$  /* LO COLLEGO AL BFS TREE */
                ENQUEUE( $Q, v$ )
         $O(1)$   $w.COLOR = BLACK$  /* NODO VISITATO COMPLETAMENTE */

```

Notiamo che l'albero è potenzialmente diverso a seconda dell'ordine di visita dei nodi adiacenti, ma la distanza da s a v è la stessa (perché è quella minima).

9.5.2 DFS - Profondita

Parto da un nodo di partenza e vado avanti seguendo un cammino, quando raggiungo un punto dove non posso raggiungere altri nodi torno indietro di un passo e vedo se da li posso raggiungere un'altro nodo. Quando ho finito tutti i cammini e torno alla sorgente e ci sono altri cammini che partono da li' posso ricominciare la DFS. Non e' detto che esista un cammino che raggiunge gli stessi nodi ma impiegandoci di meno.

Dato un grafo $G(V, E)$ ed un nodo sorgente $s \in v$. La DFS cerca di esplorare G continuando ad allontanarsi da s piu' che puo', quando non puo' andare piu' avanti torna indietro di un passo e cerca di esplorare altri cammini (sempre guardare nodi ancora non visitati). Quando ha terminato la visita partendo da un altro nodo non visitato se ne sono rimasti.

L'algoritmo costruisce un albero (foresta) di DFS FOREST, e' un insieme di alberi di visita radicati nella sorgente, uno per ogni chiamata. Anche in questo caso abbiamo una serie di variabili da associare a ciascun nodo:

- i colori per capire se ho gia' visitato il nodo:
 - Bianco: non ho ancora guardato, scoperto
 - Grigio: sono gia' passato (l'ho scoperto) ma non ho ancora visitato tutte le ramificazioni
 - Nero: e' stato scoperto ed anche i suoi nodi adiacenti
- parent: da dove sono arrivato
- tempo di inizio visita: in sostanza e' quando v diventa grey
- tempo di fine visita: quando v diventa black

Algoritmo:

Costo

Visto che un nodo non puo' ritornare bianco, quindi non chiamero' mai la DFS due volte sullo stesso nodo, quindi il costo sara' $O(v)$ per l'inizializzazione piu' $O(E)$, $O(|V| + |E|)$. Quindi e' come se andassi a guardare nel caso peggiore tutti gli archi.

Classificazione egli archi - DFS

Vengono classificati gli archi in diverse catergorie che ci permettono di verificare se un grafo e' ciclico o meno.

Facciamo prima delle considerazioni dei tempi di inizio e fine visita. Se il tempo di inizio e quello di fine sono contenuti in due nodi c'e' una relazione di ancestor, se invece i tempi di due nodi sono completamente disgiunti significa che i nodi sono in due rami completamente disgiunti della foresta.

Gli archi possono essere classificati:

DFS(G)

```

FOR EACH VERTEX  $w \in G.V$  DO
     $w.COLOR = WHITE$     /* COLORE BIANCO */
     $w.P = NULL$           /* GENITORE IN DFS-TREE */
} → INIT.

TIME = 0    /* TIMESTAMPA DA USARE PER I NODI */
FOR EACH VERTEX  $w \in G.V$  DO /* CICLO SU DIVERSE POTENZIALI SORGENTI */
    IF  $w.COLOR = WHITE$ 
        THEN DFS.VISIT(G, w) /* DFS CON SORGENTE w */

```

DFS.VISIT(G, w)

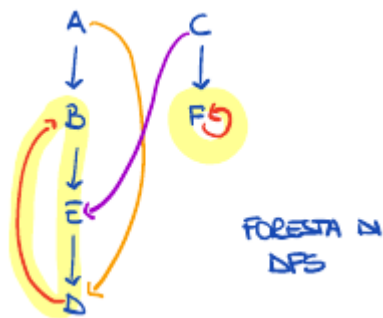
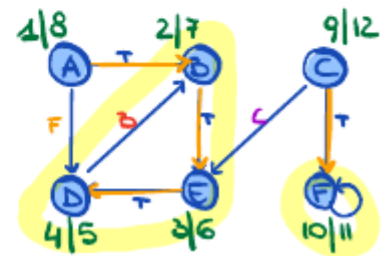
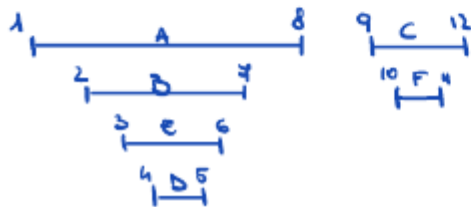
```

TIME = TIME + 1
 $w.D = TIME$     /* TEMPO INIZIO VISITA */
 $w.COLOR = GRAY$  /* INIZIO A GUARDARE AL */
FOR EACH  $v \in G.ADJ[w]$  DO /* GUARDA I NODI ADJACENTI */
    IF  $v.COLOR = WHITE$ 
        THEN  $v.P = w$  /* COLLEGO v AD DFS-TREE */
        DFS.VISIT(G, v) /* CHIAMATA RICORRENTA PER VISITARE v CON DFS */
 $w.COLOR = BLACK$  /* TERMINA LA VISITA DI w */
TIME = TIME + 1
 $w.F = TIME$     /* SEGNO IL TEMPO DI FINE */

```

- Tree: fanno parte della foresta (sono quello che ho percorso durante la visita del grafo)
- Back: sono quegli archi (u,v) dove v e' un antenato di u . Inclusi i self loop
- Forward: sono gli archi (u,v) dove v e' un discendente di u .
- Cross: sono quelli che collegano nodi che fanno parte di cammini di alberi diversi, che quindi non sono in relazione ancestor descendant.

- $[w.d, w.f]$
 $[v.d, v.f]$
- 1) $[w.d, w.f]$ A
 $[v.d, v.f]$ C
 SONO DISGIUNTI
 $\Rightarrow$ DUE RAMI O ALBERI DIVISI
- 2) $[w.d, w.f]$ E
 $\hat{=}$ CONTENUTO IN
 $[v.d, v.f]$ A
 $\Rightarrow w$ DISCENDENTE DI v
- 3) $[v.d, v.f]$ F
 $\hat{=}$ CONTENUTO IN
 $[w.d, w.f]$ C
 $\Rightarrow v$ E' DISCENDENTE DI w



Quello che consiglia se dobbiamo fare l'esercizio e' disegnare la foresta e poi classificare gli archi che non ci sono. Se esiste un arco back all'interno del grafo allora il grafo e' ciclico, questo non cambia anche se guardiamo in ordine diverso i nodi.

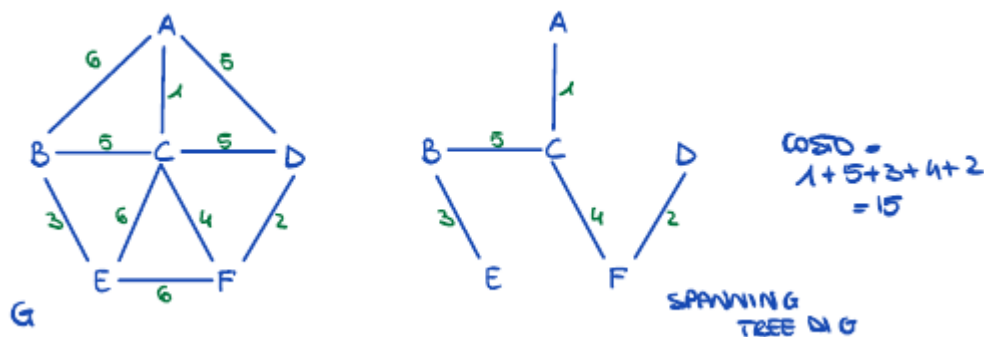
Se il grafo non e' orientato ci sono solo archi Tree e Back, il motivo e' che posso percorrere l'arco in entrambe le direzioni.

La DFS viene usata per l'ordinamento topologico, ho un grafo dove ogni nodo rappresenta una attivita' e la relazione d'ordine mi dice l'ordine in cui devo svolgere questa attivita'. Se faccio una DFS su questo grafo e percorro la foresta al contrario trovo l'ordine in cui devo svolgere le attivita'. Inoltre viene utilizzata per individuare le componenti connesse (se G e' non orientato, basta farlo svolgendo una volta la DFS, nel momento in cui completiamo la prima visita e dobbiamo ripartire vuol dire che non e' connesso. Se il grafo e' orientato dobbiamo fare due volte la DFS con gli archi normale e ribaltati per vedere se il grafo e' connesso). Individuare componenti fortemente connesse (se G e' orientato).

9.6 Minimum Cost Spanning Tree (MST)

Vediamo un altro problema che dobbiamo analizzare sui grafi. E' quello della individuazione dell'albero coprente a costo minimo (MST). Ho un grafo, in particolare non orientato, pesato (ogni arco e' associato ad un valore, si considerano di solito pesi positivi e Reali (noi interi)) e quello che voglio fare e' trovare un albero il cui costo totale e' quello piu' basso possibile per toccare tutti i nodi. E' un albero perche' non ci sono cicli (libero e non radicato) se ho N nodi selezione $N-1$ archi, devo toccare tutti i nodi e a costo minimo, perche' la somma dei costi degli archi selezionati e' quella minima per toccare tutti i nodi.

Il costo di uno spanning tree e' la somma del costo dei suoi archi.



Una nota importante e' che il MST non da informazioni sui cammini tra due nodi all'interno del grafico, non troviamo il costo minimo di un cammino con lo spanning tree, in complesso stiamo minimizzando il costo della rete non del cammino. Nel nostro esempio il cammino minimo da A ad E nell'MST costa 9 (A, C, B, E), mentre nel grafo originale G costa 7 (A, C, E).

Esistono vari modi per risolvere i problemi di ottimizzazione, ma quello usato

da noi e' un algoritmo GREEDY, ad ogni passo della sue esecuzione compie una scelta che al momento sembra la migliore, senza pero' considerare le conseguenze della scelta nei passi successivi. In questo caso essere ottimistici nella scelta attuale e' una buona soluzione per il nostro problema, ma in generale questi algoritmi greedy non garantiscono di trovare la soluzione ottima. Vedremo due algoritmi, entrambi si basano che ad ogni iterazione dell'algoritmo che va a costruire l'MST va ad aggiungere un arco ad ogni passo, quindi il nocciolo della questione e' come scegliere l'arco giusto ad ogni passo (il numero dei passi e' $N-1$ con N nodi). Devo scegliere l'arco in modo che non chiuda cicli e che in quel momento deve essere quello con costo minimo. Mi fermo quando tutti i nodi nel grafo sono connessi ed ho scelto $N-1$ archi.

- Aggiungo un arco alla volta, che dev'essere minimo
- Prima di ogni aggiunta, gli archi che ho sono quelli di MST. Per ogni arco aggiunto deve essere parte dell'MST

In linea generale gli algoritmi saranno fatti cosi':

$A = \emptyset$ /* INSIRE IN ARCHI MST */

WHILE A IS NOT A MST FOR G DO

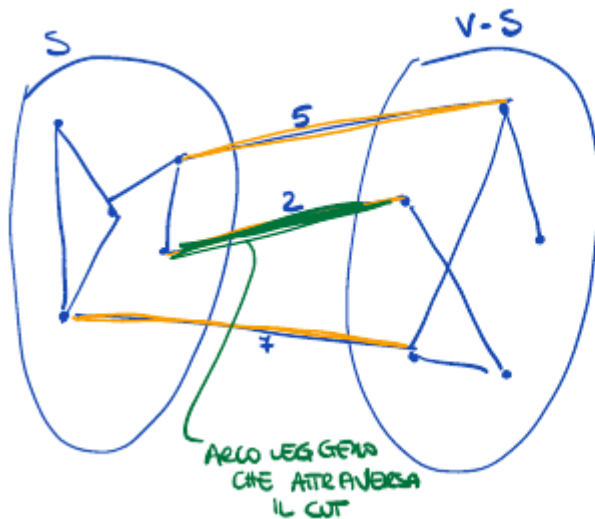
FIND $(u, v) \in E$ THAT SATISFIES THE SAFE EDGE INVARIANT

$A = A \cup \{(u, v)\}$

RETURN A

↓
CONCATENATE ALL
PARTS MST

Come seleziono gli archi che fanno parte dell'MST? Immaginiamo di avere due insiemi di nodi, S ed un insieme che e' il complemento di S : $V-S$. Ciascuno ha al suo interno dei nodi, che sono collegati tra di loro con degli archi. Esistono archi che collegano nodi tra i due insiemi.



GRAFO G

$CUT(S, V-S) =$
PARTIZIONE DEI
VERTICI IN DUE
INSIEMI $(S, V-S)$

(u, v) ATTRAVERSA
IL CUT SE
 $u \in S$ e $v \in V-S$
O VI CEVERSA

Tra tutti gli archi che attraversano il CUT esiste un arco leggero che e' quello con peso minore. Ci interessa perche' se voglio costruire un MST l'arco leggero che attraversa il CUT sicuramente ne fa parte, perche' e' il modo meno costoso per collegare un nodo in S con un nodo in $V-S$.

Gli algoritmi che vedremo funzionano cosi': Parto con l'insieme A di archi vuoto, con un ciclo aggiungo ad ogni passo ad A l'arco leggero che attraversa il CUT, una volta che ho fatto $|V| - 1$ iterazioni mi fermo per non creare un ciclo.

Ci sono due algoritmi per vedere come scegliere l'arco leggero:

- Kruskal: e' incrementale, da un albero con 1 nodo vado ad aggiungere gli altri
- Prim: parte da una foresta con n alberi ciascuno di 1 nodo e man mano li fonde

9.6.1 Kruskal

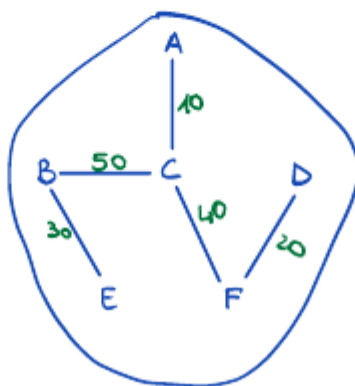
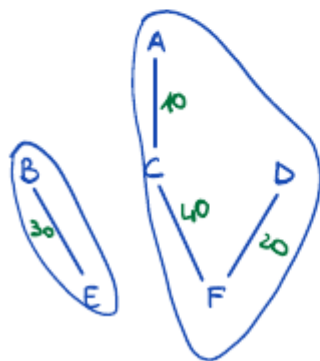
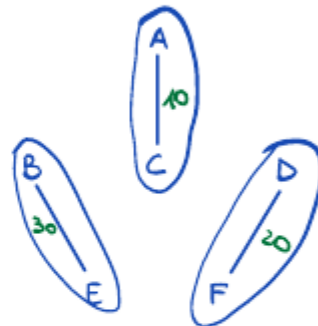
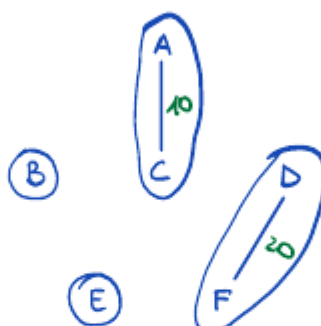
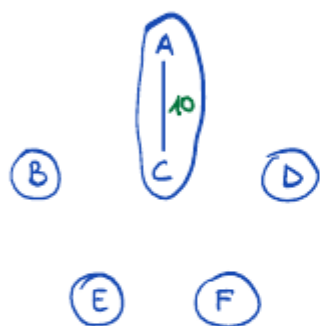
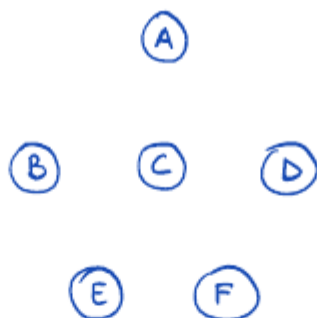
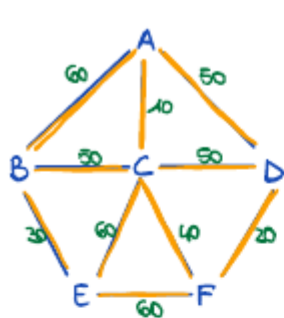
```

HST_KRUSKAL( $G, w$ ) /*  $G$  GRAFO,  $w$  FUNZIONE PESO */
 $O(V)$  {  $A = \emptyset$  /* ARCHI IN HST */
   $O(V)$  { FOR EACH VERTEX  $v \in V$  DO
        MAKE-SET( $v$ ) /* CREO UN INSIEME PER OGNI VERTICE */
   $O(E \log E)$  SORT THE EDGES IN  $G.E$  IN NON-DECREASING ORDER BY WEIGHT  $w$ 
   $O(E)$  { FOR EACH EDGE  $(u, v) \in G.E$  TAKEN IN NON-DECREASING ORDER BY WEIGHT  $w$  DO
        IF FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) /* SE  $u$  E  $v$  APPARTENGONO AD ALBERI DIVERSI */
        THEN  $A = A \cup \{(u, v)\}$ 
        UNION( $u, v$ )
  RETURN  $A$ 
  COSTO  $O(E \cdot \log E)$ 

```

L'idea è che prendo l'arco che costa meno che collega due alberi diversi, partendo da tutti nodi disgiunti (quindi tutti alberi di un elemento).

Esempio



$$\text{Costo} = 10 + 20 + 30 + 40 + 50 = 150$$

9.6.2 PRIM

Anzichè ordinare gli archi complessivamente su tutto il grafo guardo quelli che escono dal perimetro dell'albero che ho già costruito e per ogni passo ne scelgo uno. Ricordiamo che si parte da un nodo e lo considero come albero di partenza da cui scegliere gli archi.

$HST_PRIM(G, w, r)$ /* G GRAFO, w PESO, r RADICE $r \in V$ */

$A = \emptyset$ /* ARCHI HST */

$S = \{r\}$

WHILE $S \neq V$ DO

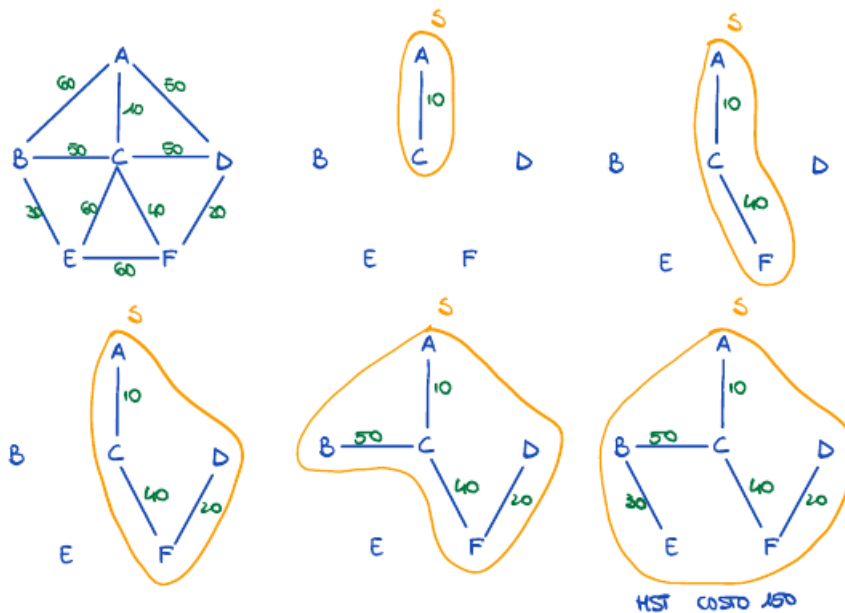
LET (u, v) THE EDGE WITH MINIMUM WEIGHT S.T. $u \in S$ AND $v \in V \setminus S$

$A = A \cup \{(u, v)\}$

$S = S \cup \{v\}$

/* ARCO CHE ATTRAVERSA IL CUT (LEGGERO) */

RETURN A

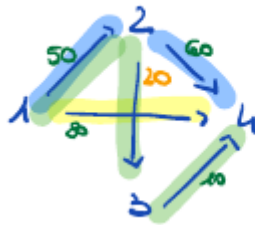


Ha un costo paragonabile con l'algoritmo di Kruskal, ma nell'implementazione migliore e' $O(|E| \log |V|)$. Sono del tutto equivalenti per quanto riguarda il costo.

9.7 Shortest Path

Dato un grafo $G(V,E)$ orientato, con peso dato dalla funzione w definita sull'insieme degli archi ed ha come risultato un numero nei Reali (noi interi). Si basa sull'idea di cercare tutti i cammini che collegano due nodi e prendere quello che costa meno di tutti, il cammino e' dato dalla somma dei pesi degli archi. Il peso del cammino minimo tra due nodi. Il cammino minimo da u a v e' un qualsiasi cammino da u a v in G con peso mino tra u e v $\delta(u,v)$.

ES. CITTA' E STRADE CHE LE COLLEGANO



$$\delta(1,4) = \min(80, 110, 80) = 80$$

$\uparrow$ $\uparrow$ $\uparrow$
 $\langle 1,4 \rangle$ $\langle 1,2,4 \rangle$ $\langle 1,2,3,4 \rangle$

CAMMINO MINIMO $\langle 1,4 \rangle$
 CAMMINO MINIMO $\langle 1,2,3,4 \rangle$

Nell'esempio abbiamo visto tutti i cammini che collegano i nodi 1 e 4, dopo di che abbiamo scelto quello con costo minore. Nel caso in cui il grafo sia non pesato per cammino minimo intendo quello con il minor numero di grafi, non ci servono questi algoritmi, basta la BFS. In relta' potremmo anche usare gli algoritmi si shortest path ma dobbiamo assumere che il peso di tutti gli archi sia uguale ad 1.

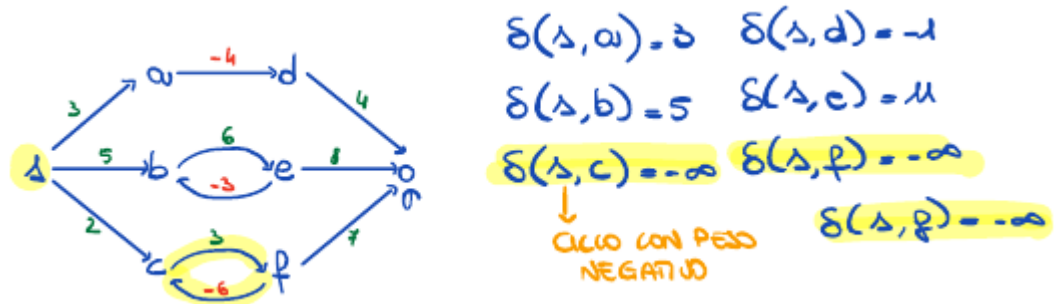
9.7.1 Varianti

- Single source: ho una sorgente e tutti gli altri nodi sono destinazione. voglio trovare d-1 cammini minimi
- Single destination: ho un unico nodo di riferimento e voglio trovare tutti i cammini che partono da qualsiasi altri nodo ed arrivano al nodo di destinazione, per farlo uso un algoritmo di single source invertendo l'orientamento degli archi.
- Single pair: ho una sorgente ed una destinazione, questo problema non viene studiato questo algoritmo nello specifico perche' ha la stessa complessita' del single source, quindi uso sempre il single source.
- All pairs: voglio trovare i cammini minimi tra qualsiasi coppia di nodi all'interno del grafo. Potrei usare algoritmi single source applicati in modo iterativo su ogni nodo, ma esistono anche algoritmi ad hoc piu' efficienti.

9.7.2 Single source shortest path

Abbiamo il grafo orientato, la funzione peso che e' definita sull'insieme dei nodi ed ha valore nei reali ed abbiamo il nodo sorgente. Vogliamo trovare il cammino che va dal nodo sorgente a tutti gli altri nodi del grafo.

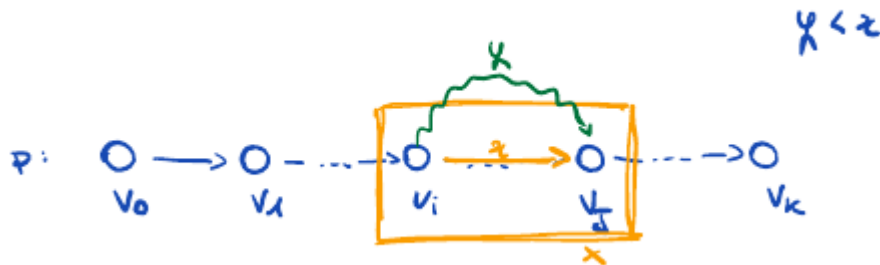
Facciamo prima una considerazione, un ciclo non fara' mai parte del cammino minimo.



Perche' se abbiamo:

- Ciclo con peso maggiore di zero, il cammino aumenta di peso (costo del ciclo), non avremmo piu' il cammino minimo.
- Ciclo con peso zero, in questo caso non mi cambia niente nel peso del cammino, in sostanza quindi non mi interessa perche' e' ininfluente.
- Ciclo con peso minore di zero (negativo): questi cicli non fanno mai parte dei cammini minimi, il problema e' che non riusciamo a priori a dire qual'e' il peso minimo del cammino con un ciclo negativo, perche' percorrendo piu' volte il ciclo vado a ridurre il peso. In sostanza in questi casi e' che il problema non e' ben definito e non posso risolverlo.

Vediamo anche un'altra proprieta', se abbiamo un cammino $p = \langle v_0, v_1, \dots, v_n \rangle$ ed e' minimo, qualsiasi suo sotto cammino $p_{ij} = \langle v_i, \dots, v_j \rangle$ e' minimo a sua volta.



La dimostrazione per assurdo:

Se costruisco il cammino minimo andando a minimizzare tutti i sottocammini che lo compongono trovo il cammino minimo in generale.

9.7.3 Algoritmo di Dijkstra

Funziona solo con archi con peso maggiore o uguale a 0. E' inoltre un algoritmo Greedy, ad ogni passo cerca di scegliere la soluzione che sembra essere la migliore per andare verso l'ottimo, nonostante sia greedy per questo tipo di problemi calcola l'ottimo.

Opera su grafi ORIENTATI e PESATI, con archi con peso POSITIVO (o nullo). Quindi per ogni arco che appartiene all'insieme degli archi, il peso dev'essere maggiore di 0.

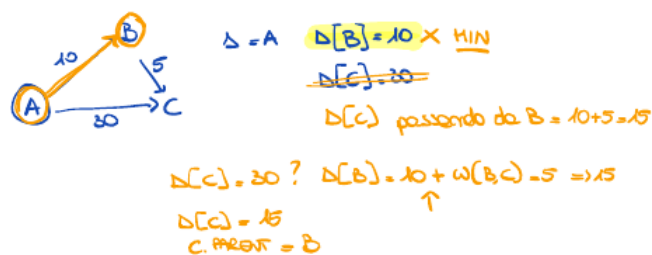
```

DIJKSTRA( $G, w, s$ ) /*  $G$  GRAFO,  $w$  PESO,  $s$  SORGENTE */
 $S = \{s\}$  /* NODI PER CUI HO GIA' CALCOLATO IL CAMMINO MINIMO */
FOR EACH  $v \in V - \{s\}$  DO /* DISTANZA DA  $s$  A  $v$  UGUALE A  $w(s, v)$  */
     $D[v] = w(s, v)$ 
WHILE  $S \neq V$  DO
    CHOOSE  $v \in V - S$  s.t.  $D[v]$  IS MINIMUM /* LA DISTANZA DA  $s$  A  $v$  NON
                                                PUO' ESSERE MIGLIORATA
                                                PASSANDO DA ALTRI NODI */
     $S = S \cup \{v\}$ 
    FOR EACH VERTEX  $u \in G.ADJ[v]$  DO /*  $u$  E' ADIACENTE A  $v$  */
        IF  $D[u] > D[v] + w(v, u)$  /* SE IL CAMMINO CHE ARRIVA A  $u$ 
                                    SI RIDUCE DI PESO
                                    PASSANDO DA  $v$  */
            THEN  $D[u] = D[v] + w(v, u)$ 
             $u.p = v$ 

```

Abbiamo in input il grafo, la funzione peso ed un nodo sorgente (perche' sto cercando tutti i cammini che partono da s ed arrivano agli altri nodi del grafo). Per prima cosa inizializziamo un insieme S che e' l'insieme che tiene traccia di quei nodi che ho gia' calcolato il cammino a peso minimo, perche' costruisce un insieme di cammini e va ad ampliare questo insieme perche' ad ogni passo va a vedere se puo' migliorare il cammino minimo, quindi i nodi in S sono quelli per cui ho gia' calcolato il cammino minimo e quindi saranno quelli che usero' per raggiungere gli altri nodi. Inoltre definisco un vettore D (distanza) che tiene

traccia per ogni nodo della lunghezza del cammino piu' breve che ho individuato fino ad adesso da s al nodo v , quindi inizialmente metto come distanza minima che conosco il peso dell'arco che parte da s ed arriva a v (solo gli archi che escono dal nodo sorgente), occhio che all'inizio il vettore potrebbe contenere dei valori infiniti, perche' ovviamente il nodo sorgente potrebbe non essere collegato con tutti gli altri nodi del grafo. Il ciclo while viene eseguito un numero di volte pari al numero di nodi nel grafo -1, come prima cosa guardiamo tutte le distanze che ho ottenuto dal nodo sorgente a tutti gli altri e prendo quella piu' bassa (perche' e' quella non migliorabile passando da altri nodi). Una volta che ho scelto quello che non e' migliorabile devo capire se il cammino minimo che ho trovato mi puo' essere utile per diminuire i costi degli altri cammini? Guardo quindi tutti gli altri cammini per gli altri nodi e guardo se sommando i cammini utilizzando quello minimo e' minore del cammino minimo che avevamo prima, se e' si aggiorno il cammino minimo, altrimenti lascio quello che c'era e vado avanti. L'algoritmo termina quando ho guardato tutti i cammini di tutti i nodi.

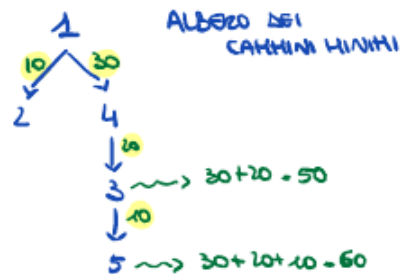
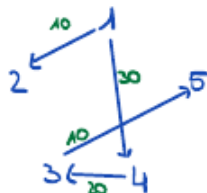
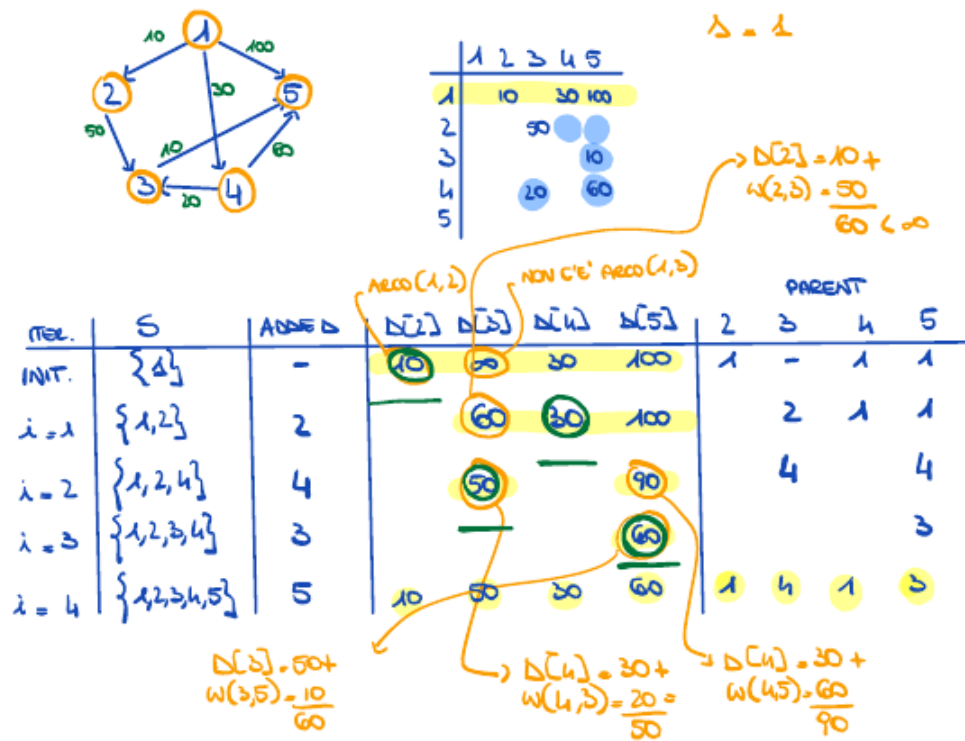


Vediamo ora un esempio di funzionamento dell'algoritmo, quello che consiglia e' di disegnare la matrice di adiacenza del grafo, che ha una riga per ogni nodo ed una colonna per ogni nodo, all'incrocio della riga e colonna ho il peso dell'arco (arco che va da A a B con peso P). Abbiamo un nodo sorgente $s=1$. Poi abbiamo una tabella per risolvere l'algoritmo. La prima colonna e' quella delle iterazioni, S sono i nodi che ho gia' raggiunto come cammino minimo, ADDE D sono i cammini minimi, D[] sono le distanze da ogni nodo, parent invece mi tiene traccia di qual'e' il nodo da cui ho raggiunto quei nodi quando ho stabilizzato il cammino minimo.

Partiamo dall'inizializzazione, sappiamo che S contiene solo il nodo sorgente, non abbiamo nessun nodo aggiunto, D[n] sono pari alla riga del nodo sorgente della matrice di adiacenza, perche' ci metto solo i costi degli archi che partono dal nodo sorgente, in parent mettiamo il nodo tramite cui abbiamo raggiunto i vari nodi.

Successivamente scelgo il cammino minimo e sapendo che non lo raggiungo piu' ci metto una barra, in S metto anche due, perche' tramite quel arco sono arrivato a 2, anche in adde metto 2, ora devo guardare gli archi che escono da due per vedere se per caso sapendo come arrivare a due vedo se posso arrivare agli altri nodi migliorando i costi che gia' conoscevo, nel nostro esempio vediamo che il nodo 2 va solo a 3, e notiamo che la somma dei costi del percorso (1, 2,

3) = 60 e' minore di infinito (quello che sapevamo prima del cammino per 3), quindi aggiorno la colonna dei costi $D[]$ e quella dei parent, perche' so che a 3 ci sono arrivato da 2. A questo punto scelgo ancora il cammino minore 30, che sottolineo e non uso' piu', aumento il numero delle iterazioni, sono ora al nodo 4 quindi aggiorno S aggiungendo 4 ed adde, a questo punto come prima guardo nella tabella di adiacenza tutti i cammini che escono da 4, e facendo i conti di quanto costano i cammini guardo se posso aggiornare qualcosa nella colonna $D[]$, e dei parent. A questo punto ancora una volta scelgo il percorso minore in $D[]$, che e' 50, lo stabilizzo ed aggiorno ancora tutto. Ora non ci sono piu' cammini minimi dato che $S = v$ e mi fermo.



Qualsiasi cammino all'interno dell'albero dei cammini minimo e' quello a costo minore, in sostanza contiene i cammini minimi da s a tutti gli altri nodi in G . Ricordiamoci che l'albero dei cammini minimi, per la proprieta' dei cammini minimi, contiene altri cammini minimi possibili, ad esempio per il cammino da 4 a 5, mentre per il cammino da 2 a 5 non possiamo dire niente, perche' non sono collegati da un cammino.

La complessita' dell'algoritmo se il grafo e' implementato come matrice di adiacenza e' $O(|v|^2)$ mentre se e' implementato come liste di adiacenza e' $O(|E| \log |V|)$.

9.7.4 Algoritmo di Bellman-Ford

Lavora con grafi orientati e pesati anche con archi con peso negativo (< 0). Il cuore dell'algoritmo e' simile a quello di Dijkstra.

Come input abbiamo il grado, la funzione peso e la sorgente. Abbiamo sempre all'inizio una fase di inizializzazione, che mi segna ad infinito il peso dei cammini minimi per raggiungere ciascuno dei vertici all'interno del grafo, quindi assumiamo di non sapere niente e di non avere nessun cammino minimo, l'unica distanza che viene posta diversa da infinita e' la distanza della sorgente da se stessa $= 0$, se la distanza della sorgente da se stessa diventa negativa abbiamo trovato un ciclo con peso negativo e dobbiamo fermarci.

L'algoritmo ha un ciclo che non va a guardar ei nodi uno alla volta ma guarda tutti gli archi, per ciascuno controlla se il nodo che risco a raggiungere e percorrendo quell'arco puo' essere raggiunto con un peso minore rispetto al cammino che conoscevo prima. Dato un arco mi chiedo se il cammino che conosco per arrivare a v e' piu' grande del cammino composto per arrivare ad un altro nodo u + l'arco da u a v , se il cammino che arriva a v che avevo ha un valore piu' basso di costo non cambia niente, in caso contrario aggiorno il cammino e la distanza. Questa operazione viene fatta tante volte quanto il numero di vertici all'interno del grafo, l'idea e' vedere se riesco a migliorare il cammino minimo un arco alla volta. Quando ho guardato v volte tutti gli archi ripeto un'altra volta il ciclo e controllo soltanto se ci sono miglioramenti nei pesi, se riesco a migliorare ulteriormente i pesi significa che c'e' un ciclo a peso negativo, quindi devo fermarmi. In caso contrario se nel ciclo finale non cambia niente so che non ci sono cicli con peso negativo e quindi mi fermo contento.

```

BELLMAN-FORD ( $G, w, s$ ) /*  $G$  GRAFO,  $w$  PESO,  $s$  SORGENTE */
  FOR EACH  $v \in G.V$  DO
     $d[v] = \infty$  /* DISTANZA DA  $s$  INIZIALE */
     $v.p = \text{NULL}$ 
   $d[s] = 0$  /* UNICO NODO A DISTANZA NON  $\infty$  ALL'INIZIO */

  FOR  $i = 1$  TO  $|G.V| - 1$  DO /* NUM. NODI - 1 */
    FOR EACH EDGE  $(u, v) \in G.E$  DO /* GUARDA TUTTI GLI ARCHI */
      IF  $d[v] > d[u] + w(u, v)$ 
        THEN  $d[v] = d[u] + w(u, v)$ 
         $v.p = u$ 
      /* PASSANDO DA ARCO  $(u, v)$ 
      TENGENDO IL COSTO
      PER RAGGIUNGERE  $v$  */

  FOR EACH EDGE  $(u, v) \in G.E$  DO
    IF  $d[v] > d[u] + w(u, v)$ 
      THEN RETURN FALSE /* CICLO CON PESO NEGATIVO */

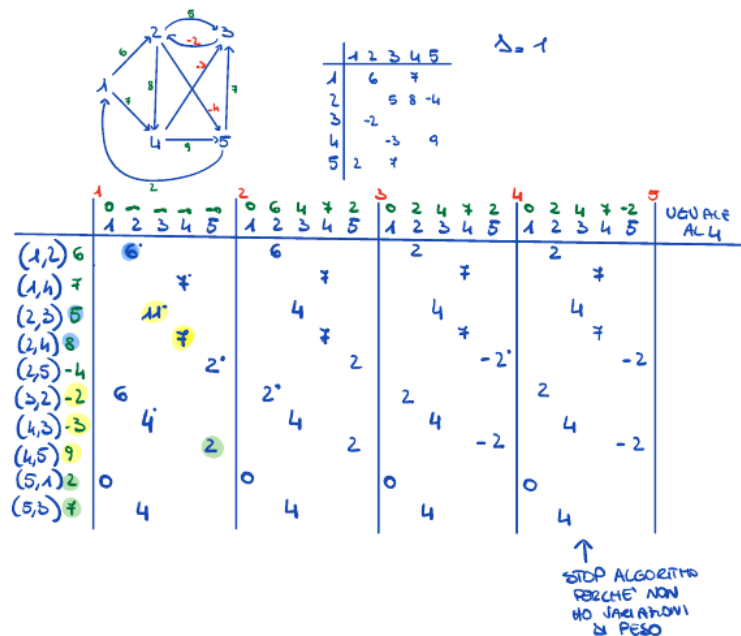
  RETURN TRUE

```

Il costo dell'algoritmo è $O(|V| \cdot |E|)$, quindi costa di più di Dijkstra e viene utilizzato quindi solo quando è necessario, ovvero se non ci sono archi con peso negativo prediligo l'utilizzo di Dijkstra perché è più efficiente.

Vediamo ora un esempio di funzionamento dell'algoritmo: Abbiamo sempre la matrice di adiacenza (se non ce la da disegniamola noi), la sorgente è il nodo 1 $s=1$.

Abbiamo una tabella per svolgere l'esercizio, mettiamo in fila sulle righe ciascuno degli archi che abbiamo sul grafo, per comodità li indichiamo in ordine del nodo di partenza con poi i valori crescenti, quindi ad esempio gli archi che escono da 1 sono gli archi (1,2) e (1,4) e mi segno di fianco i rispettivi pesi, faccio questa cosa per tutti gli archi che escono da tutti i nodi, notiamo che per indicare gli archi basta leggere la matrice di adiacenza riga per riga. Poi di fianco a questa colonna mettiamo tutte le varie iterazioni che dobbiamo fare, segnando per ogni nodo la distanza dalla sorgente ed i genitori. Passando dall'arco 1,2 riesco a raggiungere il nodo 2 pagando meno di quello che sto pagando adesso, l'arco 1,2 dev'essere sommato alla distanza che mi serve per raggiungere il nodo da cui parto (in questo primo caso la sorgente). Poi per tutti gli altri nodi che partono da nodi diversi dalla sorgente devo controllare se il costo che ho per arrivare al nodo più il costo dell'arco che parte da quel nodo (ad esempio 2,3) è minore della distanza che avevo già segnato, se è minore la aggiorno ed aggiorno il genitore per arrivare a quel nodo, altrimenti no. Dato che ci sono state modifiche devo fare un'altra iterazione calcolando ancora i pesi utilizzando i nuovi pesi per arrivare ai nodi.



Chapter 10

Algoritmi di ordinamento

In questa parte del corso non abbiamo piu' nessuna ADT da guardare, ma ci andiamo a focalizzare su un problema utilizzando le strutture dati piu' opportune per poterlo risolvere.

Il problema dell'ordinamento riceve in input una sequenza di valori $a_1, a_2, \dots, a_n$ dello stesso dominio, che dev'essere caratterizzato da una relazione d'ordine. L'output e' una permutazione/ri-ordine della sequenza che mi era stata data come input, tale per cui $a_1^i \leq a_2^i \leq \dots \leq a_n^i$.

Esistono diverse classi di algoritmi e possiamo classificarli secondo due criteri:

- Utilizzo dello spazio di memoria:
 - Algoritmi interni, che lavorano in locale e non gli serve ulteriore spazio, ed a parte di qualche variabile di supporto, utilizzo il vettore in cui sono messi gli elementi per ordinarli.
 - Algoritmi esterni, necessitano di spazio aggiuntivo (altre strutture dati rispetto a quelle utilizzate per memorizzare l'input)
- Complessita' computazionale:
 - Semplici: hanno un costo di $O(N^2)$, sono facili da scrivere ma sono un po' meno efficienti del massimo che possiamo ottenere.
 - Piu' sofisticati: hanno costo $O(n \log n)$.
 - Algoritmi special: possono costare anche $O(n)$, ma non possono essere usati sempre, ma solo se l'input rispetta determinate proprieta', ad esempio se sono valori numerici in un intervallo particolarmente compatto, non utilizzano il confronto tra valori per costruire la sequenza ordinata.

10.1 Bubble Sort

E' un algoritmo semplice (quindi la complessita' e' $O(N^2)$) ed e' un algoritmo interno. L'idea e' di far salire verso l'alto (in cima al vettore, le prime posizioni) gli elementi piu' piccoli, mano mano che l'algoritmo procede. Diciamo verso l'alto perche' immaginiamo il vettore posto in verticale con l'elemento i in alto. E' quindi un algoritmo che scandisce il vettore dal basso verso l'alto e scambia due elementi se il piu' piccolo e' sotto. Quello che ottengo dopo la prima iterazione ho il valore piu' piccolo in assoluto, dopo i scansioni ho i primi i elementi ordinati e dopo $n-1$ scansioni ho il vettore ordinato.

BUBBLE SORT(A)

```

FOR i = 1 TO A.LENGTH - 1 DO /* FA n-1 SCANSIONI DEL VETTORE */
  FOR j = A.LENGTH DOWNTO i+1 DO /* SCANDISCE IL VETTORE DAL
                                  BASSO */
    IF A[j] < A[j-1]
      THEN TEMP = A[j] /* SCAMBIO DI A[j] E A[j-1] */
           A[j] = A[j-1]
           A[j-1] = TEMP

```

Costo computazionale: nel caso peggiore, medio e migliore $O(n^2) \sum_{i=1}^n c(n-i) = O(n^2)$

Vediamo un esempio:

ESEMPIO

	VETTORE	i=1	i=2	i=3	i=4	i=5	RISULTATO
1	60	60 10	10	10	10	10	10
2	20	20 40	20 20	20	20	20	20
3	30	30 20	30 20	30 30	30	30	30
4	10	10 30	30	30 40	40	40	40
5	60	60 50	50	50	50	50	50
6	50	50 60	60	60	60	60	60

Notiamo che anche se non dovessi fare mai scambi nel vettore comunque il costo sarebbe $O(n^2)$ perche' i due cicli sono fatti per arrivare fino in fondo al

vettore. Per fare i confronti partiamo dal basso!

10.2 Insertion Sort

E' un algoritmo semplice con costo computazionale $O(n^2)$, e' interno. Funziona come per ordinare le carte in mano quando se ne pescano di nuove, prendo una carta e la metto nella posizione corretta rispetto a quello che ho in mano.

INSERTION SORT (A)



FOR $j = 2$ TO $A.LENGTH$ DO

$KEY = A[j]$ /* ELEMENTO DA INSERIRE NELLA PARTE ORDINATA
 $A[1 \dots j-1]$ */

$i = j - 1$

WHILE $i > 0$ AND $A[i] > KEY$ DO /* SE KEY E' PIU' PICCOLO
 DELL'ELEMENTO $A[i]$
 NELLA PARTE ORDINATA */

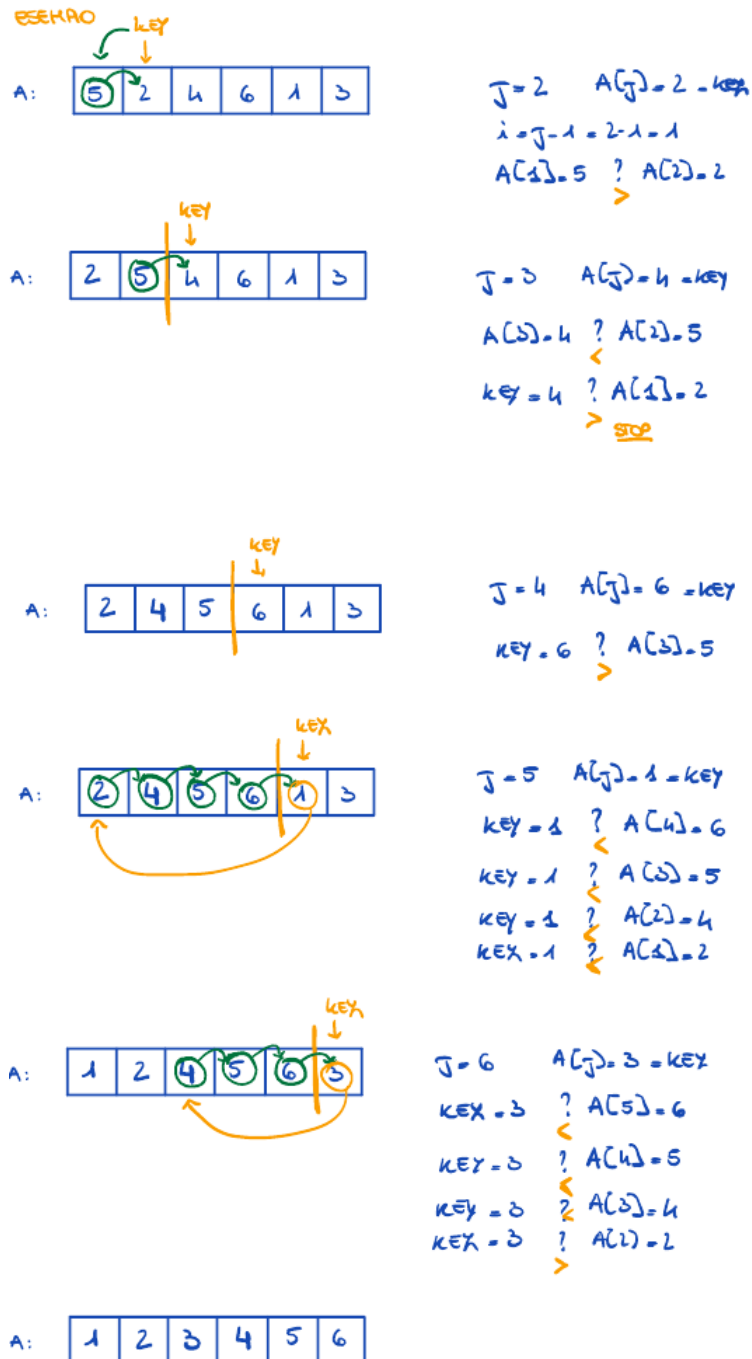
$A[i+1] = A[i]$ /* SPOSTA DI UNO
 A DX $A[i]$ */

$i = i - 1$

$A[i+1] = KEY$ /* INSERISCE L'ELEMENTO NELLO SPATIO VUOTO
 CHE SI E' CREATO */

Riceve in input una sequenza A da ordinare (un vettore di numeri), parte dalla posizione 2 e scorre tutto il vettore, questo primo ciclo for fa finta di pescare le carte una alla volta dal vettore e controlla dove dev'essere inserito, la variabile key mi dice in che posizione devo mettere la carta, il ciclo while dopo parte dalla posizione appena prima rispetto all'elemento che devo andare a controllare e confronta l'elemento del vettore rispetto a quello che sta andando ad inserire nella sottosequenza ordinata e se l'elemento nel vettore e' piu' grande sposta avanti di una posizione la sottosequenza.

Vediamo un esempio del funzionamento:



Essendo un algoritmo semplice ha un costo nel caso peggiore che è $O(n^2)$, dovuto dal fatto che ho due cicli, quello esterno fatto sempre il numero di elementi del vettore, quello interno dipende, dal caso peggiore scorro tutti gli elementi del vettore, ma se l'elemento che devo ordinare è più grande di tutti quelli ordinati il ciclo interno non viene mai eseguito.

10.3 Selection sort

Anch'esso è un algoritmo semplice (quindi con costo $O(n^2)$, anche questo è un algoritmo interno. Ho ancora l'idea di avere la parte iniziale del vettore ordinata, ad ogni passo scelgo l'elemento che va dalla posizione i alla posizione n che è più piccolo e lo porto in posizione i , scambiandolo con l'elemento che c'era in posizione i , così via finché non arrivo in fondo.

SELECTION_SORT(A)

FOR $i = 1$ TO $A.LENGTH - 1$ DO

[LOWINDEX = i ** POSIZIONE ELEMENTO A_i PIU' PICCOLO **
 LOWKEY = $A[i]$ ** ELEMENTO A_i PIU' PICCOLO **

FOR $j = i + 1$ TO $A.LENGTH$ DO ** CERCO IL MINIMO IN $A[i \dots n]$ **

IF $A[j] < LOWKEY$

THEN [LOWKEY = $A[j]$
 LOWINDEX = j

[TEMP = $A[i]$ ** SCAMBIO $A[LOWINDEX]$ CON $A[i]$ **
 $A[i] = A[LOWINDEX]$
 $A[LOWINDEX] = TEMP$

Prende in input un vettore A di n elementi. Abbiamo un primo ciclo che mi serve per fare lo scambio dei valori, la variabile `lowkey` mi tiene traccia dell'elemento più piccolo che ho trovato e `lowindex` mi dice qual'è l'indice dell'elemento. Una volta che esco dal ciclo faccio lo scambio dell'elemento che c'era in i e quello in `lowindex`.

ESEMPIO

	VELORE	i=1	i=2	i=3	i=4	i=5	RISULTATO
1	40	10	10	10	10	10	10
2	20	20	20	20	20	20	20
3	30	30	30	30	30	30	30
4	10	40	40	40	40	40	40
5	60	60	60	60	60	50	50
6	50	50	50	50	50	60	60

Il costo è uguale al bubble sort perché hanno lo stesso numero di cicli, il vantaggio è che fa un numero di scambi nettamente inferiore agli altri, il bubble sort potenzialmente ad ogni visita faceva uno scambio di valori, il selection sort fa un solo scambio per ogni iterazione.

Un'altra caratteristica è che questo algoritmo ha lo stesso costo $O(n^2)$ sia per il caso medio, peggiore e migliore, perché in ogni caso il ciclo for esterno viene eseguito $n-1$ volte ed il ciclo for interno viene eseguito $i-1$ volte.

10.4 Merge Sort

E' un algoritmo complesso, ha un costo di $O(n \log n)$ ed e' un algoritmo esterno. Si basa sul principio del divide et impera, e' un principio con cui vengono progettati degli algoritmi. L'idea e' quella di spezzare il problema generale in tanti sottoproblemi piu' piccoli che sono istanze piu' piccole del problema originale. Alla fine poi quando ho risolto i sottoproblemi ne combino il risultato per ottenere il risultato del problema di partenza. Funzionano bene perche' ad ogni passo della ricorsione diminuisco la dimensione della sequenza, arrivando al caso base che e' velocissimo da ordinare (un elemento solo).

Divide et impera opera in tre fasi:

1. Divide: divido il problema in istanze piu' piccole dello stesso. Nel caso del merge sort prendo la sequenza da ordinare e la divido in due sequenze che contengono la meta' degli elementi. Quindi passo dalla sequenza di n elementi a due sotto sequenze di $\frac{n}{2}$ elementi ciascuno.
2. Risolve/Conquista: risolvo in modo ricorsivo i sottoproblemi fermandomi al caso base che e' semplice da risolvere. Ad esempio nel merge sort il caso base e' la sequenza di 1 elemento, che per definizione e' ordinata.
3. Combinazione: le soluzioni dei sottoproblemi vengono combinate, secondo una logica, per ottenere la soluzione dell'istanza di partenza del problema. Ad esempio nel merge sort si uniscono due sottosequenze ordinate ottenendo un'unica sequenza ordinata.

10.4.1 Caso base

E' quello che mi da la fase di risoluzione e' banale, devo ottenere una sequenza di 1 elemento, quando trovo una sequenza di 1 elemento non devo fare nulla.

10.4.2 Ricombinazione

MERGE, e' il cuore dell'algoritmo. Riceve in input due sottosequenze ordinate che saranno piu' o meno lunghe uguali, l'output e' una sottosequenza ordinata che contiene tutti i valori in input. Funziona tramite lo scorrimento parallelo delle due sottosequenze, le guardo contemporaneamente partendo dalla prima posizione e scrivo nel risultato il valore piu' piccolo che trovo tra le due, quindi ad ogni passo scrivo nel risultato l'elemento di una delle due sottosequenze che e' piu' piccolo, poi porto avanti la sequenza in base a quello che ho scelto.

```

MERGE(A, p, q, r) /* A[p...q] 1° SOTTOSEQUENZA, A[q+1...r] 2° SOTTOSEQUENZA */

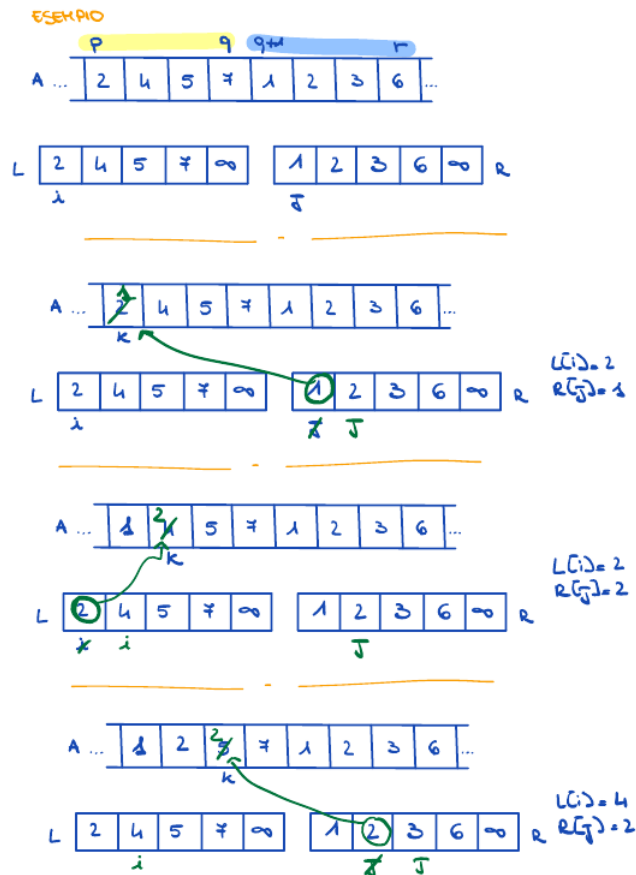
  m1 = q - p + 1 /* LUNGHEZZA 1° SOTTOSEQUENZA */
  m2 = r - q      /* LUNGHEZZA 2° SOTTOSEQUENZA */
  LET L[1...m1+1] AND R[1...m2+1] BE TWO NEW ARRAYS /* O(1) */
  FOR i = 1 TO m1 DO
    L[i] = A[p+i-1]
  FOR j = 1 TO m2 DO
    R[j] = A[q+j]
  /* O(m) = O(m1) + O(m2)  COPIARE A[p...q] IN L E A[q+1...r] IN R */

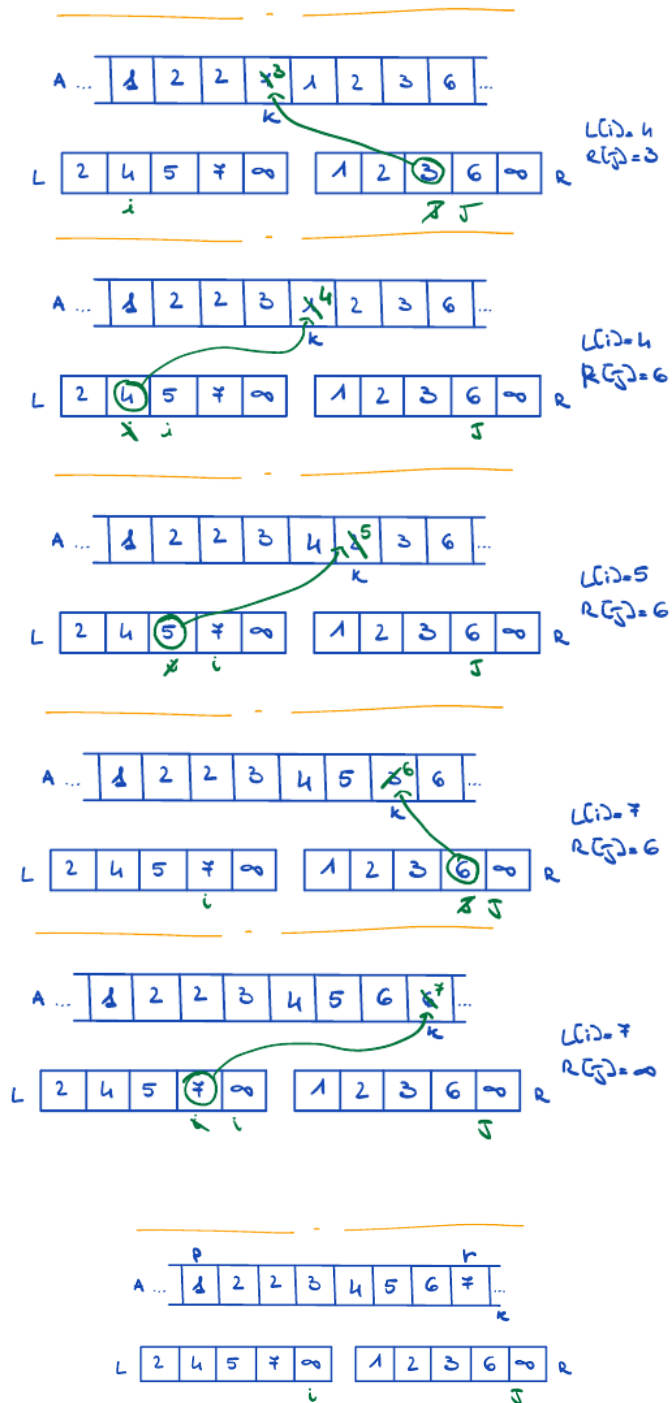
  L[m1+1] = ∞ /* TAPPO FINALE DELLE SOTTOSEQUENZE */
  R[m2+1] = ∞
  i = 1 /* INDICE DI SCORRIMENTO DI L */
  j = 1 /* INDICE DI SCORRIMENTO DI R */

  FOR k = p TO r DO → O(m)
    IF L[i] ≤ R[j]
      THEN A[k] = L[i] /* SE L E' A' PIU' PICCOLO, SCRIVO L NEL RISULTATO E MUOVO AVANTI IL SUO SCORRIMENTO */
      i = i + 1
    ELSE A[k] = R[j] /* SE R E' A' PIU' PICCOLO, SCRIVO R NEL RISULTATO E MUOVO AVANTI IL SUO SCORRIMENTO */
      j = j + 1
  
```

Lo pseudocodice e' complicato dal fatto che ad ogni iterazione sto guardando

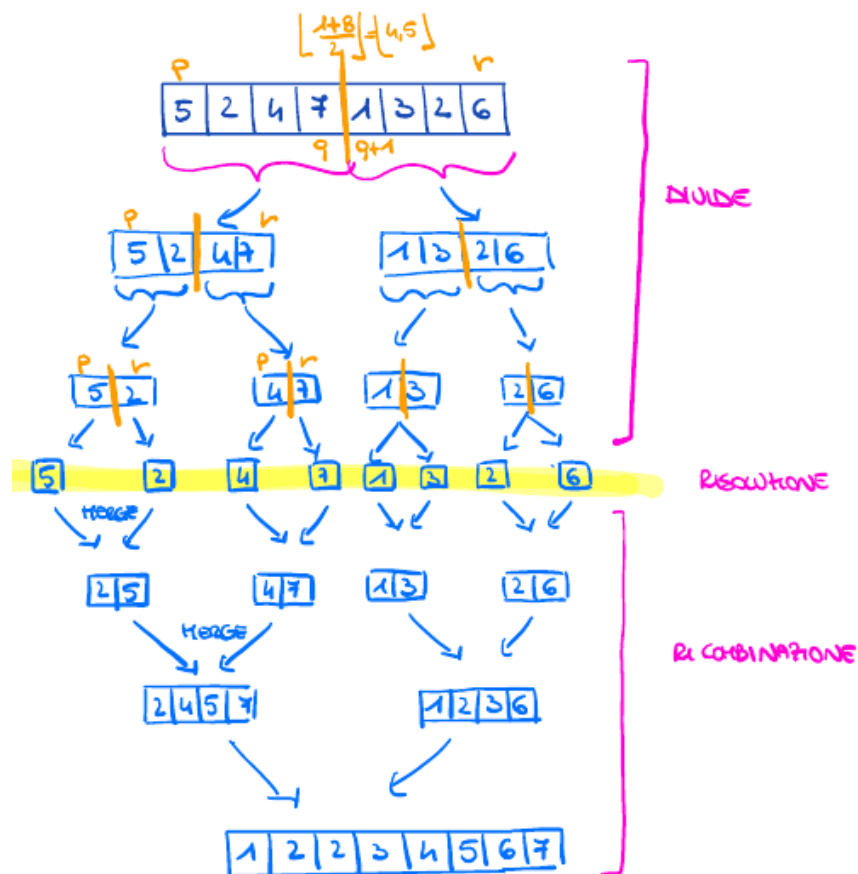
un pezzo di vettore che puo' essere posizionato ovunque. L'input di questa funzione MERGE e' il vettore A (quello che contiene gli elementi della sequenza) e poi tre indici p,q,r, sono i limiti delle due sottosequenze che sto andando a fondere, la prima inizia i p e finisce in q, la seconda comincia da q+1 e finisce in r. Prima individua la lunghezza delle due sottosequenze (n_1 ed n_2), utilizza due vettori ausiliari (spazio esterno) L ed R, dove copia la prima e la seconda sottosequenza rispettivamente (i primi due for) perche' se non mi tenessi una copia delle sottosequenze le andrei a perdere. L e R hanno un elemento in piu' che setto a piu' infinito perche' so che quando arrivo in fondo posso copiare tutto quello che e' rimasto nell'altro vettore ausiliario in A, perche' e' gia' ordinato e qualsiasi valore trovo nell'altro vettore sara' sempre minore di infinito. Successivamente faccio lo scorrimento parallelo, se $R[j]$ e' piu' piccolo di $L[i]$ scrivo $L[i]$, altrimenti $R[j]$ e porto avanti il contatore del vettore da cui ho scritto l'elemento in A.





$\text{MERGESORT}(A, p, r)$ /* A SEQUENZA p : INIZIO r : FINE */
 $O(1)$ IF $p < r$ /* CASO BASE QUANDO $p = r \Rightarrow$ SEQUENZA DI 1 ELEMENTO */
 $O(1)$ THEN $q = \lfloor (p+r)/2 \rfloor$ /* SPETTA A META' LA SEQUENZA */
 $\left[\begin{array}{l} \text{MERGESORT}(A, p, q) \\ \text{MERGESORT}(A, q+1, r) \end{array} \right] 2T(m/2)$
 $\text{MERGE}(A, p, q, r) O(m)$

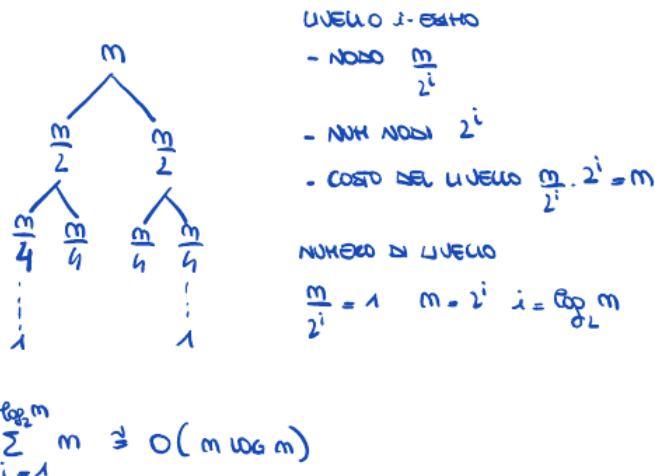
L'if e' il caso base, quando e' piu' grande trovo il punto medio della sequenza e chiamo il merge sort sulle due sottosequenza per poi ricombinare il risultato con il merge. Esempio di iterazioni:



10.4.3 Costo

Dato che e' un algoritmo ricorsivo abbiamo bisogno delle equazioni di ricorrenza.

$$T(m) = 2T(m/2) + O(m)$$



10.5 Heap Sort

E' un algoritmo complesso ed implica il costo di $O(n \log n)$, e' un algoritmo interno quindi non serve altro spazio oltre quello per memorizzare l'input. La caratteristica principale e' che utilizza una particolare struttura dati heap, che e' una struttura dati generica che si puo' utilizzare per implementare anche altri tipi di algoritmi e strutture.

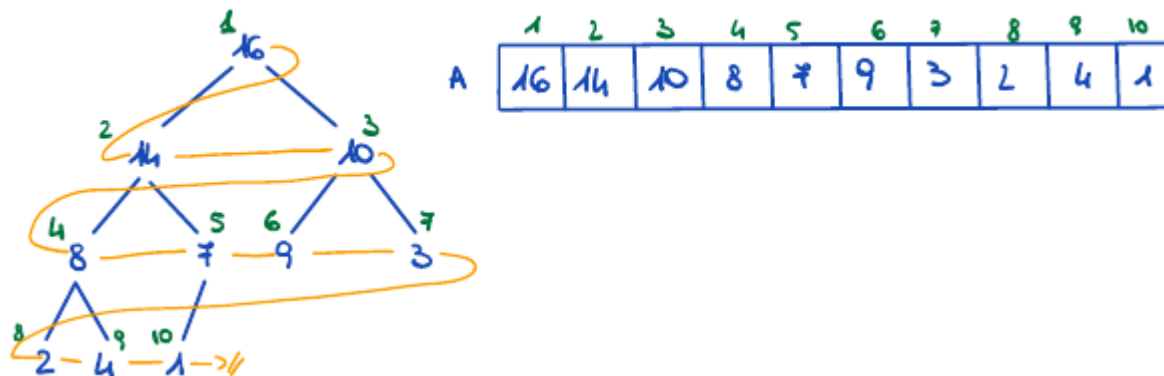
10.5.1 Heap

E' un particolare tipo di albero binario parzialmente ordinato ed e' quasi completo, ovvero che tutti i livelli dell'albero sono completi ad eccezione dell'ultimo che viene riempito partendo da sinistra ed andando a destra. La caratteristica dell'ordinamento di distingue per:

- Max-heap: il valore del nodo e' sempre piu' piccolo rispetto a quello del padre. In alto i valori grandi ed in basso i valori piccoli. Quindi mi aspetto di trovare il valore piu' grande di tutti nella radice.
- Mini-heap: il valore del nodo e' sempre piu' grande rispetto al valore del nodo padre. In alto i valori piccoli ed in basso i valori grandi. Nella radice mi aspetto di trovare il valore minimo tra tutti quelli che ci sono nella struttura dati.

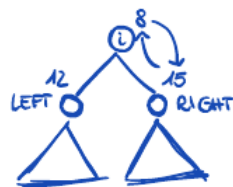
Rispetto al fratello di un nodo non posso dire niente, possono anche essere in ordine, ma non e' una caratteristica determinante per lo heap.

E' implementato tramite un array $A[1...N]$ dove ho una cella per ciascuno dei nodi nell'albero, la posizione $A[1]$ e' occupata dalla radice e per ogni nodo quello in posizione i ha il padre nella posizione $\frac{i}{2}$, arrotondato per difetto. Il figlio sinistro si trova in posizione $2i$ ed il figlio destro si trova in posizione $2i+1$.



Leggo l'albero secondo il percorso arancione, partendo dall'alto e leggendo ogni grado dell'albero da sinistra verso destra. Il vantaggio dell'utilizzo dello heap e' che l'altezza essendo un albero praticamente completo e' un albero basso, quindi il costo e' $\Theta(\log n)$ e questo implica il costo della maggior parte delle operazioni essere $\Theta(\log n)$ (n e' il numero dei nodi nell'albero).

Immaginiamo di avere due alberi che sono entrambi max-heap e di avere un elemento qualsiasi che mettiamo come radice a questi due alberi, vediamo come mettere questo genitore rispetto ancora la forma del max-heap:



$A[i] > \text{LEFT}[i]$
 $A[i] > \text{RIGHT}[i]$

SE NON VALE
 ↓
 SCAMBIO $A[i]$
 CON IL SUO FIGLIO
 PIÙ GRANDE
 ↓
 CONTINUO IN MODO
 RICORSIVO FINO A CHE
 LA PROPRIETÀ NON È
 SODDISFATTA O ARRIVO
 ALLA FOGLIA

Facendo ricorsivamente questa cosa partendo dalle foglie ed arrivando alla radice ho trovato un max-heap. Quello che faccio e' controllare la relazione padre figlio di ogni figlio con il nodo, se noto che la radice dell'albero che abbiamo

inserito e' piu' piccola di un suo figlio la scambio con il suo figlio piu' grande, faccio questa cosa ricorsivamente perche' quando scambio potrebbe essere che tutti quello che stava nel sottoalbero che ha cambiato radice non rispetta piu' la proprieta' del max-heap.

$O(\log n)$

$\text{MAX-HEAPIFY}(A, i)$ /* A VETTORE CHE RAPPRESENTA IL HEAP, $A[i]$ E' L'ELEMENTO DA SISTEMARE */

$O(1)$ $\left[\begin{array}{l} p = \text{LEFT}(i) \text{ /* } p = 2i \text{ */} \\ r = \text{RIGHT}(i) \text{ /* } r = 2i + 1 \text{ */} \end{array} \right.$

$O(1)$ $\left[\begin{array}{l} \text{IF } p \leq A.\text{HEAP_SIZE AND } A[p] > A[i] \text{ /* INDIVIDUA FRA } A[i], A[p], A[r] \text{ IL VALORE PIU' GRANDE */} \\ \quad \text{THEN } \text{LARGEST} = p \\ \text{ELSE } \text{LARGEST} = i \\ \text{IF } r \leq A.\text{HEAP_SIZE AND } A[r] > A[\text{LARGEST}] \\ \quad \text{THEN } \text{LARGEST} = r \end{array} \right.$

$\text{IF } \text{LARGEST} \neq i$

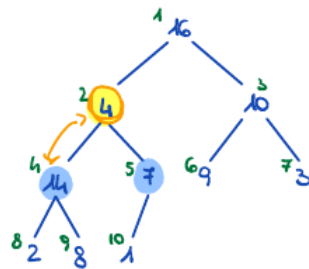
$\text{THEN } \left[\begin{array}{l} \text{TEMP} = A[i] \\ \text{/* } O(1) \text{ */} \\ A[i] = A[\text{LARGEST}] \\ A[\text{LARGEST}] = \text{TEMP} \end{array} \right] \text{ /* SOSTITUISCO } A[i] \text{ CON IL PIU' GRANDE DEI SUOI FIGLI */}$

$\left[\begin{array}{l} \text{MAX-HEAPIFY}(A, \text{LARGEST}) \text{ /* CHIAMATA RICORSIVA PER GARANTIRE CORRETTA */} \\ \text{CHIAMATA AL MASSIMO } (\log n) \text{ VOLTE} \end{array} \right.$

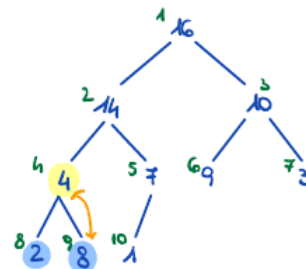
La funzione che fa questa cosa si chiama Maxheapify, prende in input un vettore A che e' l'insieme di valori e la posizione i del valore da sistemare. Innanzitutto controllo di non essere a livello della foglia (perche' chiaramente una foglia non ha figli) e non rispetta la proprieta' del max heap tengo in una variabile il valore piu' grande tra il figlio sinistro e destro del valore i, altrimenti e' il valore della posizione i-esima. Se il figlio di destra esiste e se la posizione di destra e' piu' grande della largest ne tengo traccia. A questo punto faccio in modo di sistemare la struttura scambiando $A[i]$ con il piu' grande dei figli, se ho fatto questo scambio chiamo ricorsivamente la funzione per capire quanto deve scendere il nodo. Notiamo che la chiamata ricorsiva avviene solo quando faccio lo scambio, se il valore $A[i]$ e' gia' piu' grande dei suoi figli lascio tutto com'e' perche' e' gia' un max-heap. Il costo di tutte le operazioni e' $O(1)$, tranne la chiamata ricorsiva, che posso chiamare nel caso peggiore l'altezza dell'albero (come se partissi dalla radice fino al caso base la foglia), dato che l'altezza dell'albero heap e' $\log n$, quindi il costo totale e' $O(\log n)$.

Vediamo un esempio:

ESEMPLO = MAX-HEAPIFY(A, 2)



$l = 2 \cdot 2 = 4 \Rightarrow A[4] = 14$
 $r = 2 \cdot 2 + 1 = 5 \Rightarrow A[5] = 7$
 $\text{MAX}(4, 14, 7) = 14$
 SCAMBIO 4 CON 14



$l = 4 \cdot 2 = 8 \Rightarrow A[8] = 2$
 $r = 4 \cdot 2 + 1 = 9 \Rightarrow A[9] = 8$
 $\text{MAX}(4, 2, 8) = 8$

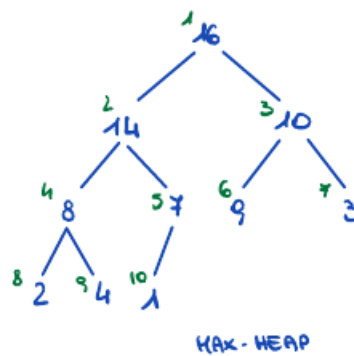
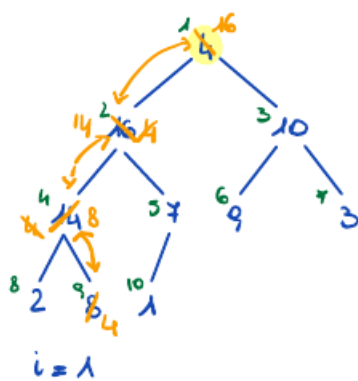
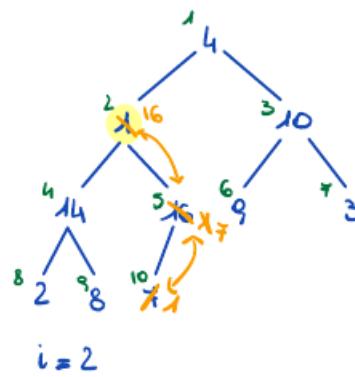
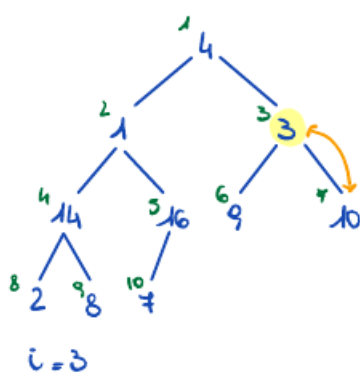
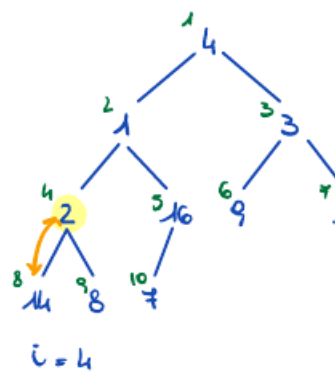
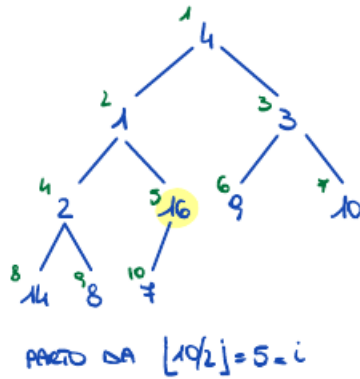
Per costruire un max-heap dato un vettore che contiene una serie di elementi che vogliamo mettere a posto uso la max heapify sui nodi interi di A partendo dal basso e salendo livello per livello da DX verso SX. Notiamo che partiamo dalla posizione $\frac{n}{2}$ chiamando la maxheapify (perché è il nodo più piccolo che ha un figlio), fino alla posizione 1.

BUILD-MAX-HEAP(A) $O(n)$
 A.HEAP-SIZE = A.LENGTH
 FOR $i = \lfloor A.LENGTH/2 \rfloor$ DOWNTO 1 DO
 MAX-HEAPIFY(A, i)

Nel caso peggiore la chiamo n volte (dove n è il numero dei nodi diviso due) per il costo della heapify. Quindi $O(n)$ volte pago $O(\log n)$ quindi il costo preciso sarebbe $O(n \log n)$. Ma nella realtà costa $O(n)$.

Vediamo un esempio:

ESEMPIO -



L'heap sort sfrutta il fatto che il valore massimo dell'heap sta nella radice, quindi posso prendere la radice e metterlo nell'ultima posizione e riordinare l'albero, facendo questa cosa ordino la sequenza.

10.5.2 Heap Sort

Dato che $A[1]$ e' il valore massimo de A (se A e' un max-heap) scambio $A[1]$ con $A[n]$, quindi $A[n]$ e' l'ultimo valore nella sequenza ordinata e lo tolgo dall'albero. Quindi ricostruisco il max-heap senza il nodo che ho tolto e opero in modo ricorsivo fino a quando non arrivo ad uno heap vuoto.

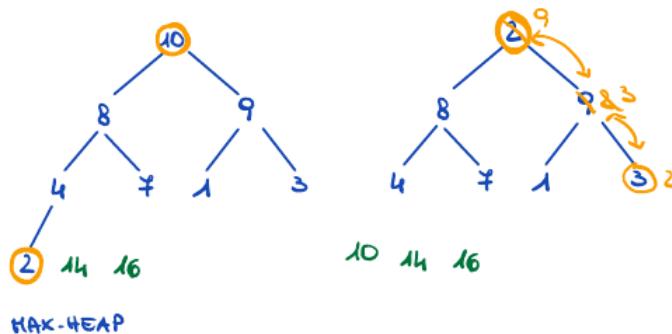
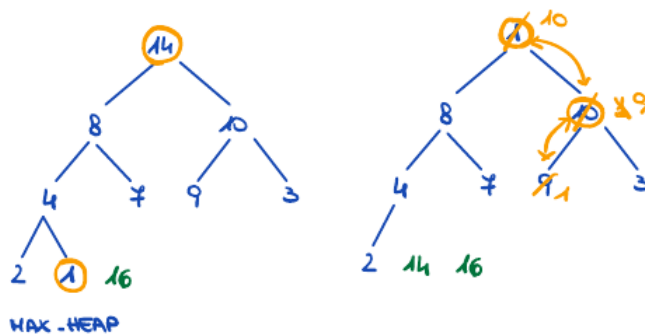
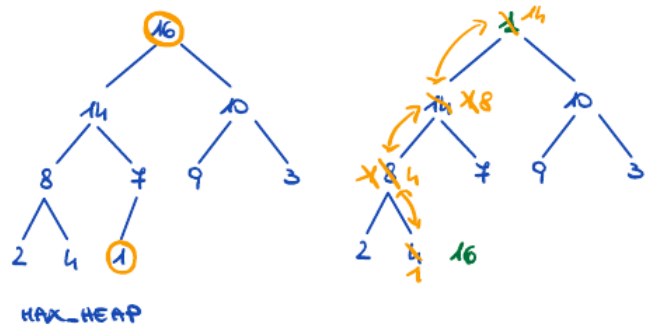
```

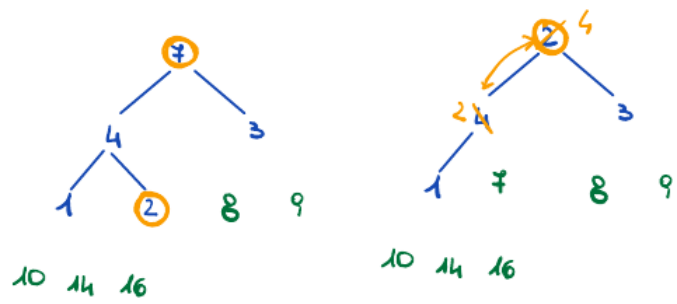
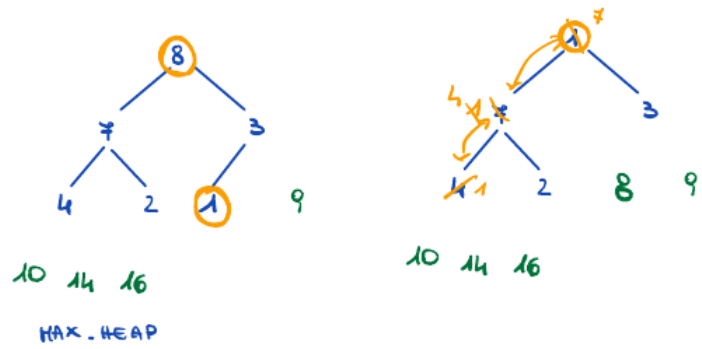
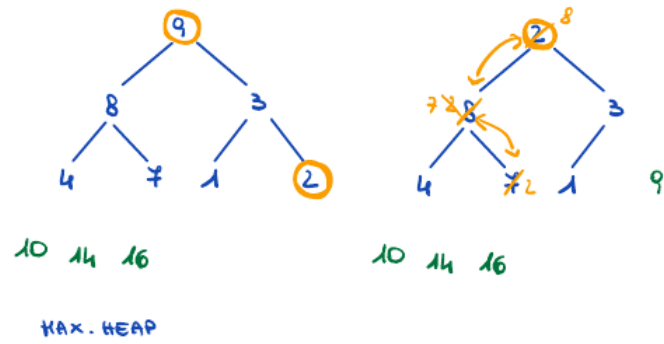
HEAP_SORT(A)            $O(m) + O((m-1) \cdot \log m) = O(m \log m)$ 
                         $\int O(m)$ 
BUILD_MAX_HEAP(A) /* COSTRUIRE LO HEAP SUIA SEQUENZA INIZIALE */
FOR i = A.LENGTH DOWNTO 2 DO    m-1 CHIAMATE
    TEMP = A[i]
    A[i] = A[1]
    A[1] = TEMP                /* METTE L'ELEMENTO MAX NELLA POSIZIONE CORRETTA */
    A.HEAP_SIZE = A.HEAP_SIZE - 1 /* DIMINUISCE DI 1 LA DIMENSIONE
                                DELO HEAP */
     $O(\log m)$  MAX_HEAPIFY(A, 1) /* RICOSTRUISCO LO HEAP */

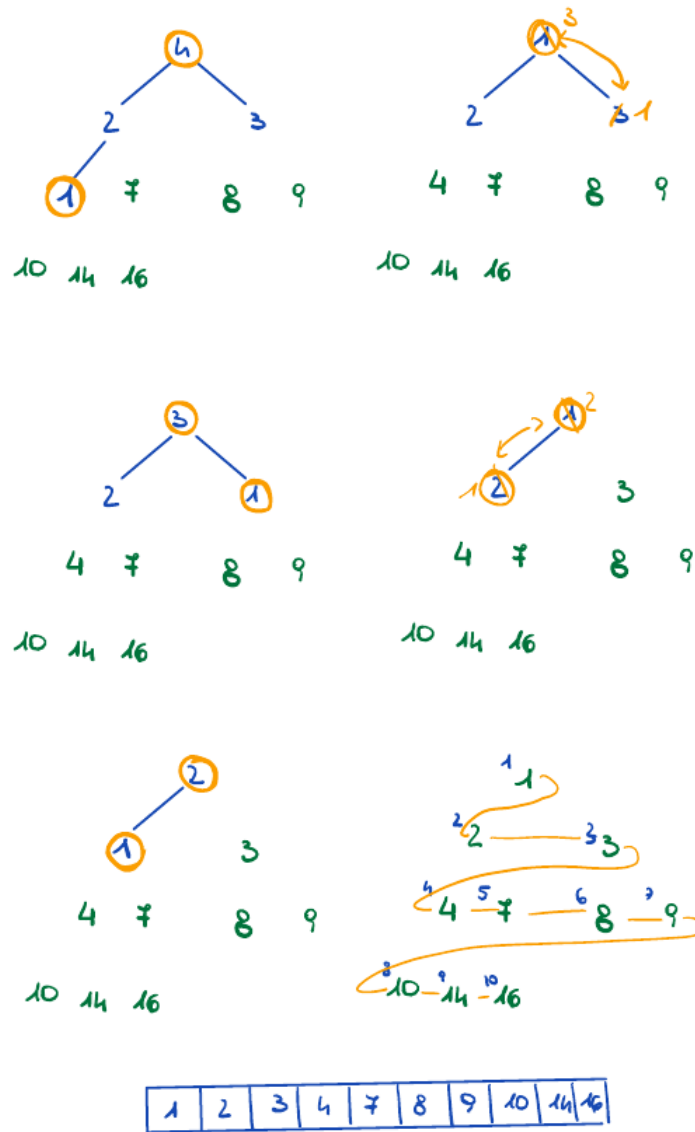
```

Vediamo un esempio: Prima costruiamo un albero partendo dalla sequenza, rendiamo questo albero un maxheap tramite la maxheapify. A questo punto scambiamo la radice con l'elemento in ultima posizione, senza però considerare l'elemento in ultima posizione parte dell'albero, ora ricostruisco il maxheap scambiando opportunamente i valori. Successivamente sposto la radice nell'ultima posizione libera della sequenza (come fatto al primo passo) staccandola dall'albero e ricostruisco il maxheap. Vado avanti finché l'albero è vuoto.

ESEMPIO - 16, 14, 10, 8, 7, 9, 3, 2, 4, 1







Come nota diciamo che lo heap puo' essere usato anche per implementare in modo efficiente le code di priorita' (scheduling processi Windows).

10.6 Quick Sort

E' un algoritmo complesso ed ha una complessita' $O(n \log n)$ nel caso medio ma $O(n^2)$ nel caso peggiore. E' un algoritmo interno e segue la logica del divide et impera:

- Dividere (la parte importante dell'algoritmo): sceglie un elemento $A[q]$ detto PIVOT e partiziona la sotto sequenza nel vettore $A[p...r]$ in due sottosequenze $A[p...q-1]$ con tutti i valori piu' piccoli del pivot e $A[q+1...r]$ con tutti i valori piu' grandi del pivot. (nella divisione nelle due sottosequenze non sto ad ordinare ogni volta).
- Risolvere: ordino i sottovettori in modo ricorsivo, il caso base e' una sequenza con un solo elemento che non puo' essere ulteriormente spezzata ed e' gia' ordinata.
- Combinazione: dato che i sottovettori sono ordinati ed il primo ha solo valori minori del pivot ed il secondo ha solo valori maggiori del pivot basta concatenarli.

```

QUICKSORT(A, p, r) /* A e' il vettore A[p..r] e' il sottovettore che
                    sto ordinando */
IF p < r
  THEN q = PARTITION(A, p, r) /* q e' la posizione del valore pivot */
       QUICKSORT(A, p, q-1)
       QUICKSORT(A, q+1, r)

```

Riceve in input un vettore A e gli estremi della sottosequenza con cui vado ad operare (p ed r). Se p e' piu' piccolo di r allora chiamo una funzione di partizionamento che mi restituisce un valore q che e' il punto di snodo del pivot e chiamo ricorsivamente i il quicksort nelle due sottosequenze.

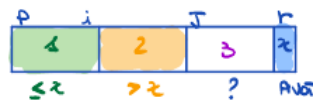
10.6.1 Partition

```

PARTITION(A, p, r)    O(m)
O(1) [ z = A[r] /* z e' IL VALORE PIVOT */
      i = p - 1 /* TUTTI I VALORI IN A[p...i] SONO PIU' PICCOLI DEL PIVOT */
      FOR j = p TO r - 1 DO /* TUTTI I VALORI IN A[j...r] NON SONO STATI CONTRIBUITI */
                              /* IN A[i+1...j] SONO MAGGIORI DEL PIVOT */
      O(m) [ IF A[j] ≤ z
              THEN i = i + 1 /* SCAMBIO A[j] CON IL PRIMO VALORE > PIVOT
                              /* IN POS. A[i+1] */
              TEMP = A[i]
              A[i] = A[j]
              A[j] = TEMP
      O(1) [ TEMP = A[i+1] /* METTERE IL PIVOT FRA I DUE SOTTO VETTORI
            A[i+1] = A[r]
            A[r] = TEMP
            RETURN i + 1 /* POSIZIONE DEL PIVOT */
  
```

Ho il pezzo di vettore che va da p ad r, tipicamente quello che si fa e' scegliere l'ultimo elemento del vettore come pivot, la funzione partition considera il sottovettore in 3 parti:

- 1: dove va a mettere i valori che sono minori o uguali al pivot, e sono tutte le posizioni che vanno dalla posizione
- 2: contiene solo i valori piu' grandi del pivot. Quindi se ho un valore piu' piccolo del pivot si trovara' tra la posizione i+1 e j-1
- 3: contiene gli elementi di cui non so ancora niente
- 4: il pivot

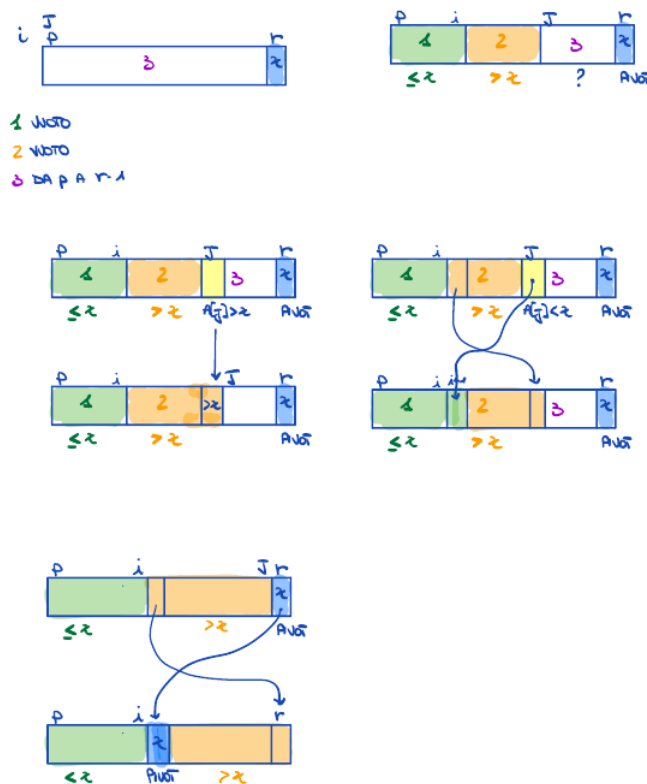


Quindi dicevamo che la funzione partition lavora sul vettore A ed i due limiti p ed r. Inizialmente la parte 1, 2 sono vuote, la parte 3 contiene tutto il vettore tranne il pivot che e' nella posizione 4. L'algoritmo utilizza due variabili di

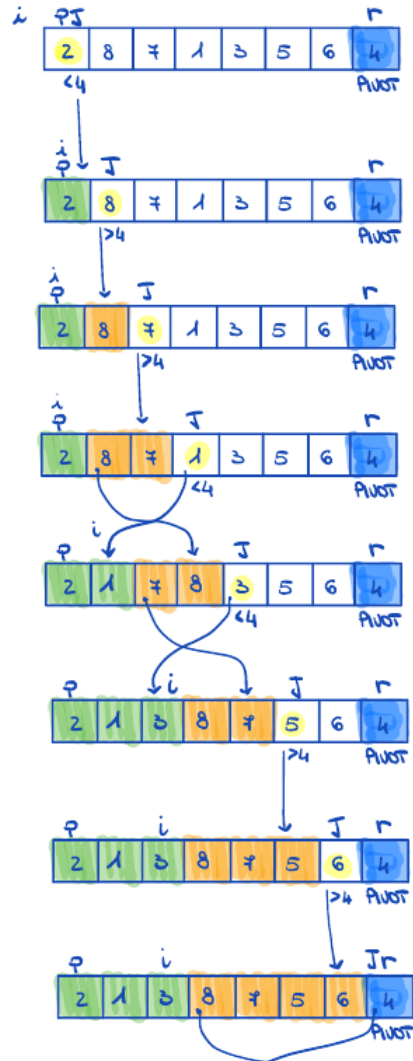
supporto i e j che fanno da spartiacque tra le posizioni. La i all'inizio viene messa fuori dal vettore e la parte j all'inizio perché è tutto vuoto. Il ciclo `for` controlla l'elemento in posizione j e lo confronta rispetto ad x , se l'elemento in posizione j è più piccolo di x è verde, per fare aumentare la parte verde devo aumentare la i , quindi scambia l'elemento in posizione $i+1$ con la j , successivamente aumento la i di uno ed anche la j . Se invece j è più grande non fa niente e la j viene incrementata di uno, aumenta quindi la parte arancio in automatico. Quando si arriva alla fine dell'esecuzione dell'algoritmo la j sarà uguale ad $r-1$, dato che ho diviso il vettore in una prima parte che contiene i valori minori di x ed una seconda parte con i valori maggiori di x ed il pivot. A questo punto devo mettere la x tra le due parti, quindi a questo punto scambio l'elemento in $i+1$ con la x (perché spendo molto meno rispetto a fare lo shift).

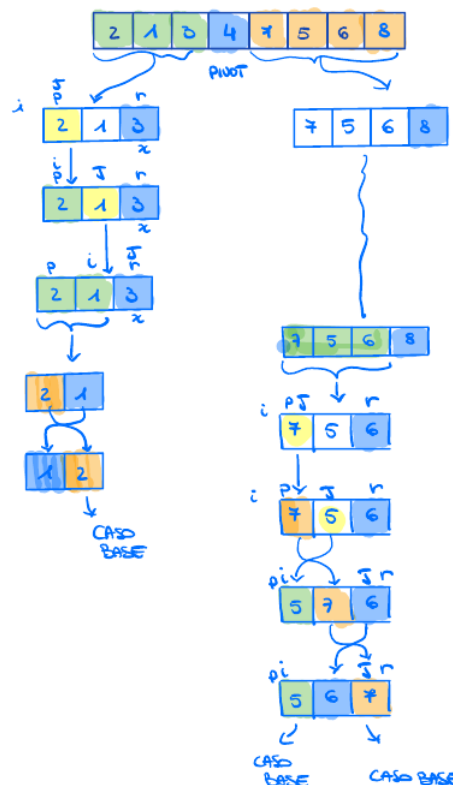
Due cose importanti:

- Se guardiamo un elemento ed è più grande lo lasciamo com'è, altrimenti lo scambiamo con il primo dei più grandi.
- Il pivot è l'ultimo elemento della sequenza



ESEMPIO





10.6.2 Costo

Notiamo che la scelta del PIVOT è fondamentale, perchè se scelgo sempre l'elemento più grande (o più piccolo) non sto spezzando un bel niente, sto solo ordinando un elemento, quindi il costo computazionale sale di molto. La dimensione dei sottoproblemi non è definita a priori essere la metà del problema, ma potrebbe andare male e divido il problema in una sottosequenza lunga $n-1$ ed in un'altra lunga 0.

Partiamo dall'idea che la funzione partition è una funzione che costa $O(n)$, l'algoritmo invece dipende dalle chiamate ricorsive. Nel caso medio ha uno split del vettore in due sottovettori (bilanciati) di dimensioni circa $\frac{n}{2}$ e pago $O(n \log n)$. Nel caso peggiore non faccio nessuno split, perchè l'unico elemento che metto a posto è il PIVOT, metto quindi $n-1$ elementi in un sotto vettore e 0 elementi nell'altro, in questo caso quindi vado a pagare $O(n^2)$.

Caso peggiore

Partizionamento sbilanciato è come dire che ogni volta chiamo il quick sort su un problema di $n-1$ elementi (quando il pivot è l'elemento più grosso):

$$T(n) = T(n-1) + \cancel{T(0)} + O(n) = T(n-1) + O(n) = O(n^2)$$

Caso migliore

Ho un partizionamento "perfetto", quindi ho due sotto sequenze di $\frac{n}{2}$ ed $\frac{n}{2}$:

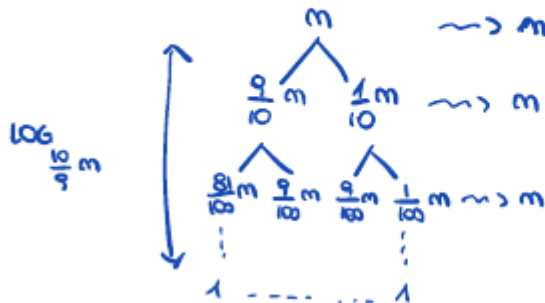
$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) = O(n \log n)$$

$$T(m) = 2T\left(\frac{m}{2}\right) + O(m) = O(m \log m)$$

**Caso medio**

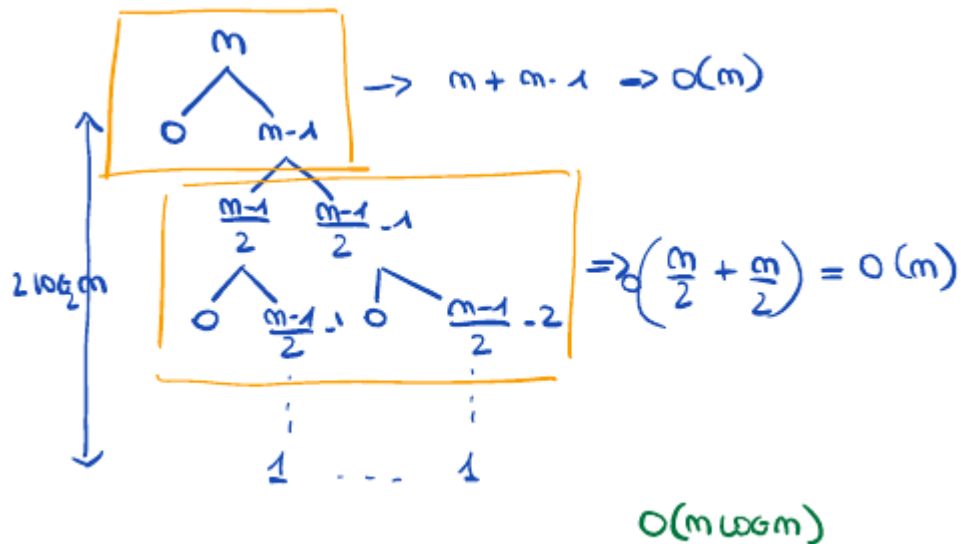
Immaginiamo una situazione in cui tutti gli split dividono la sequenza al 90% da una parte e al 10% dall'altra (rapporto 9 a 1):

$$T(m) = T\left(\frac{9m}{10}\right) + T\left(\frac{m}{10}\right) + O(m) = O(m \log_{\frac{10}{9}} m) = O(m \log m)$$



Il ramo più lungo dell'albero è quello dove divido per 9, quindi per calcolare l'altezza devo calcolare $\left(\frac{9}{10}\right)^i n = 1$, varrà 1 quando $i = \log_{\frac{10}{9}} n$, il numero dei livelli ed il costo, dato che le costanti non ci interessano è sempre $O(n \log n)$.

Ora vediamo cosa succede se alterniamo uno split bilanciato ($\frac{n}{2}e\frac{n}{2}$) con uno split sbilanciato (caso peggiore, $n - 1$ e 0).



Posso vedere i nodi come raggruppati su due livelli alla volta, quindi una chiamata buona ed una cattiva le posso riassumere insieme come se fosse un unico nodo che costa $n - n - 1$, quindi alla fine è sempre dell'ordine di $O(n)$. L'idea è che mi servono $2 \log_2 n$ passi per arrivare alle foglie dell'albero.

Abbiamo quindi dimostrato che nel caso medio si comporta come nel caso migliore $O(n \log n)$, cambiano solo le costanti, ma per il livello di analisi che facciamo noi va bene.

La scelta del PIVOT è importante, scegliere il pivot in modo deterministico (sempre in ultima posizione) in realtà non va tanto bene quindi, è molto meglio scegliere il pivot in una posizione casuale, l'idea è:

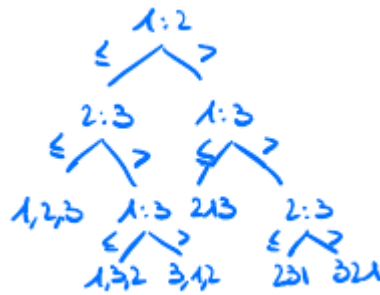
- Scelgo un numero casuale tra x fra 1 ed m
- Scambio $A[n]$ con $A[x]$
- Eseguo il Quick Sort

10.6.3 Nota

In tutti gli algoritmi che abbiamo visto fino ad ora ad un certo punto abbiamo sempre un'istruzione di CONFRONTO tra due valori all'interno della sequenza. Quindi tutti gli algoritmi che abbiamo visto fino ad adesso si chiamano algoritmo basati sul confronto. Tutti questi algoritmi hanno un costo computazionale di almeno $O(n \log n)$, perchè è il minimo numero di confronti che devo fare tra gli elementi per determinarne l'ordine in una sequenza. Negli algoritmi semplici

confronto sempre tutte le coppie ($O(n^2)$), mentre in quelli complessi sfrutto i confronti che avevo fatto prima ($O(n \log n)$).

Ci sono algoritmi di ordinamento che costano meno, ma non si basano sull'ordinamento, sono gli algoritmi speciali.



10.7 Counting Sort

E' un algoritmo di ordinamento speciale, non basato sui confronti, esterno perchè utilizza delle strutture dati ausiliarie (devi vettori, per mettere i risultati che ottiene man mano). Il costo che ha è circa lineare nella dimensione dell'input.

E' usato per ordinare una sequenza di valori compresi tra 0 a k, devono essere in un intervallo ristretto entro cui sono compresi i valori che stiamo andando ad ordinare, altrimenti se non c'è l'intervallo non si ha il vantaggio del costo lineare. Ordina una sequenza di n elementi(valori) fra 0 e k. Il ragionamento è che sappiamo quanti valori sono più piccoli del valore che sto andando a posizionare, così lo inserisco nella posizione giusta senza fare confronti. Uso un vettore ausiliario come contatore, per ogni indice c'è scritto il numero di valori più piccoli dell'elemento in indice nella sequenza. Determina, per ogni elemento x nella sequenza A, quanti sono i valori più piccoli di x in A, questo determina la posizione di x nella sequenza ordinata.

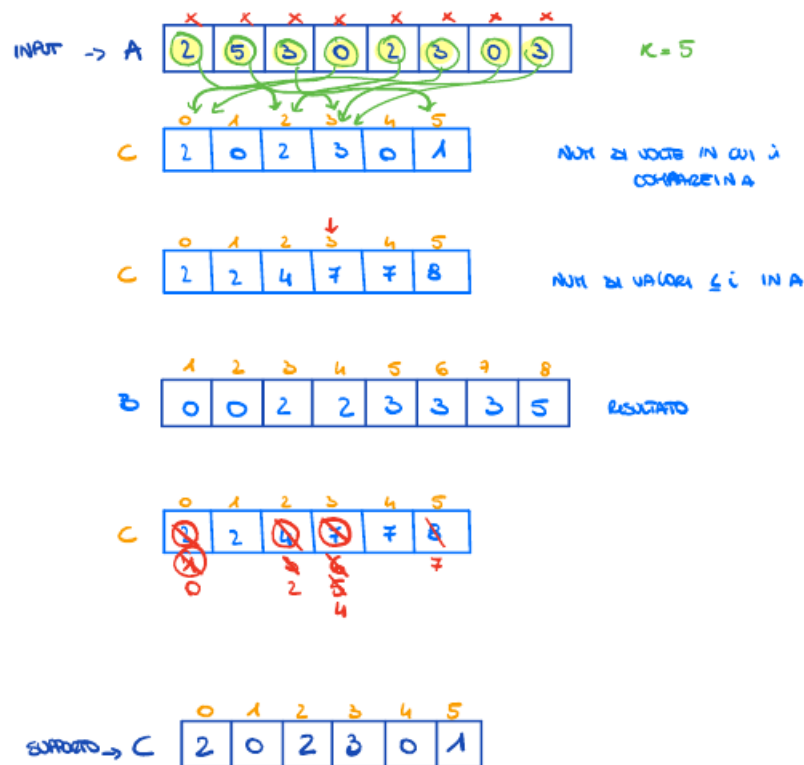
```

COUNTING.SORT(A,B,k) /* A INPUT, B OUTPUT, k VALORE MASSIMO CHE HO IN A */
O(k) LET C[0...k] BE A NEW ARRAY /* CONTATORI */
O(k) FOR i = 0 TO k DO C[i] = 0 /* PONE CONTATORI A 0 */
O(m) {
  FOR j = 1 TO A.LENGTH DO /* CONTA PER OGNI j = 1..k IL NUMERO
    DI VOLTE IN CUI COMPARE IN A */
    C[A[j]] = C[A[j]] + 1
}
O(k) {
  FOR i = 1 TO k DO /* C CONTIENE IL NUMERO DI ELEMENTI < I PER OGNI C[i] */
    C[i] = C[i] + C[i-1]
}
O(m) {
  FOR q = A.LENGTH DOWNTO 1 DO /* SCORRO A E PER OGNI VALORE V
    LO METTO IN B SECONDO LA POSIZIONE
    DETTATA DA C[V] */
    B[C[A[q]]] = A[q]
    C[A[q]] = C[A[q]] - 1
}

costo: O(k) + O(m)    SE k = O(m) ALLORA COSTO O(m)
  
```

Riceve in input un vettore A (che contiene i valori da ordinare), c'è un vettore B che è l'output, k è il valore massimo che posso trovare all'interno di A. Come prima cosa definisco un terzo vettore C che tiene traccia dei conteggi, è un vettore di contatori. All'inizio A è popolato C tutti zeri e B è vuoto. Ora devo fare diverse passate dei vettori, all'inizio conto per ogni valore il numero di volte che il valore corrispondente all'indice del vettore C compare in A, quindi avrò alla fine un vettore da 0 a k e per ogni indice mi dice quante volte compare in A. Scorro il vettore A e per ogni valore che trovo incremento il vettore C nella posizione corrispondente che sto guardando in A (il valore nel vettore A mi fa da indice nel vettore C dove sistemo i contatori). Il prossimo step è trovare i

numeri che precedono il numero dato, quindi scorro il vettore C e faccio delle somme parziali, l'elemento in posizione 1 ha il valore di quello che c'è scritto in 0 ed 1. Quindi finito il secondo ciclo in C so quanti sono i valori più piccoli di i in C . L'ultimo ciclo va a costruire il risultato, quindi riscontro il vettore A e guardo in C dove va messo quel valore, quando ho messo a posto il primo valore dell'indice decremento il contatore, in modo che se dopo dovessi ritrovare lo stesso valore in A non vado a metterlo sempre nella stessa posizione.



Per scandire A parto dall'ultimo elemento, perchè è più facile gestire i duplicati. Lo guardiamo più a scopo didattico, perchè nella pratica le situazioni in cui potremmo utilizzarlo sono molto particolari. Lo utilizziamo solo quando il valore del range è proporzionale alla lunghezza della sequenza.

Il costo è $O(k) + O(n)$ (n è la lunghezza della sequenza A). Succede che se $K = O(n)$ allora il costo è $O(n)$.